MINI PROJECT / CHAPTER 4

Efficient Monitoring of Website Activity Using Data Stream Algorithms

An interactive, responsive web application and reproducible experiment on real NASA HTTP access logs

Student	Course	Submission
Mohammed Khalifah 481013386	Big Data Mining CSD 5113	Assignment 2 Semester I, 2026-2027

Project overview

StreamLab is the project's browser interface. Playback controls, a live request feed, charts and algorithm-state views make the stream processing visible. The prepared offline HTML opens directly in a browser; the modular version runs with a small standard-library Python static server.

This project replays 1,891,714 genuine HTTP requests as a sequential stream. Three independently implemented summaries answer different questions: whether a host has appeared before, how many different hosts have appeared, and how many recent requests produced HTTP errors. Exact baselines make the approximation costs visible.

The source contains 81,982 distinct host strings and 10,980 error responses. Results are generated from the included data, rather than inserted as example numbers. More compact summaries do not automatically produce faster execution or accurate estimates at every parameter setting.

Reading guide: Sections 1-3 explain the task and implementation; Sections 4-5 present the measured evidence; Section 6 contains the complete Python source; Section 7 lists references.

1. Mini Project Description

A website operator may need a quick repeat-host check, a running estimate of distinct hosts, and a recent error count. Keeping every request makes memory grow with the stream. A summary can preserve selected information while discarding individual events. The objective here is to measure the resulting accuracy, state size and processing cost.

Observed dataset property	Value
Origin	NASA Kennedy Space Center WWW server, Internet Traffic Archive [1]
Valid requests / distinct hosts	1,891,714 / 81,982
Actual first / last timestamps	01/Jul/1995:00:00:01 -0400 / 28/Jul/1995:13:32:25 -0400
HTTP errors (400-599)	10,980 (0.580% of requests)
Rejected / unknown-byte records	1 trailing fragment / 19,726 unknown response sizes
Compressed file size	20,676,672 bytes

The archive description labels this file as July 1995, but inspection finds valid records only through 28 July at 13:32:25. The final raw line is the incomplete fragment 'alyssa.p'. It is counted as rejected; no missing events are synthesized. The gzip file was read through EOF with its integrity check, and no timestamp inversions were found among valid records.

One stream element is one HTTP request, including asset requests and repeated requests. A host is an IP address or hostname; it is not a verified person. Missing response size '-' is kept as None because size is not needed by these algorithms. Raw order and duplicate requests are preserved. Only general traffic patterns are studied, following the archive's request, and redistribution is permitted [1].

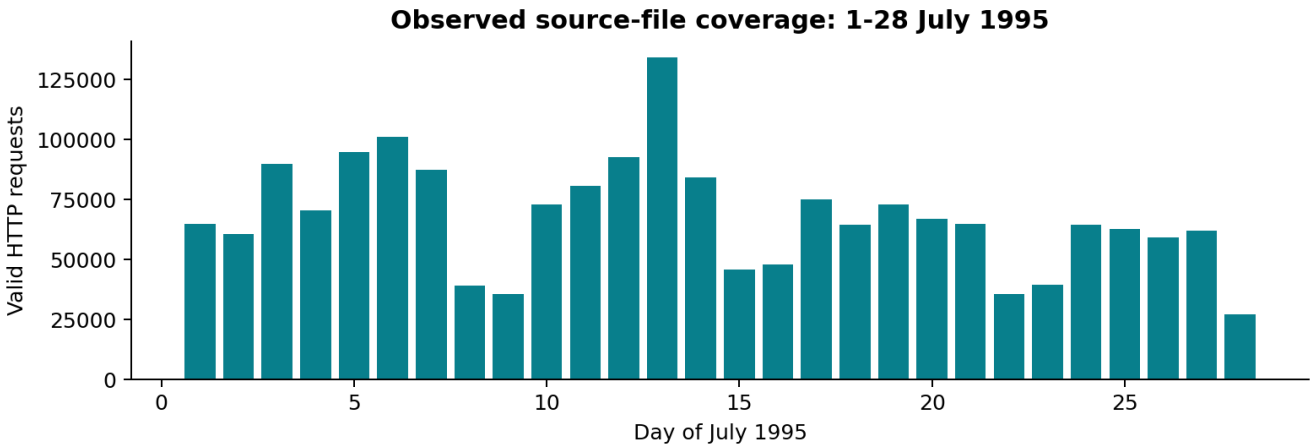


Figure 1. Counts calculated directly from valid source records. Days after 28 July are not represented as zero traffic.

2. Selected Concepts: Bloom Filtering

Purpose and mathematical model

The filter answers whether a host may have appeared earlier in the stream. It stores m bits and uses k hash positions per host. A zero in any queried position proves absence; all queried bits being one gives only possible membership. Collisions can make an unseen host appear present. Insertions never clear bits, so previously inserted hosts have no false negatives [2, 3].

```
P(bit stays 0 after d distinct inserts) = (1 - 1/m)^(k*d)
p_FP(d) ~= (1 - exp(-k*d/m))^k
k_opt ~= (m/d) * ln(2)
Online FPR = false positives / queries for truly unseen hosts
```

The probability expression assumes approximately independent uniform hash positions. Keyed 64-bit BLAKE2b hashes provide deterministic, well-distributed positions for this experiment; distinct keys are used for the k functions. The implementation packs eight bits per byte using masks, rather than allocating a Python Boolean per bit.

Update algorithm

```
for each incoming host x:
    result = true
    for each hash function h_j:
        p = h_j(x) mod m
        if bit[p] == 0: result = false
        bit[p] = 1
    compare result with exact prior membership
    add x to the exact validation set
```

The query is evaluated against prior state, before insertion. A false positive therefore means a first-ever host request was incorrectly classified as a repeat. Each first-seen host contributes one negative query. Later requests from that host are positive queries. The exact set is an evaluation instrument and is not part of the Bloom state.

Trade-off and limitations

State payload is ceil(m/8) bytes and each fused query/insertion costs O(k) hashes. Increasing m reduces saturation for fixed d and k. Increasing k helps only up to an appropriate occupancy-dependent range and always adds hashing work. This filter cannot list hosts or delete an individual host. It represents all previous history, not a sliding window.

The saved final_theoretical_fpr column describes occupancy at the end. The measured online FPR averages queries over a filter that starts empty. They are different quantities; the endpoint formula should not be interpreted as the expected cumulative observed rate.

2. Selected Concepts: Flajolet-Martin

Purpose and estimator

Distinct counting asks for the number of different host identifiers seen so far. Repeated requests should not increase that count. Hashing maps each host to a reproducible 64-bit string, so repeats produce the same evidence. Let rho(h(x)) be the number of trailing zero bits. Long zero runs are rare and become more likely as distinct hosts accumulate [3, 4].

```
P(rho(h(x)) >= r) = 2^(-r)       [uniform-hash model]
R_j = max over observed x of rho(h_j(x))
Single estimate E_j = 2^(R_j)
Group mean A_g = mean(E_j for j in group g)
Final estimate = median(A_1, ..., A_G)
```

This project implements the chapter's maximum-trailing-zero teaching variant. It does not implement the original first-zero bitmap estimator or HyperLogLog. In particular, the bitmap correction constant 0.77351 is not attached to a maximum-zero statistic. The versions use 1 hash; 16 hashes split into four groups of four; and 64 hashes split into eight groups of eight. For an even group count, the median is the average of the two middle group means.

Update algorithm

```
initially R_j = 0 for every hash; seen_any = false
for each host x:
    seen_any = true
    for j in 1..H:
        z = trailing_zeros(hash_j(x))
        R_j = max(R_j, z)
on query: return 0 if empty, otherwise median of group means
```

The all-zero 64-bit hash has a defined trailing-zero count of 64. Each register uses one byte, so H registers have H bytes of payload plus keys and Python object overhead. An update costs O(H), and a query costs O(H) plus sorting the few group means.

Limitations to investigate

The estimate changes in jumps. A rare hash with many trailing zeros can create a very large value of 2^R. Arithmetic group means retain sensitivity to such outliers; taking their median reduces some instability but supplies no small-error guarantee here. Five predetermined seeds expose that variation. More hashes can improve some runs without improving every seed or removing this variant's bias. No empirical correction is fitted to the NASA answers.

2. Selected Concepts: DGIM and Sliding Windows

Purpose and binary representation

For request t, set b_t to one for HTTP status 400-599 and zero otherwise. The requested answer is the sum of these bits in the last N valid requests. A request-count window does not represent a fixed number of seconds when traffic rates change. DGIM summarizes the ones using buckets whose sizes are powers of two [3, 5].

```
C_t(N) = sum b_i for max(1,t-N+1) <= i <= t
bucket = (number of ones, timestamp of newest one)
at most two buckets of each size
estimate = sum(bucket sizes) - half the oldest retained size
```

A timestamp here is the integer event index t. On every bit, including zero, time advances and buckets whose newest timestamp is at most t-N expire. A one creates a size-one bucket. If three buckets share a size, merge the two oldest and keep the newer of their endpoint timestamps; repeat if the larger size now has three buckets.

```
advance event index and expire old buckets
if bit == 1:
    insert (1, t) as newest bucket
    while three buckets have the same size:
        merge the oldest two, keeping the newer endpoint
query: sum all sizes, with half-weight on the oldest bucket
```

When t is at most N, no prior observations have expired, so the implementation returns the sum exactly. A retained size-one oldest bucket is also exact because its single one is known to be inside the window. An empty bucket list returns zero. Otherwise half-weighting approximates the uncertain contribution of the oldest bucket.

The basic two-bucket rule gives the standard coarse relative-error bound of at most 50% for a positive true count. Zero-count windows are evaluated separately and should return zero. There are O(log N) buckets. Classical modulo-N timestamps use O(log N) bits each, giving O(log-squared N) bits overall. This readable Python version uses absolute event indices and Python tuples, so its actual overhead is measured separately.

The exact baseline is a circular bytearray with a running total: O(N) bytes and constant-time updates and queries. A packed exact bit ring could use only ceil(N/8) payload bytes; that theoretical alternative is shown alongside the implemented baseline.

3. Mini Project Design and Algorithms

Stage	Behavior
Source and parser	Read gzip line by line; validate the timestamp and HTTP fields; retain valid requests in original order.
Bounded replay	Collect at most 1,000 parsed events per batch. Stream speed is simulated without real-time sleeps.
Approximate branches	Host -> Bloom; host -> FM; error bit -> DGIM. Each branch has independent state.
Exact validation	Prior-host set for Bloom; distinct-host set for FM; byte ring and sum for DGIM.
Measurement	Time updates/queries separately; compare answers outside timing; measure final reachable state.
Outputs	CSV rows and checkpoints -> seven figures -> report; preserve raw repetitions and provenance.

Each replay is one forward traversal. Experiments reopen the file for independent runs; algorithms do not revisit previous events within a run. The profiler and exact sets deliberately use more memory to establish ground truth. Shared parsing buffers, validation state and plots are excluded from sketch-state comparisons.

The web implementation adds a responsive HTML/CSS interface, a JavaScript Web Worker, and lossless compact browser inputs. The worker incrementally decompresses 20-byte records and updates the three summaries plus exact validation. The catalog preserves host and request strings. Prepared BLAKE2b hash words and zero counts reproduce the Python hashes without millions of slow browser hash operations; summary states and answers are calculated on every event, rather than loaded as prepared results.

Experiment	Settings
Bloom parameter sweep	m = 65,536; 262,144; 1,048,576 bits; k = 2, 4, 7 (9 combinations)
FM parameter sweep	H = 1, 16, 64; fixed grouping described in Section 2
DGIM parameter sweep	N = 1,000; 10,000; 100,000 requests
Sweep stream and seeds	First 100,000 valid requests; seeds 7, 19, 42, 73, 101; deterministic DGIM run once
Default full-stream settings	Bloom m=1,048,576 and k=7; FM H=16; DGIM N=10,000; seed 42
Scaling and timing	10,000; 100,000; 500,000; EOF snapshots within three full replays per algorithm

The 1,000-event batch is also the FM/DGIM query interval. Bloom requires one membership query per event. Timed blocks include their updates and queries; parsing, comparison, CSV writing and memory inspection are outside those blocks. Approximate/exact comparisons are made within a task, rather than claiming the three different tasks have identical costs.

4. Demonstration of Concepts

At EOF, 1,891,714 valid requests have been processed. The same source-file order is used for both approximate and exact calculations. The table reports the first full replay with seed 42; repeated-run timing is reported separately.

Concept	Observed result	Exact comparison
Bloom prior-host membership	27 false positives; online FPR 0.03%	81,982 genuinely unseen-host queries; 0 false negatives
FM distinct hosts	Estimate 442,368.0; relative error 439.59%	81,982 distinct host strings
DGIM recent errors	Estimate 28.0; relative error 6.67%	30 errors in the final 10,000 requests

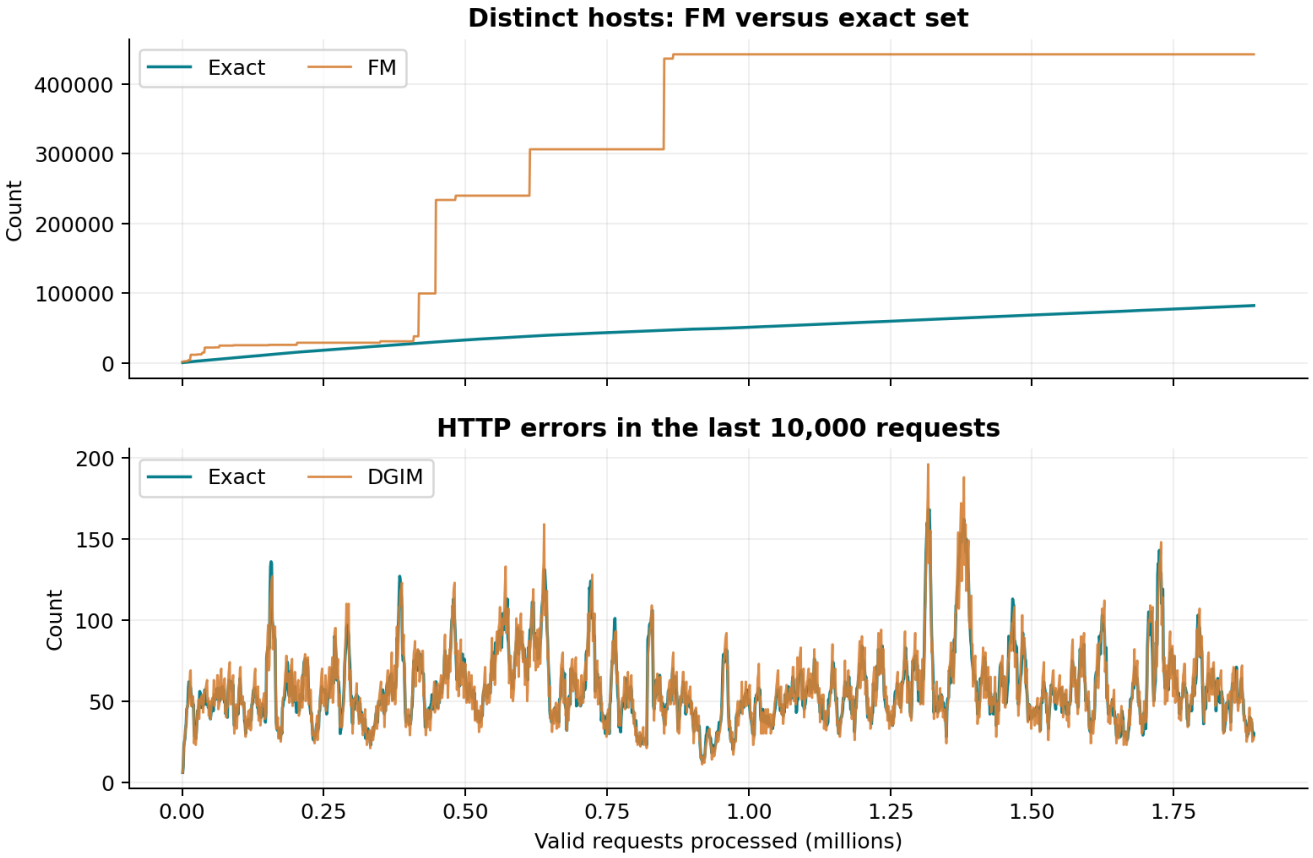


Figure 2. Checkpoints every 1,000 requests plus EOF. FM is cumulative; DGIM describes a moving request-count window.

Bloom's uncertain positive result is visible as a measured false-positive rate. FM's stepwise changes and possible overestimation are visible against exact cardinality. DGIM follows changes in recent errors with a much smaller summary than the full request history. The checkpoint CSVs retain the plotted numbers for inspection.

4. Demonstration: The Interactive Web Application

The web application turns the mini project into a working visual lab. It offers historical replay of the first 10,000, 100,000, 500,000 or all 1,891,714 valid requests. This is live computation on historical records, not a connection to current NASA traffic.

Interface	What it demonstrates
Overview	Current counts, exact-versus-approximate curves, source timestamps, recent real requests and logical summary payload.
Replay controls	Start, pause, resume, reset, and exactly one event per Step. Speed ranges from one to 100,000 requested events per second; compute capacity can limit it.
Bloom state	128 actual bit positions at a time, paging and jump-to-hash; current membership decision, prior exact membership and false positives.
FM state	The actual maximum-zero registers, powers-of-two evidence, group means, current estimate and relative error.
DGIM state	Power-of-two buckets ordered by recency; current merges/expirations and last 128 actual binary request events.
Experiments	The 63 recorded Python parameter cases, memory and throughput comparisons, CSV export and transparent FM error.
Dataset and source	Provenance, actual dates, response distribution, checksum, original request inspection, complete source explorer and project ZIP download.

Changing a sketch parameter or hash seed restarts the replay. This prevents an existing state from being labeled with settings that did not produce it. Changing speed affects only pacing. Work is done in a worker so the controls and layout can remain responsive. The sidebar becomes a mobile menu; cards and charts stack, while wide data tables scroll inside their panels. Keyboard focus, accessible labels and reduced-motion behavior are included.

```
python3 serve_web.py --open
# Or open output/StreamLab_Offline.html directly.
cd web
npm test
```

Eight web tests include a full replay of every real event and the actual worker's initialization, step, playback and one-event-per-second pacing. The JavaScript implementation matches Python exactly: 81,982 distinct hosts, 27 Bloom false positives, FM estimate 442,368, DGIM estimate 28 versus 30 exact recent errors, and 10,980 total HTTP errors. Browser checks also verified initial loading, one-step processing and a 100,000-event replay.

Live memory badges show logical payload only. The browser has additional input/hash caches, JavaScript overhead and exact state. In particular, the standalone HTML embeds compressed inputs and is larger than the sketch state. The Experiments view labels measured Python state separately. A hosting source push was blocked by DNS restrictions in the task environment; the delivered app works locally and offline.

5. Evaluation: Bloom Filter Parameters

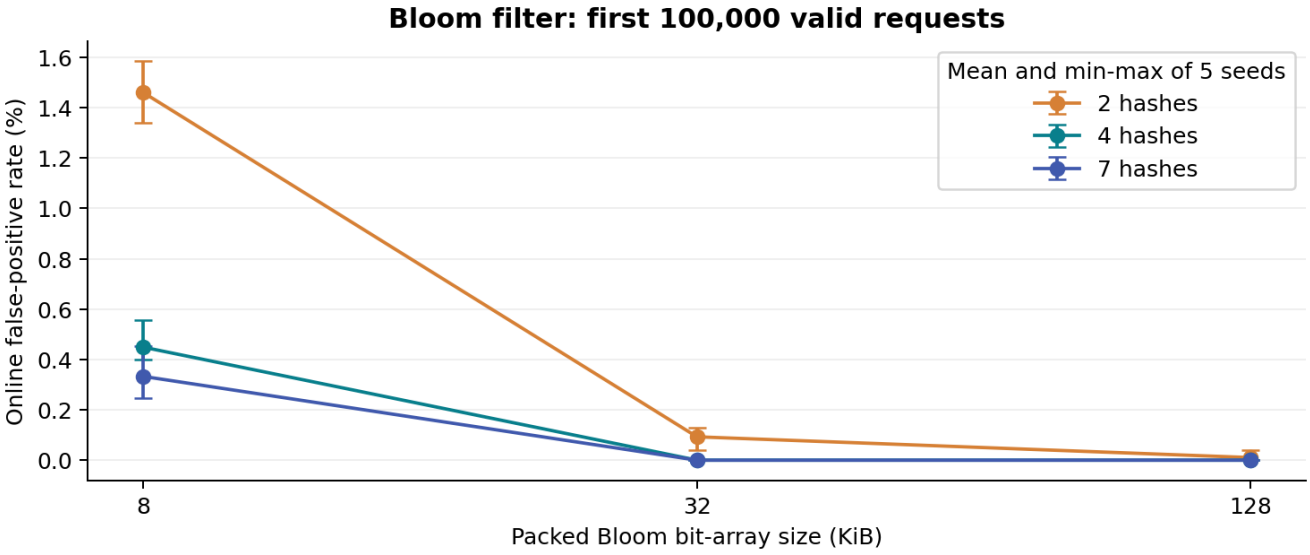


Figure 3. Online FPR across nine settings. Error bars span the five fixed seeds; they are not confidence intervals.

Bit payload	Hashes	Mean online FPR	Min-max FPR
8 KiB	2	1.46%	1.34% - 1.59%
8 KiB	4	0.45%	0.40% - 0.55%
8 KiB	7	0.33%	0.25% - 0.45%
32 KiB	2	0.09%	0.04% - 0.13%
32 KiB	4	0.00%	0.00% - 0.00%
32 KiB	7	0.00%	0.00% - 0.00%
128 KiB	2	0.01%	0.00% - 0.04%
128 KiB	4	0.00%	0.00% - 0.00%
128 KiB	7	0.00%	0.00% - 0.00%

At fixed k, the larger array spreads the inserted hosts over more bits and reduces collisions. Hash count is not a universal accuracy knob: the best k depends on m/d. If the filter becomes crowded, setting more positions can increase saturation. When the array is very large relative to the 7,752 distinct hosts in this prefix, some observed rates are zero. That means no false positives were observed in these finite tests; it does not prove a zero underlying probability.

The prefix sweep should not be mistaken for the full-month occupancy. The default full-stream filter receives many more distinct hosts. Its memory stays fixed while the risk of false positives grows. No false negatives were observed in the evaluated runs, consistent with the insertion-only design.

5. Evaluation: FM Variability

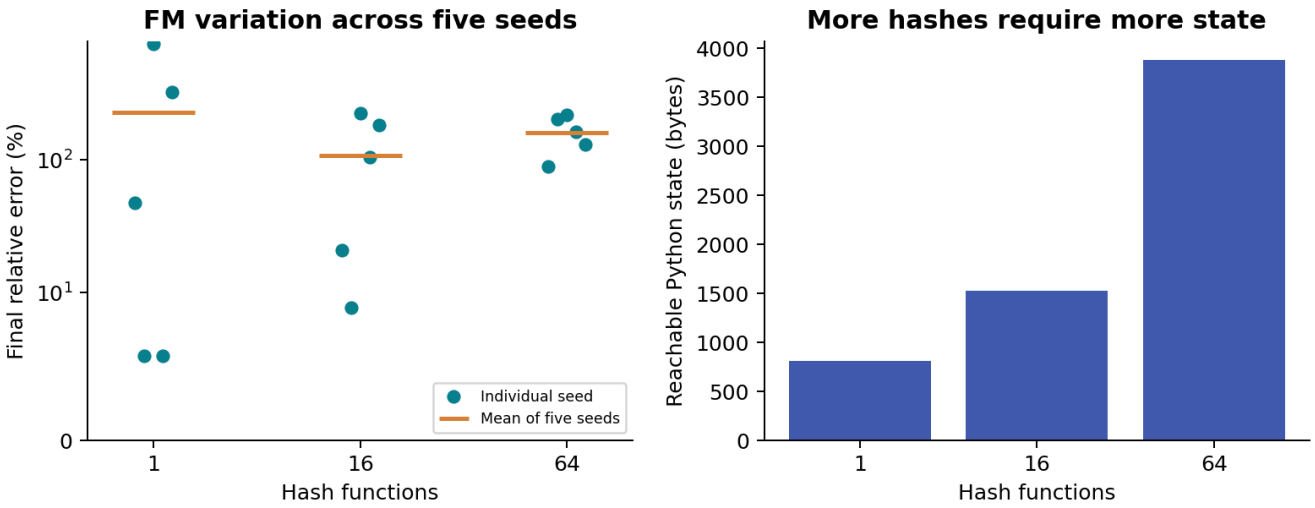


Figure 4. Per-seed final errors and maintained state on the first 100,000 requests. The nonlinear error scale preserves outliers.

Hashes	Mean error	Minimum	Maximum	State bytes
1	225.33%	5.68%	745.41%	809
16	107.76%	8.98%	223.63%	1,523
64	159.57%	88.24%	217.03%	3,883

A single unusually long zero run can dominate 2^R, which explains the large variation for a single hash. Grouping multiple estimates changes the distribution but arithmetic means remain sensitive to extreme estimates. The results show the actual errors rather than claiming that increasing H guarantees improvement. These groups and seeds were fixed before inspecting outcomes; no seed was selected to make a favorable plot.

The default full-stream estimate is 442,368.0 against 81,982 exact hosts. Its error is 439.59%. This is an important practical limitation of the uncalibrated teaching estimator. It demonstrates how a tiny hash summary operates, but this experiment does not support using it for a strict-accuracy production counter. A separately evaluated bitmap FM or a modern cardinality sketch would be a useful extension, not a result claimed here.

5. Evaluation: DGIM Window Size

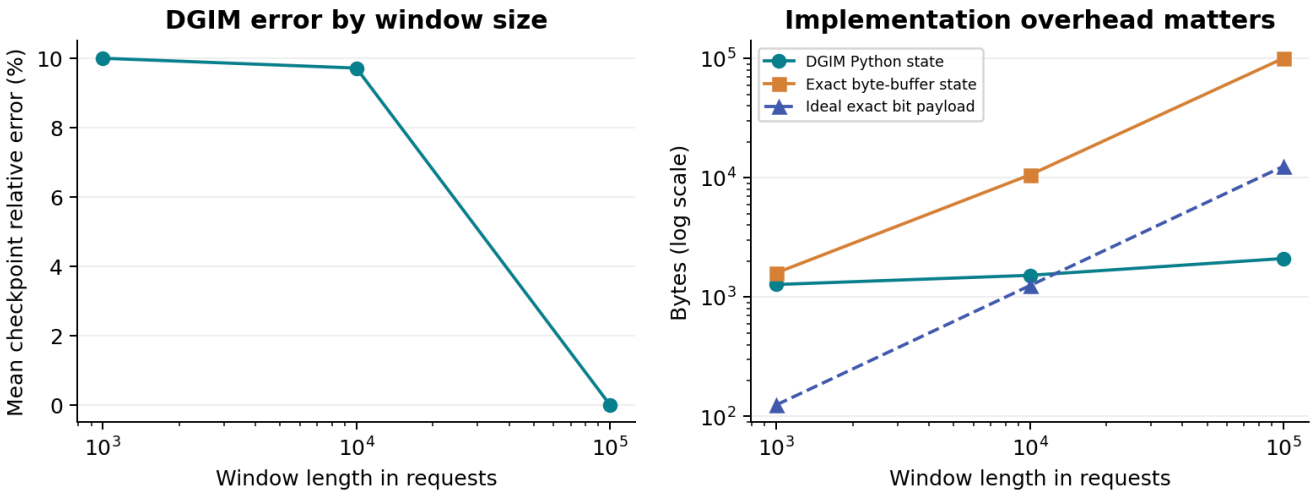


Figure 5. Mean checkpoint error and final state at the 100,000-request prefix. The ideal packed exact line is payload only.

Window	Mean rel. error	Final exact / est.	DGIM bytes	Exact bytes
1,000	10.00%	17 / 17.0	1276	1598
10,000	9.71%	55 / 57.0	1524	10590
100,000	0.00%	505 / 505.0	2108	100582

A larger window includes more old events and can require buckets at more scales. Its exact byte buffer grows linearly with N, while the bucket count grows slowly. Error does not have to grow monotonically with the window: it depends on how recent ones fall around the oldest bucket boundary. The N=100,000 run at this prefix is a startup case with no expired data, so its zero error is expected and is not evidence of exact behavior after the window starts sliding.

Mean relative error is the unweighted mean across the 1,000-request checkpoints, including zero-error empty windows. Absolute errors are also stored. For a zero true count, a zero estimate is recorded as zero error; a nonzero estimate would have an undefined relative error and would not silently be divided by zero. Small true counts can turn a small absolute error into a large percentage.

At small N, Python tuple/list overhead can outweigh the theoretical saving compared with an efficiently packed exact ring. This is why the figure includes the ideal N/8 exact payload. Claims of space efficiency must state which representation is used.

5. Evaluation: Memory and Processing Cost

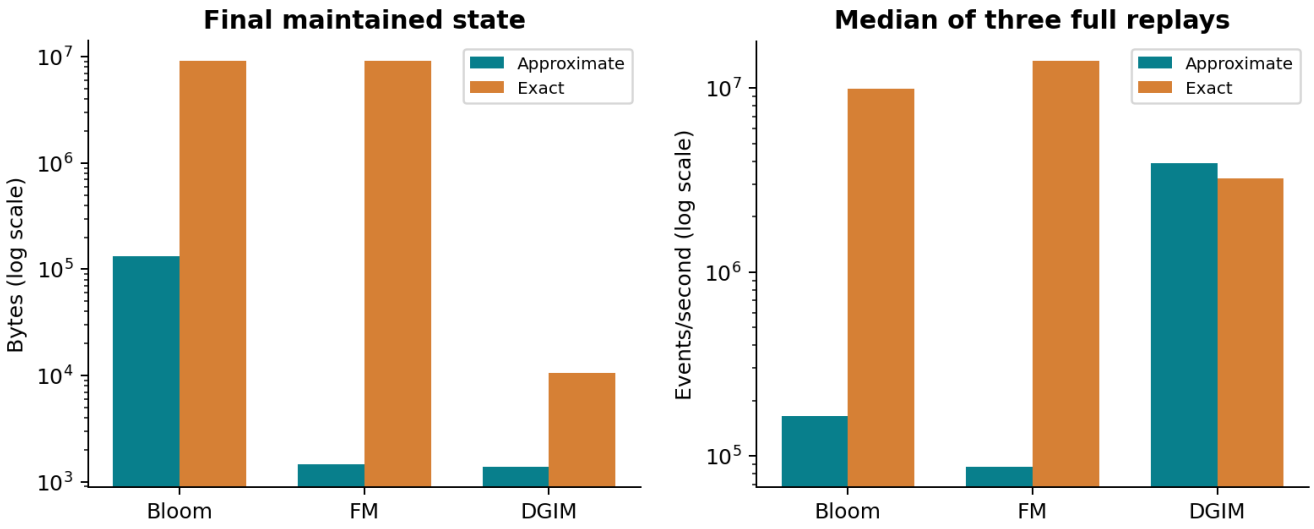


Figure 6. Default parameters at EOF. Timing excludes parsing and validation; throughput uses the median of three full replays.

Task	Approx. state	Exact state	Approx. events/s	Exact events/s
BLOOM	131,927	9,086,764	164,793	9,941,799
FM	1,459	9,086,764	87,774	13,993,327
DGIM	1,396	10,574	3,902,897	3,212,609

Memory is the recursively measured reachable state using sys.getsizeof, with each object counted once per algorithm. It includes arrays, keys, containers and Python overhead. It is final maintained state, not peak process RSS or a claim about total application memory. Temporary allocations, interpreter code and the shared input batch are excluded consistently. Exact states are measured independently.

Python's exact set uses an optimized built-in hash table. Bloom and FM perform several BLAKE2b calculations per event and execute their loops in Python, so their smaller state need not lead to higher throughput. DGIM processes zeros cheaply, and only about 0.58% of requests are errors; that sparsity helps this implementation. Results are workload- and machine-dependent, rather than a general ranking of the concepts.

Measured environment: Python 3.12.7; macOS-27.0.1-arm64-arm-64bit; architecture arm64. Raw timings, minimum/maximum cumulative durations and all repetitions are included in scaling_raw.csv and scaling_summary.csv.

5. Evaluation: Scalability and Reproducibility

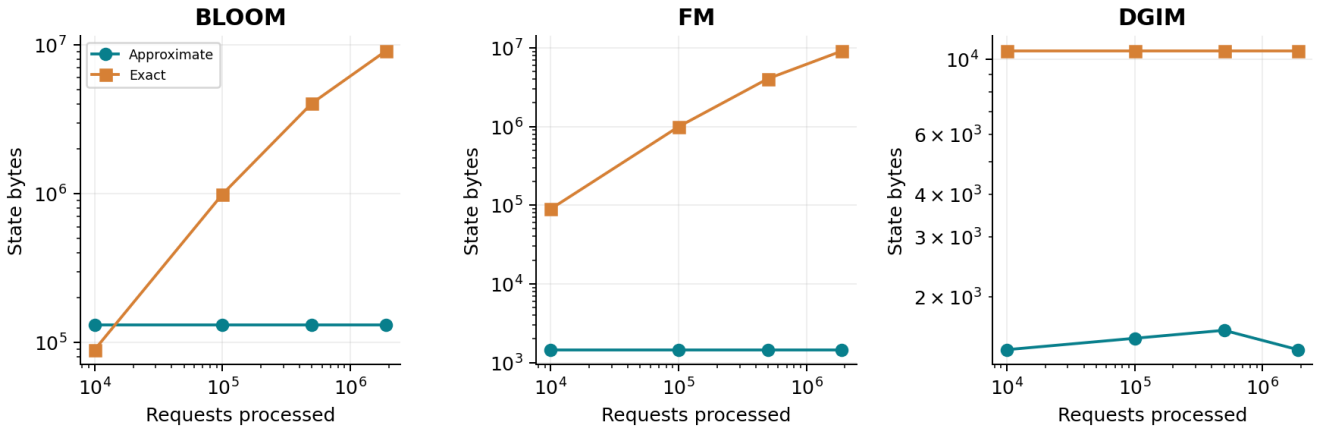


Figure 7. Prefix snapshots within three full replays. Window size stays fixed; distinct-host exact state grows as new hosts appear.

Bloom state is fixed by m and k, and FM state by H. They summarize all history. DGIM depends on its fixed recent window and the current bucket pattern. The exact distinct set grows with distinct hosts, not directly with request count; many repeated requests can arrive without adding any keys. The exact circular buffer remains fixed once N is chosen.

Validation

Fifteen unittest cases cover Bloom membership and saturation; query-before-insert; FM empty streams, duplicate/order invariance and an independent trailing-zero calculation; parser failures and valid-row limits; and DGIM invariants, expiry, alternation, sparse input and randomized streams. DGIM is also checked exhaustively on every 9-bit sequence for six window lengths, against an independently summed history. The tests verify the 50% positive-count error bound on these fixtures. Artificial fixtures are used only for tests, not for the experimental results.

Reproduction and limitations

```
python main.py --mode demo
python -m unittest discover -s tests -v
python main.py --mode experiments --profile
python generate_report.py
```

The ZIP includes the compressed real dataset, exact parameter choices, five seeds, raw results and plotting/report scripts. Deterministic estimates reproduce on the same input, whereas timings and Python object sizes can vary across systems. Only one historical server workload is evaluated. Error sparsity, repeated hosts, the capture's partial-month coverage and Python implementation overhead limit generalization.

The evidence supports compact summary states, while exposing their task-specific costs: Bloom can misclassify new hosts, this FM variant can be substantially inaccurate, and DGIM trades exact recent counts for bucket-based estimates. None of the three summaries replaces the full log when individual request retrieval is required.

6. Code Appendix

Complete project source follows, including the three core algorithms, replay and evaluation code, web interface, worker, browser algorithms, data preparation, offline bundler, figures, report generator, download/packaging helpers and tests. The matching standalone files in the ZIP can be executed directly. Wrapped continuation lines in this appendix retain the original source line number; the source files remain authoritative.

bloom_filter.py

```python
"""Bloom membership filter with a packed bit array and deterministic hashes."""
import hashlib
import math


class BloomFilter:
    def __init__(self, bits=1_048_576, hashes=7, seed=42):
        if bits <= 0 or hashes <= 0:
            raise ValueError("bits and hashes must be positive")
        self.bits, self.hashes = bits, hashes
        self.array = bytearray((bits + 7) // 8)
        self.keys = [(seed + i).to_bytes(8, "big") for i in range(hashes)]

    def _positions(self, value):
        raw = value.encode("utf-8")
        for key in self.keys:
            digest = hashlib.blake2b(raw, digest_size=8, key=key).digest()
            yield int.from_bytes(digest, "big") % self.bits

    def contains(self, value):
        return all(self.array[p // 8] & (1 << (p % 8))
                   for p in self._positions(value))

    def add(self, value):
        for p in self._positions(value):
            self.array[p // 8] |= 1 << (p % 8)

    def check_and_add(self, value):
        """Query BEFORE insertion; compute all positions only once."""
        present = True
        for p in self._positions(value):
            mask = 1 << (p % 8)
            if not self.array[p // 8] & mask:
                present = False
            self.array[p // 8] |= mask
        return present

    def predicted_fpr(self, distinct_inserted):
        """Approximate FPR at current occupancy, not cumulative online FPR."""
        return (1 - math.exp(-self.hashes * distinct_inserted / self.bits)) ** self.hashes
```

dgim.py

```python
"""DGIM: at most two power-of-two buckets per size; newest bucket first."""


class DGIM:
    def __init__(self, window=10_000):
        if window <= 0:
            raise ValueError("window must be positive")
        self.window, self.time = window, 0
        self.buckets = []  # (number of ones, timestamp of newest one)

    def add(self, bit):
        if bit not in (0, 1):
            raise ValueError("DGIM accepts only 0 or 1")
        self.time += 1  # Zeros also advance the event-count window.
        cutoff = self.time - self.window
        while self.buckets and self.buckets[-1][1] <= cutoff:
            self.buckets.pop()
        if bit == 0:
            return
        self.buckets.insert(0, (1, self.time))
        size = 1
        while True:
            indexes = [i for i, b in enumerate(self.buckets) if b[0] == size]
            if len(indexes) <= 2:
                break
            newer, older = indexes[-2], indexes[-1]
            # Keep the end timestamp of the more recent of the merged buckets.
            self.buckets[newer] = (2 * size, self.buckets[newer][1])
            del self.buckets[older]
            size *= 2

    def estimate(self):
        if not self.buckets:
            return 0.0
        total = sum(size for size, _ in self.buckets)
        if self.time <= self.window:
            return float(total)  # No observations have expired yet.
        oldest = self.buckets[-1][0]
        # A size-one bucket is a single known in-window observation.
        return float(total) if oldest == 1 else total - oldest / 2


class ExactWindow:
    """Exact byte-per-event circular buffer; O(1) update and query."""
    def __init__(self, window):
        if window <= 0:
            raise ValueError("window must be positive")
        self.array = bytearray(window)
        self.position = 0
        self.total = 0

    def add(self, bit):
        self.total += bit - self.array[self.position]
        self.array[self.position] = bit
        self.position = (self.position + 1) % len(self.array)

    def estimate(self):
        return self.total
```

download_data.py

```python
"""Optional public-source download; the submission already includes the data."""
import argparse
import gzip
import json
from pathlib import Path
import urllib.request

from stream import DATA, ROOT, SOURCE_URL, checksum


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--destination", type=Path, default=DATA)
    args = parser.parse_args()
    if args.destination.exists():
        raise SystemExit("Destination exists; choose a different --destination to download again.")
    args.destination.parent.mkdir(parents=True, exist_ok=True)
    temporary = args.destination.with_suffix(".part")
    try:
        with urllib.request.urlopen(SOURCE_URL, timeout=90) as src, temporary.open("wb") as dst:
            while chunk := src.read(1024 * 1024):
                dst.write(chunk)
        # Read through EOF to verify the gzip CRC, not only its header.
        with gzip.open(temporary, "rb") as f:
            while f.read(1024 * 1024):
                pass
        manifest = ROOT / "data" / "manifest.json"
        if manifest.exists():
            expected = json.loads(manifest.read_text())["sha256"]
            actual = checksum(temporary)
            if actual != expected:
                raise ValueError("Checksum differs from the evaluated dataset")
        temporary.replace(args.destination)
    finally:
        temporary.unlink(missing_ok=True)
    print(args.destination)


if __name__ == "__main__":
    main()
```

experiments.py

```
"""Replay bounded batches, measure algorithm state, export honest comparisons."""
from collections import deque
import csv
import gc
import itertools
import math
import platform
import statistics
import sys
from time import perf_counter
from pathlib import Path

from bloom_filter import BloomFilter
from flajolet_martin import FlajoletMartin
from dgim import DGIM, ExactWindow
from stream import DATA, ROOT, events, write_json, checksum

SEEDS = [7, 19, 42, 73, 101]
RESULTS = ROOT / "results"


def deep_size(value, seen=None):
    """Reachable state in bytes; count each referenced object once per state.

    This includes Python overhead. It excludes interpreter/code objects,
    temporary update allocations, the input batch, and the validation state.
    """
    seen = set() if seen is None else seen
    if id(value) in seen:
        return 0
    seen.add(id(value))
    total = sys.getsizeof(value)
    if isinstance(value, dict):
        total += sum(deep_size(k, seen) + deep_size(v, seen) for k, v in value.items())
    elif isinstance(value, (list, tuple, set, frozenset, deque)):
        total += sum(deep_size(v, seen) for v in value)
    elif hasattr(value, "__dict__"):
        total += deep_size(vars(value), seen)
    return total


def write_csv(path, rows):
    if not rows:
        raise ValueError("No result rows")
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)


def run_case(kind, params=None, limit=None, seed=42, path=DATA, capture=False,
             targets=None):
    """The same parsed batches feed separate approximate and exact states.

    Timing excludes parsing, comparisons and deep_size. It includes state
    updates and queries. Query cadence is 1000 events (also the bounded batch
    size); Bloom membership is naturally queried on EVERY event.
    """
    params = params or {}
    if kind == "bloom":
        sketch, exact = BloomFilter(seed=seed, **params), set()
    elif kind == "fm":
        sketch, exact = FlajoletMartin(seed=seed, **params), set()
    elif kind == "dgim":
        sketch = DGIM(**params)
        exact = ExactWindow(sketch.window)
    else:
        raise ValueError(kind)
    n = fp = negatives = false_negatives = 0
    approx_seconds = exact_seconds = 0.0
    errors, absolute_errors, history, snapshots = [], [], [], []
    targets = set(targets or [])
    stats = {}
    iterator = iter(events(path, limit, stats))
    final_snapshot = None
    while True:
        batch = list(itertools.islice(iterator, 1000))
        if not batch:
            break
        values = [e.error if kind == "dgim" else e.host for e in batch]
        start = perf_counter()
```

```
        if kind == "bloom":
            approximate_answers = [sketch.check_and_add(v) for v in values]
        else:
            for v in values:
                sketch.add(v)
            estimate = sketch.estimate()
        approx_seconds += perf_counter() - start
        start = perf_counter()
        if kind == "bloom":
            exact_answers = []
            for v in values:
                exact_answers.append(v in exact)
                exact.add(v)
        elif kind == "fm":
            for v in values:
                exact.add(v)
            truth = len(exact)
        else:
            for v in values:
                exact.add(v)
            truth = exact.estimate()
        exact_seconds += perf_counter() - start
        n += len(batch)
        if kind == "bloom":
            for predicted, actual in zip(approximate_answers, exact_answers):
                negatives += int(not actual)
                fp += int(predicted and not actual)
                false_negatives += int(actual and not predicted)
            truth, estimate = len(exact), None
            error = fp / negatives if negatives else 0.0
            absolute = None
        else:
            absolute = abs(estimate - truth)
            error = absolute / truth if truth else (0.0 if estimate == 0 else None)
            if error is not None:
                errors.append(error)
            absolute_errors.append(absolute)
        if capture:
            history.append(dict(events=n, exact=truth, estimate=estimate,
                                relative_error=error, absolute_error=absolute))
        if n in targets or (limit is not None and n == limit):
            final_snapshot = dict(events=n, exact=truth, estimate=estimate,
                relative_error=error, approximate_seconds=approx_seconds,
                exact_seconds=exact_seconds, state_bytes=deep_size(sketch),
                exact_state_bytes=deep_size(exact))
            snapshots.append(final_snapshot)
    if n == 0:
        raise ValueError("The input contains no valid events")
    # Include the actual EOF, even if it was not a multiple of 1000.
    if not final_snapshot or final_snapshot["events"] != n:
        snapshots.append(dict(events=n, exact=truth, estimate=estimate,
            relative_error=error, approximate_seconds=approx_seconds,
            exact_seconds=exact_seconds, state_bytes=deep_size(sketch),
            exact_state_bytes=deep_size(exact)))
    row = dict(algorithm=kind, events=n, seed=seed, bits=sketch.bits if kind == "bloom" else "",
        hashes=params.get("hashes", 7 if kind == "bloom" else 16 if kind == "fm" else ""),
        window=params.get("window", 10_000 if kind == "dgim" else ""),
        exact=truth, estimate=estimate, relative_error=error,
        mean_checkpoint_relative_error=statistics.mean(errors) if errors else None,
        mean_checkpoint_absolute_error=statistics.mean(absolute_errors) if absolute_errors else None,
        false_positives=fp, negative_queries=negatives, false_negatives=false_negatives,
        final_theoretical_fpr=sketch.predicted_fpr(len(exact)) if kind == "bloom" else None,
        state_bytes=deep_size(sketch), exact_state_bytes=deep_size(exact),
        ideal_payload_bytes=((sketch.bits + 7) // 8 if kind == "bloom" else
                             sketch.hashes if kind == "fm" else None),
        exact_packed_window_bytes=math.ceil(sketch.window / 8) if kind == "dgim" else None,
        approximate_seconds=approx_seconds, exact_seconds=exact_seconds,
        throughput=n / approx_seconds, exact_throughput=n / exact_seconds,
        buckets=len(sketch.buckets) if kind == "dgim" else None,
        skipped_lines=stats["skipped"])
    return row, history, snapshots


def run_experiments(path=DATA):
    RESULTS.mkdir(exist_ok=True)
    sweep = []
    configurations = [("bloom", {"bits": m, "hashes": k})
                      for m in [65_536, 262_144, 1_048_576] for k in [2, 4, 7]]
    configurations += [("fm", {"hashes": k}) for k in [1, 16, 64]]
    configurations += [("dgim", {"window": n}) for n in [1000, 10_000, 100_000]]
    for kind, params in configurations:
        for seed in ([42] if kind == "dgim" else SEEDS):
            row, _, _ = run_case(kind, params, 100_000, seed, path)
            sweep.append(row)
        print("Parameter sweep:", kind, params, flush=True)
        write_csv(RESULTS / "parameter_sweep.csv", sweep)
```

```
    # A full pass per algorithm and repetition also provides cumulative timing
    # at each requested prefix, avoiding redundant replays of shorter prefixes.
    scaling, defaults, full_checkpoints = [], [], {}
    for kind in ["bloom", "fm", "dgim"]:
        for repeat in range(1, 4):
            gc.collect()
            row, history, snapshots = run_case(kind, path=path, capture=repeat == 1,
                targets=[10_000, 100_000, 500_000])
            for snapshot in snapshots:
                scaling.append(dict(algorithm=kind, repetition=repeat, **snapshot))
            if repeat == 1:
                defaults.append(row)
                full_checkpoints[kind] = history
                write_csv(RESULTS / f"{kind}_checkpoints.csv", history)
            print("Full pass:", kind, "repeat", repeat, "events", row["events"], flush=True)
            write_csv(RESULTS / "scaling_raw.csv", scaling)
    aggregates = []
    for kind in ["bloom", "fm", "dgim"]:
        for n in sorted({r["events"] for r in scaling if r["algorithm"] == kind}):
            rows = [r for r in scaling if r["algorithm"] == kind and r["events"] == n]
            a, e = [r["approximate_seconds"] for r in rows], [r["exact_seconds"] for r in rows]
            aggregates.append(dict(algorithm=kind, events=n, repetitions=len(rows),
                median_seconds=statistics.median(a), min_seconds=min(a), max_seconds=max(a),
                median_exact_seconds=statistics.median(e), throughput=n / statistics.median(a),
                exact_throughput=n / statistics.median(e),
                state_bytes=rows[0]["state_bytes"], exact_state_bytes=rows[0]["exact_state_bytes"],
                exact=rows[0]["exact"], estimate=rows[0]["estimate"],
                relative_error=rows[0]["relative_error"]))
    write_csv(RESULTS / "scaling_summary.csv", aggregates)
    write_csv(RESULTS / "full_summary.csv", defaults)
    write_json(RESULTS / "run_metadata.json", dict(python=sys.version, platform=platform.platform(),
        machine=platform.machine(), seeds=SEEDS, batch_size=1000, timing_repetitions=3,
        dataset_sha256=checksum(path), sweep_valid_events=100_000,
        timing="update + query; excludes parsing, validation, memory measurement and plotting",
        memory="final reachable Python state, counted independently; not peak process RSS",
        scaling="prefix snapshots within three full replays per algorithm"))
    return defaults
```

flajolet_martin.py

```
"""Chapter-4 trailing-zero FM estimator, with a median of group means.

This is the maximum-trailing-zero teaching variant. It is not the original
first-zero bitmap estimator; no 0.77351 bitmap correction is applied.
"""
import hashlib
import statistics


class FlajoletMartin:
    def __init__(self, hashes=16, seed=42):
        groups = {1: 1, 16: 4, 64: 8}
        if hashes not in groups:
            raise ValueError("hashes must be 1, 16 or 64")
        self.hashes = hashes
        self.groups = groups[hashes]
        self.max_zeros = bytearray(hashes)
        self.keys = [(seed + i).to_bytes(8, "big") for i in range(hashes)]
        self.seen_any = False

    def add(self, value):
        self.seen_any = True
        raw = value.encode("utf-8")
        for i, key in enumerate(self.keys):
            h = int.from_bytes(hashlib.blake2b(raw, digest_size=8, key=key).digest(), "big")
            zeros = (h & -h).bit_length() - 1 if h else 64
            if zeros > self.max_zeros[i]:
                self.max_zeros[i] = zeros

    def estimate(self):
        if not self.seen_any:
            return 0.0
        width = self.hashes // self.groups
        means = [sum(2.0 ** r for r in self.max_zeros[j:j + width]) / width
                 for j in range(0, self.hashes, width)]
        return statistics.median(means)
```

generate_report.py

```
"""Build the assignment PDF from actual saved measurements, including all code."""
from collections import defaultdict
import csv
from datetime import date
from html import escape
import json
from pathlib import Path
import statistics
import textwrap

from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.platypus import (SimpleDocTemplate, Paragraph, Spacer, PageBreak,
    Table, TableStyle, Image, Preformatted, KeepTogether)

from stream import ROOT

NAME, ID = "Mohammed Khalifah", "481013386"
REPORT = ROOT / "output" / "pdf" / f"{ID}_MohammedKhalifah_5113_Assignment_1.pdf"
NAVY, TEAL = colors.HexColor("#182B49"), colors.HexColor("#087F8C")
GRAY = colors.HexColor("#516174")
WIDTH = A4[0] - 96
styles = getSampleStyleSheet()
styles.add(ParagraphStyle(name="Body", fontName="Helvetica", fontSize=10,
    leading=14.2, textColor=NAVY, spaceAfter=9))
styles.add(ParagraphStyle(name="TitleCustom", fontName="Helvetica-Bold", fontSize=27,
    leading=32, textColor=NAVY, spaceAfter=16))
styles.add(ParagraphStyle(name="Section", fontName="Helvetica-Bold", fontSize=18,
    leading=23, textColor=NAVY, spaceAfter=14))
styles.add(ParagraphStyle(name="Sub", fontName="Helvetica-Bold", fontSize=11.5,
    leading=15, textColor=TEAL, spaceBefore=8, spaceAfter=7))
styles.add(ParagraphStyle(name="SmallCustom", fontName="Helvetica", fontSize=8.2,
    leading=11, textColor=GRAY, spaceAfter=7))
styles.add(ParagraphStyle(name="Cell", fontName="Helvetica", fontSize=8,
    leading=10.5, textColor=NAVY))
styles.add(ParagraphStyle(name="CodeCustom", fontName="Courier", fontSize=6.6,
    leading=8.4, textColor=NAVY, spaceAfter=4))
story = []


def p(text, style="Body"):
    story.append(Paragraph(text, styles[style]))


def section(title):
    if story:
        story.append(PageBreak())
    p(title, "Section")


def table(headers, rows, widths=None):
    header_style = ParagraphStyle(name="WhiteCell", parent=styles["Cell"], textColor=colors.white)
    values = [[Paragraph(escape(str(x)), header_style) for x in headers]]
    values += [[Paragraph(escape(str(x)), styles["Cell"]) for x in row] for row in rows]
    t = Table(values, colWidths=widths, repeatRows=1, hAlign="LEFT")
    t.setStyle(TableStyle([("BACKGROUND", (0, 0), (-1, 0), NAVY),
        ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
        ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.HexColor("#EDF4F6"), colors.white]),
        ("VALIGN", (0, 0), (-1, -1), "TOP"),
        ("LEFTPADDING", (0, 0), (-1, -1), 7),
        ("RIGHTPADDING", (0, 0), (-1, -1), 7),
        ("TOPPADDING", (0, 0), (-1, -1), 6),
        ("BOTTOMPADDING", (0, 0), (-1, -1), 6),
        ("LINEBELOW", (0, 0), (-1, 0), .8, TEAL)]))
    story.extend([t, Spacer(1, 10)])


def figure(name, caption, width=WIDTH):
    from PIL import Image as PILImage
    path = ROOT / "results" / "figures" / (name + ".png")
    with PILImage.open(path) as im:
        height = width * im.height / im.width
    story.extend([Image(str(path), width=width, height=height), Spacer(1, 5)])
    p(caption, "SmallCustom")


def code(text):
    story.append(Preformatted(text.strip("\n"), styles["CodeCustom"]))
    story.append(Spacer(1, 7))


```

```
def read_csv(name):
    with (ROOT / "results" / name).open() as f:
        return list(csv.DictReader(f))


def percent(value):
    return f"{100 * float(value):.2f}%"


def frame(canvas, doc):
    canvas.saveState()
    w, h = A4
    canvas.setFillColor(TEAL)
    canvas.rect(0, h - 8, w, 8, stroke=0, fill=1)
    canvas.setFont("Helvetica", 8)
    canvas.setFillColor(GRAY)
    canvas.drawString(48, h - 31, f"{NAME}  |  Student ID: {ID}")
    canvas.drawRightString(w - 48, h - 31, "CSD 5113 - Assignment 2")
    canvas.setStrokeColor(colors.HexColor("#D8E4E8"))
    canvas.line(48, 40, w - 48, 40)
    canvas.drawString(48, 27, "Mining Data Streams | Real NASA HTTP requests")
    canvas.drawRightString(w - 48, 27, f"{doc.page}")
    canvas.restoreState()


def main():
    manifest = json.loads((ROOT / "data" / "manifest.json").read_text())
    metadata = json.loads((ROOT / "results" / "run_metadata.json").read_text())
    sweep, summaries, scaling = (read_csv(n) for n in
        ["parameter_sweep.csv", "full_summary.csv", "scaling_summary.csv"])
    full = {r["algorithm"]: r for r in summaries}
    final_n = int(full["fm"]["events"])
    timings = {r["algorithm"]: r for r in scaling if int(r["events"]) == final_n}
    bloom, fm, dgim = [full[k] for k in ["bloom", "fm", "dgim"]]
    story.append(Spacer(1, 65))
    p("MINI PROJECT / CHAPTER 4", "Sub")
    p("Efficient Monitoring<br/>of Website Activity<br/>Using Data Stream Algorithms", "TitleCustom")
    p("An interactive, responsive web application and reproducible experiment on real NASA HTTP access logs",
"Body")
    story.append(Spacer(1, 18))
    table(["Student", "Course", "Submission"],
          [[f"{NAME}\n{ID}", "Big Data Mining\nCSD 5113", "Assignment 2\nSemester I, 2026-2027"]],
          [WIDTH*.36, WIDTH*.32, WIDTH*.32])
    p("Project overview", "Sub")
    p("StreamLab is the project's browser interface. Playback controls, a live request feed, "
      "charts and algorithm-state views make the stream processing visible. The prepared "
      "offline HTML opens directly in a browser; the modular version runs with a small "
      "standard-library Python static server.")
    p(f"This project replays {final_n:,} genuine HTTP requests as a sequential stream. "
      "Three independently implemented summaries answer different questions: whether a host "
      "has appeared before, how many different hosts have appeared, and how many recent "
      "requests produced HTTP errors. Exact baselines make the approximation costs visible.")
    p(f"The source contains {manifest['exact_hosts']:,} distinct host strings and "
      f"{manifest['errors']:,} error responses. Results are generated from the included data, "
      "rather than inserted as example numbers. More compact summaries do not automatically "
      "produce faster execution or accurate estimates at every parameter setting.")
    p("Reading guide: Sections 1-3 explain the task and implementation; Sections 4-5 present "
      "the measured evidence; Section 6 contains the complete Python source; Section 7 lists references.",
"SmallCustom")

    section("1. Mini Project Description")
    p("A website operator may need a quick repeat-host check, a running estimate of distinct "
      "hosts, and a recent error count. Keeping every request makes memory grow with the stream. "
      "A summary can preserve selected information while discarding individual events. The "
      "objective here is to measure the resulting accuracy, state size and processing cost.")
    table(["Observed dataset property", "Value"], [
        ["Origin", "NASA Kennedy Space Center WWW server, Internet Traffic Archive [1]"],
        ["Valid requests / distinct hosts", f"{final_n:,} / {manifest['exact_hosts']:,}"],
        ["Actual first / last timestamps", f"{manifest['first_timestamp']} / {manifest['last_timestamp']}"],
        ["HTTP errors (400-599)", f"{manifest['errors']:,} ({100*manifest['errors']/final_n:.3f}% of
requests)"],
        ["Rejected / unknown-byte records", f"{manifest['skipped']} trailing fragment /
{manifest['unknown_bytes']:,} unknown response sizes"],
        ["Compressed file size", f"{manifest['compressed_bytes']:,} bytes"],
    ], [WIDTH*.39, WIDTH*.61])
    p("The archive description labels this file as July 1995, but inspection finds valid records "
      "only through 28 July at 13:32:25. The final raw line is the incomplete fragment 'alyssa.p'. "
      "It is counted as rejected; no missing events are synthesized. The gzip file was read through "
      "EOF with its integrity check, and no timestamp inversions were found among valid records.")
    p("One stream element is one HTTP request, including asset requests and repeated requests. "
      "A host is an IP address or hostname; it is not a verified person. Missing response size '-' "
      "is kept as None because size is not needed by these algorithms. Raw order and duplicate "
      "requests are preserved. Only general traffic patterns are studied, following the archive's "
      "request, and redistribution is permitted [1].")
    figure("01_dataset", "Figure 1. Counts calculated directly from valid source records. Days after 28 July are
 not represented as zero traffic.")
```

```
165
166     section("2. Selected Concepts: Bloom Filtering")
167     p("Purpose and mathematical model", "Sub")
168     p("The filter answers whether a host may have appeared earlier in the stream. It stores "
169       "m bits and uses k hash positions per host. A zero in any queried position proves absence; "
170       "all queried bits being one gives only possible membership. Collisions can make an unseen "
171       "host appear present. Insertions never clear bits, so previously inserted hosts have no "
172       "false negatives [2, 3].")
173     code("P(bit stays 0 after d distinct inserts) = (1 - 1/m)^(k*d)\n"
174          "p_FP(d) ~= (1 - exp(-k*d/m))^k\n"
175          "k_opt ~= (m/d) * ln(2)\n"
176          "Online FPR = false positives / queries for truly unseen hosts")
177     p("The probability expression assumes approximately independent uniform hash positions. "
178       "Keyed 64-bit BLAKE2b hashes provide deterministic, well-distributed positions for this "
179       "experiment; distinct keys are used for the k functions. The implementation packs eight "
180       "bits per byte using masks, rather than allocating a Python Boolean per bit.")
181     p("Update algorithm", "Sub")
182     code("for each incoming host x:\n"
183          "    result = true\n"
184          "    for each hash function h_j:\n"
185          "        p = h_j(x) mod m\n"
186          "        if bit[p] == 0: result = false\n"
187          "        bit[p] = 1\n"
188          "    compare result with exact prior membership\n"
189          "    add x to the exact validation set")
190     p("The query is evaluated against prior state, before insertion. A false positive therefore "
191       "means a first-ever host request was incorrectly classified as a repeat. Each first-seen "
192       "host contributes one negative query. Later requests from that host are positive queries. "
193       "The exact set is an evaluation instrument and is not part of the Bloom state.")
194     p("Trade-off and limitations", "Sub")
195     p("State payload is ceil(m/8) bytes and each fused query/insertion costs O(k) hashes. "
196       "Increasing m reduces saturation for fixed d and k. Increasing k helps only up to an "
197       "appropriate occupancy-dependent range and always adds hashing work. This filter cannot "
198       "list hosts or delete an individual host. It represents all previous history, not a sliding window.")
199     p("The saved final_theoretical_fpr column describes occupancy at the end. The measured "
200       "online FPR averages queries over a filter that starts empty. They are different quantities; "
201       "the endpoint formula should not be interpreted as the expected cumulative observed rate.", "SmallCustom")
202
203     section("2. Selected Concepts: Flajolet-Martin")
204     p("Purpose and estimator", "Sub")
205     p("Distinct counting asks for the number of different host identifiers seen so far. Repeated "
206       "requests should not increase that count. Hashing maps each host to a reproducible 64-bit "
207       "string, so repeats produce the same evidence. Let rho(h(x)) be the number of trailing zero "
208       "bits. Long zero runs are rare and become more likely as distinct hosts accumulate [3, 4].")
209     code("P(rho(h(x)) >= r) = 2^(-r)       [uniform-hash model]\n"
210          "R_j = max over observed x of rho(h_j(x))\n"
211          "Single estimate E_j = 2^(R_j)\n"
212          "Group mean A_g = mean(E_j for j in group g)\n"
213          "Final estimate = median(A_1, ..., A_G)")
214     p("This project implements the chapter's maximum-trailing-zero teaching variant. It does "
215       "not implement the original first-zero bitmap estimator or HyperLogLog. In particular, "
216       "the bitmap correction constant 0.77351 is not attached to a maximum-zero statistic. "
217       "The versions use 1 hash; 16 hashes split into four groups of four; and 64 hashes split "
218       "into eight groups of eight. For an even group count, the median is the average of the "
219       "two middle group means.")
220     p("Update algorithm", "Sub")
221     code("initially R_j = 0 for every hash; seen_any = false\n"
222          "for each host x:\n"
223          "    seen_any = true\n"
224          "    for j in 1..H:\n"
225          "        z = trailing_zeros(hash_j(x))\n"
226          "        R_j = max(R_j, z)\n"
227          "on query: return 0 if empty, otherwise median of group means")
228     p("The all-zero 64-bit hash has a defined trailing-zero count of 64. Each register uses "
229       "one byte, so H registers have H bytes of payload plus keys and Python object overhead. "
230       "An update costs O(H), and a query costs O(H) plus sorting the few group means.")
231     p("Limitations to investigate", "Sub")
232     p("The estimate changes in jumps. A rare hash with many trailing zeros can create a very "
233       "large value of 2^R. Arithmetic group means retain sensitivity to such outliers; taking "
234       "their median reduces some instability but supplies no small-error guarantee here. "
235       "Five predetermined seeds expose that variation. More hashes can improve some runs "
236       "without improving every seed or removing this variant's bias. No empirical correction "
237       "is fitted to the NASA answers.")
238
239     section("2. Selected Concepts: DGIM and Sliding Windows")
240     p("Purpose and binary representation", "Sub")
241     p("For request t, set b_t to one for HTTP status 400-599 and zero otherwise. The requested "
242       "answer is the sum of these bits in the last N valid requests. A request-count window "
243       "does not represent a fixed number of seconds when traffic rates change. DGIM "
244       "summarizes the ones using buckets whose sizes are powers of two [3, 5].")
245     code("C_t(N) = sum b_i for max(1,t-N+1) <= i <= t\n"
246          "bucket = (number of ones, timestamp of newest one)\n"
247          "at most two buckets of each size\n"
248          "estimate = sum(bucket sizes) - half the oldest retained size")
249     p("A timestamp here is the integer event index t. On every bit, including zero, time "
250       "advances and buckets whose newest timestamp is at most t-N expire. A one creates a "
```

```
        "size-one bucket. If three buckets share a size, merge the two oldest and keep the "
        "newer of their endpoint timestamps; repeat if the larger size now has three buckets.")
    code("advance event index and expire old buckets\n"
         "if bit == 1:\n"
         "    insert (1, t) as newest bucket\n"
         "    while three buckets have the same size:\n"
         "        merge the oldest two, keeping the newer endpoint\n"
         "query: sum all sizes, with half-weight on the oldest bucket")
    p("When t is at most N, no prior observations have expired, so the implementation returns "
      "the sum exactly. A retained size-one oldest bucket is also exact because its single "
      "one is known to be inside the window. An empty bucket list returns zero. Otherwise "
      "half-weighting approximates the uncertain contribution of the oldest bucket.")
    p("The basic two-bucket rule gives the standard coarse relative-error bound of at most "
      "50% for a positive true count. Zero-count windows are evaluated separately and should "
      "return zero. There are O(log N) buckets. Classical modulo-N timestamps use O(log N) "
      "bits each, giving O(log-squared N) bits overall. This readable Python version uses "
      "absolute event indices and Python tuples, so its actual overhead is measured separately.")
    p("The exact baseline is a circular bytearray with a running total: O(N) bytes and "
      "constant-time updates and queries. A packed exact bit ring could use only ceil(N/8) "
      "payload bytes; that theoretical alternative is shown alongside the implemented baseline.")

    section("3. Mini Project Design and Algorithms")
    table(["Stage", "Behavior"], [
        ["Source and parser", "Read gzip line by line; validate the timestamp and HTTP fields; retain valid
requests in original order."],
        ["Bounded replay", "Collect at most 1,000 parsed events per batch. Stream speed is simulated without
real-time sleeps."],
        ["Approximate branches", "Host -> Bloom; host -> FM; error bit -> DGIM. Each branch has independent
state."],
        ["Exact validation", "Prior-host set for Bloom; distinct-host set for FM; byte ring and sum for DGIM."],
        ["Measurement", "Time updates/queries separately; compare answers outside timing; measure final
reachable state."],
        ["Outputs", "CSV rows and checkpoints -> seven figures -> report; preserve raw repetitions and
provenance."],
    ], [WIDTH*.27, WIDTH*.73])
    p("Each replay is one forward traversal. Experiments reopen the file for independent "
      "runs; algorithms do not revisit previous events within a run. The profiler and exact "
      "sets deliberately use more memory to establish ground truth. Shared parsing buffers, "
      "validation state and plots are excluded from sketch-state comparisons.")
    p("The web implementation adds a responsive HTML/CSS interface, a JavaScript Web Worker, "
      "and lossless compact browser inputs. The worker incrementally decompresses 20-byte "
      "records and updates the three summaries plus exact validation. The catalog preserves "
      "host and request strings. Prepared BLAKE2b hash words and zero counts reproduce the "
      "Python hashes without millions of slow browser hash operations; summary states and "
      "answers are calculated on every event, rather than loaded as prepared results.")
    table(["Experiment", "Settings"], [
        ["Bloom parameter sweep", "m = 65,536; 262,144; 1,048,576 bits; k = 2, 4, 7 (9 combinations)"],
        ["FM parameter sweep", "H = 1, 16, 64; fixed grouping described in Section 2"],
        ["DGIM parameter sweep", "N = 1,000; 10,000; 100,000 requests"],
        ["Sweep stream and seeds", "First 100,000 valid requests; seeds 7, 19, 42, 73, 101; deterministic DGIM
run once"],
        ["Default full-stream settings", "Bloom m=1,048,576 and k=7; FM H=16; DGIM N=10,000; seed 42"],
        ["Scaling and timing", "10,000; 100,000; 500,000; EOF snapshots within three full replays per
algorithm"],
    ], [WIDTH*.34, WIDTH*.66])
    p("The 1,000-event batch is also the FM/DGIM query interval. Bloom requires one membership "
      "query per event. Timed blocks include their updates and queries; parsing, comparison, "
      "CSV writing and memory inspection are outside those blocks. Approximate/exact comparisons "
      "are made within a task, rather than claiming the three different tasks have identical costs.")

    section("4. Demonstration of Concepts")
    p(f"At EOF, {final_n:,} valid requests have been processed. The same source-file order "
      "is used for both approximate and exact calculations. The table reports the first full "
      "replay with seed 42; repeated-run timing is reported separately.")
    table(["Concept", "Observed result", "Exact comparison"], [
        ["Bloom prior-host membership", f"{bloom['false_positives']} false positives; online FPR
{percent(bloom['relative_error'])}",
         f"{int(bloom['negative_queries']):,} genuinely unseen-host queries; {bloom['false_negatives']} false
negatives"],
        ["FM distinct hosts", f"Estimate {float(fm['estimate']):,.1f}; relative error
{percent(fm['relative_error'])}",
         f"{int(fm['exact']):,} distinct host strings"],
        ["DGIM recent errors", f"Estimate {float(dgim['estimate']):,.1f}; relative error
{percent(dgim['relative_error'])}",
         f"{int(dgim['exact']):,} errors in the final 10,000 requests"],
    ], [WIDTH*.27, WIDTH*.37, WIDTH*.36])
    figure("05_full_estimates", "Figure 2. Checkpoints every 1,000 requests plus EOF. FM is cumulative; DGIM
describes a moving request-count window.")
    p("Bloom's uncertain positive result is visible as a measured false-positive rate. FM's "
      "stepwise changes and possible overestimation are visible against exact cardinality. "
      "DGIM follows changes in recent errors with a much smaller summary than the full request "
      "history. The checkpoint CSVs retain the plotted numbers for inspection.")

    section("4. Demonstration: The Interactive Web Application")
    p("The web application turns the mini project into a working visual lab. It offers "
      "historical replay of the first 10,000, 100,000, 500,000 or all 1,891,714 valid requests. "
```

```
325         "This is live computation on historical records, not a connection to current NASA traffic.")
326     table(["Interface", "What it demonstrates"], [
327         ["Overview", "Current counts, exact-versus-approximate curves, source timestamps, recent real requests
    and logical summary payload."],
328         ["Replay controls", "Start, pause, resume, reset, and exactly one event per Step. Speed ranges from one
    to 100,000 requested events per second; compute capacity can limit it."],
329         ["Bloom state", "128 actual bit positions at a time, paging and jump-to-hash; current membership
    decision, prior exact membership and false positives."],
330         ["FM state", "The actual maximum-zero registers, powers-of-two evidence, group means, current estimate
    and relative error."],
331         ["DGIM state", "Power-of-two buckets ordered by recency; current merges/expirations and last 128 actual
    binary request events."],
332         ["Experiments", "The 63 recorded Python parameter cases, memory and throughput comparisons, CSV export
    and transparent FM error."],
333         ["Dataset and source", "Provenance, actual dates, response distribution, checksum, original request
    inspection, complete source explorer and project ZIP download."],
334     ], [WIDTH*.25, WIDTH*.75])
335     p("Changing a sketch parameter or hash seed restarts the replay. This prevents an existing "
336       "state from being labeled with settings that did not produce it. Changing speed affects "
337       "only pacing. Work is done in a worker so the controls and layout can remain responsive. "
338       "The sidebar becomes a mobile menu; cards and charts stack, while wide data tables scroll "
339       "inside their panels. Keyboard focus, accessible labels and reduced-motion behavior are included.")
340     code("python3 serve_web.py --open\n"
341          "# Or open output/StreamLab_Offline.html directly.\n"
342          "cd web\n"
343          "npm test")
344     p("Eight web tests include a full replay of every real event and the actual worker's "
345       "initialization, step, playback and one-event-per-second pacing. The JavaScript implementation "
346       "matches Python exactly: 81,982 distinct hosts, 27 Bloom false positives, FM estimate "
347       "442,368, DGIM estimate 28 versus 30 exact recent errors, and 10,980 total HTTP errors. "
348       "Browser checks also verified initial loading, one-step processing and a 100,000-event replay.")
349     p("Live memory badges show logical payload only. The browser has additional input/hash "
350       "caches, JavaScript overhead and exact state. In particular, the standalone HTML embeds "
351       "compressed inputs and is larger than the sketch state. The Experiments view labels "
352       "measured Python state separately. A hosting source push was blocked by DNS restrictions "
353       "in the task environment; the delivered app works locally and offline.", "SmallCustom")
354
355     section("5. Evaluation: Bloom Filter Parameters")
356     figure("02_bloom", "Figure 3. Online FPR across nine settings. Error bars span the five fixed seeds; they
    are not confidence intervals.")
357     groups = defaultdict(list)
358     for r in sweep:
359         if r["algorithm"] == "bloom":
360             groups[(int(r["bits"]), int(r["hashes"]))].append(r)
361     table(["Bit payload", "Hashes", "Mean online FPR", "Min-max FPR"], [
362         [f"{m//8//1024} KiB", k, percent(statistics.mean(float(r['relative_error']) for r in rs)),
363          f"{percent(min(float(r['relative_error']) for r in rs))} - {percent(max(float(r['relative_error']) for
    r in rs))}"]
364         for (m, k), rs in sorted(groups.items())], [WIDTH*.22, WIDTH*.15, WIDTH*.28, WIDTH*.35])
365     p("At fixed k, the larger array spreads the inserted hosts over more bits and reduces "
366       "collisions. Hash count is not a universal accuracy knob: the best k depends on m/d. "
367       "If the filter becomes crowded, setting more positions can increase saturation. When the "
368       "array is very large relative to the 7,752 distinct hosts in this prefix, some observed "
369       "rates are zero. That means no false positives were observed in these finite tests; it "
370       "does not prove a zero underlying probability.")
371     p("The prefix sweep should not be mistaken for the full-month occupancy. The default "
372       "full-stream filter receives many more distinct hosts. Its memory stays fixed while "
373       "the risk of false positives grows. No false negatives were observed in the evaluated "
374       "runs, consistent with the insertion-only design.")
375
376     section("5. Evaluation: FM Variability")
377     figure("03_fm_parameters", "Figure 4. Per-seed final errors and maintained state on the first 100,000
    requests. The nonlinear error scale preserves outliers.")
378     fm_rows = []
379     for k in [1, 16, 64]:
380         rs = [r for r in sweep if r["algorithm"] == "fm" and int(r["hashes"]) == k]
381         errs = [float(r["relative_error"]) for r in rs]
382         fm_rows.append([k, percent(statistics.mean(errs)), percent(min(errs)), percent(max(errs)),
383                         f"{statistics.mean(float(r['state_bytes']) for r in rs):,.0f}"])
384     table(["Hashes", "Mean error", "Minimum", "Maximum", "State bytes"], fm_rows,
385           [WIDTH*.13, WIDTH*.23, WIDTH*.21, WIDTH*.21, WIDTH*.22])
386     p("A single unusually long zero run can dominate 2^R, which explains the large variation "
387       "for a single hash. Grouping multiple estimates changes the distribution but arithmetic "
388       "means remain sensitive to extreme estimates. The results show the actual errors rather "
389       "than claiming that increasing H guarantees improvement. These groups and seeds were "
390       "fixed before inspecting outcomes; no seed was selected to make a favorable plot.")
391     p(f"The default full-stream estimate is {float(fm['estimate']):,.1f} against "
392       f"{int(fm['exact']):,} exact hosts. Its error is {percent(fm['relative_error'])}. "
393       "This is an important practical limitation of the uncalibrated teaching estimator. "
394       "It demonstrates how a tiny hash summary operates, but this experiment does not support "
395       "using it for a strict-accuracy production counter. A separately evaluated bitmap FM or "
396       "a modern cardinality sketch would be a useful extension, not a result claimed here.")
397
398     section("5. Evaluation: DGIM Window Size")
399     figure("04_dgim_parameters", "Figure 5. Mean checkpoint error and final state at the 100,000-request prefix.
     The ideal packed exact line is payload only.")
```

```
    drows = [r for r in sweep if r["algorithm"] == "dgim"]
    table(["Window", "Mean rel. error", "Final exact / est.", "DGIM bytes", "Exact bytes"], [
        [f"{int(r['window']):,}", percent(r['mean_checkpoint_relative_error']),
         f"{r['exact']} / {float(r['estimate']):.1f}", r['state_bytes'], r['exact_state_bytes']]
        for r in drows], [WIDTH*.17, WIDTH*.23, WIDTH*.24, WIDTH*.18, WIDTH*.18])
    p("A larger window includes more old events and can require buckets at more scales. "
      "Its exact byte buffer grows linearly with N, while the bucket count grows slowly. "
      "Error does not have to grow monotonically with the window: it depends on how recent "
      "ones fall around the oldest bucket boundary. The N=100,000 run at this prefix is a "
      "startup case with no expired data, so its zero error is expected and is not evidence "
      "of exact behavior after the window starts sliding.")
    p("Mean relative error is the unweighted mean across the 1,000-request checkpoints, "
      "including zero-error empty windows. Absolute errors are also stored. For a zero true "
      "count, a zero estimate is recorded as zero error; a nonzero estimate would have an "
      "undefined relative error and would not silently be divided by zero. Small true counts "
      "can turn a small absolute error into a large percentage.")
    p("At small N, Python tuple/list overhead can outweigh the theoretical saving compared "
      "with an efficiently packed exact ring. This is why the figure includes the ideal "
      "N/8 exact payload. Claims of space efficiency must state which representation is used.")

    section("5. Evaluation: Memory and Processing Cost")
    figure("06_full_tradeoffs", "Figure 6. Default parameters at EOF. Timing excludes parsing and validation; throughput uses the median of three full replays.")
    table(["Task", "Approx. state", "Exact state", "Approx. events/s", "Exact events/s"], [
        [kind.upper(), f"{int(full[kind]['state_bytes']):,}", f"{int(full[kind]['exact_state_bytes']):,}",
         f"{float(timings[kind]['throughput']):,.0f}", f"{float(timings[kind]['exact_throughput']):,.0f}"]
        for kind in ["bloom", "fm", "dgim"]],
        [WIDTH*.16, WIDTH*.21, WIDTH*.21, WIDTH*.21, WIDTH*.21])
    p("Memory is the recursively measured reachable state using sys.getsizeof, with each "
      "object counted once per algorithm. It includes arrays, keys, containers and Python "
      "overhead. It is final maintained state, not peak process RSS or a claim about total "
      "application memory. Temporary allocations, interpreter code and the shared input "
      "batch are excluded consistently. Exact states are measured independently.")
    p("Python's exact set uses an optimized built-in hash table. Bloom and FM perform several "
      "BLAKE2b calculations per event and execute their loops in Python, so their smaller "
      "state need not lead to higher throughput. DGIM processes zeros cheaply, and only "
      "about 0.58% of requests are errors; that sparsity helps this implementation. Results "
      "are workload- and machine-dependent, rather than a general ranking of the concepts.")
    p(f"Measured environment: Python {escape(metadata['python'].split()[0])}; "
      f"{escape(metadata['platform'])}; architecture {escape(metadata['machine'])}. "
      "Raw timings, minimum/maximum cumulative durations and all repetitions are included "
      "in scaling_raw.csv and scaling_summary.csv.", "SmallCustom")

    section("5. Evaluation: Scalability and Reproducibility")
    figure("07_scaling", "Figure 7. Prefix snapshots within three full replays. Window size stays fixed; distinct-host exact state grows as new hosts appear.")
    p("Bloom state is fixed by m and k, and FM state by H. They summarize all history. DGIM "
      "depends on its fixed recent window and the current bucket pattern. The exact distinct "
      "set grows with distinct hosts, not directly with request count; many repeated requests "
      "can arrive without adding any keys. The exact circular buffer remains fixed once N is chosen.")
    p("Validation", "Sub")
    p("Fifteen unittest cases cover Bloom membership and saturation; query-before-insert; "
      "FM empty streams, duplicate/order invariance and an independent trailing-zero "
      "calculation; parser failures and valid-row limits; and DGIM invariants, expiry, "
      "alternation, sparse input and randomized streams. DGIM is also checked exhaustively "
      "on every 9-bit sequence for six window lengths, against an independently summed history. "
      "The tests verify the 50% positive-count error bound on these fixtures. Artificial "
      "fixtures are used only for tests, not for the experimental results.")
    p("Reproduction and limitations", "Sub")
    code("python main.py --mode demo\n"
         "python -m unittest discover -s tests -v\n"
         "python main.py --mode experiments --profile\n"
         "python generate_report.py")
    p("The ZIP includes the compressed real dataset, exact parameter choices, five seeds, "
      "raw results and plotting/report scripts. Deterministic estimates reproduce on the "
      "same input, whereas timings and Python object sizes can vary across systems. Only "
      "one historical server workload is evaluated. Error sparsity, repeated hosts, the "
      "capture's partial-month coverage and Python implementation overhead limit generalization.")
    p("The evidence supports compact summary states, while exposing their task-specific "
      "costs: Bloom can misclassify new hosts, this FM variant can be substantially inaccurate, "
      "and DGIM trades exact recent counts for bucket-based estimates. None of the "
      "three summaries replaces the full log when individual request retrieval is required.")

    section("6. Code Appendix")
    p("Complete project source follows, including the three core algorithms, replay "
      "and evaluation code, web interface, worker, browser algorithms, data preparation, "
      "offline bundler, figures, report generator, download/packaging helpers and tests. "
      "The matching standalone files in the ZIP can be executed directly. Wrapped continuation "
      "lines in this appendix retain the original source line number; the source files remain authoritative.")
    source_paths = sorted(ROOT.glob("*.py")) + sorted((ROOT / "tests").glob("*.py"))
    source_paths += sorted(p for p in (ROOT / "web" / "dist").glob("*")
                           if p.suffix in {".js", ".html", ".css", ".svg"})
    source_paths += sorted((ROOT / "web" / "tests").glob("*.mjs"))
    for index, path in enumerate(source_paths):
        story.append(PageBreak())
        p(str(path.relative_to(ROOT)), "Sub")
```

```
        numbered = []
        for n, line in enumerate(path.read_text(encoding="utf-8").splitlines(), 1):
            # Courier at 6.6pt permits 120 characters in the printable width.
            wrapped = textwrap.wrap(line.expandtabs(4), width=112, replace_whitespace=False,
                drop_whitespace=False) or [""]
            numbered.append(f"{n:4}  " + wrapped[0])
            numbered.extend("      " + continuation for continuation in wrapped[1:])
        story.append(Preformatted("\n".join(numbered), styles["CodeCustom"]))

    section("7. References")
    refs = [
        "[1] Internet Traffic Archive. NASA-HTTP: Two Months of HTTP Logs from the KSC-NASA WWW Server. "
        "Collected by Jim Dumoulin; contributed by Martin Arlitt and Carey Williamson. "
        "https://ita.ee.lbl.gov/html/contrib/NASA-HTTP.html (accessed 23 October 2026).",
        "[2] Bloom, B. H. (1970). Space/time trade-offs in hash coding with allowable errors. "
        "Communications of the ACM, 13(7), 422-426. https://doi.org/10.1145/362686.362692.",
        "[3] Leskovec, J., Rajaraman, A., and Ullman, J. D. Mining of Massive Datasets, "
        "Chapter 4: Mining Data Streams. Author-hosted materials: https://www.mmds.org/; "
        "chapter: https://infolab.stanford.edu/~ullman/mmds/ch4.pdf.",
        "[4] Flajolet, P., and Martin, G. N. (1985). Probabilistic counting algorithms for data base "
        "applications. Journal of Computer and System Sciences, 31(2), 182-209. "
        "https://doi.org/10.1016/0022-0000(85)90041-8.",
        "[5] Datar, M., Gionis, A., Indyk, P., and Motwani, R. (2002). Maintaining stream statistics "
        "over sliding windows. SIAM Journal on Computing, 31(6), 1794-1813. "
        "https://doi.org/10.1137/S0097539701398363.",
    ]
    for ref in refs:
        p(escape(ref))
    p("Data provenance and integrity", "Sub")
    p("Original download: " + escape(manifest["source_url"]), "SmallCustom")
    p("SHA-256 (compressed source file):", "SmallCustom")
    code(manifest["sha256"][:32] + "\n" + manifest["sha256"][32:])
    p("The included references.bib provides a reusable bibliography. Results and narrative "
      "are specific to this project's measured runs; external sources provide the dataset "
      "provenance and algorithm foundations.")
    REPORT.parent.mkdir(parents=True, exist_ok=True)
    doc = SimpleDocTemplate(str(REPORT), pagesize=A4, rightMargin=48, leftMargin=48,
        topMargin=53, bottomMargin=53, title="Efficient Monitoring of Website Activity Using Data Stream
Algorithms",
        author=NAME, subject="CSD 5113 Assignment 2 - Mining Data Streams")
    doc.build(story, onFirstPage=frame, onLaterPages=frame)
    print(REPORT)


if __name__ == "__main__":
    main()
```

main.py

```
"""Run an offline demo, full replay, or all reproducible experiments."""
import argparse
import json
from pathlib import Path

from stream import DATA, ROOT, profile, write_json
from experiments import run_case, run_experiments, write_csv


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--mode", choices=["demo", "full", "experiments"], default="demo")
    parser.add_argument("--data", type=Path, default=DATA)
    parser.add_argument("--profile", action="store_true", help="audit the full source file first")
    args = parser.parse_args()
    if not args.data.is_file():
        parser.error("Data file missing. See README.md / download_data.py.")
    if args.profile:
        print("Profiling source data...", flush=True)
        write_json(ROOT / "data" / "manifest.json", profile(args.data))
    if args.mode == "experiments":
        rows = run_experiments(args.data)
        from plots import make_plots
        make_plots()
    else:
        rows = []
        for kind in ["bloom", "fm", "dgim"]:
            row, history, _ = run_case(kind, limit=10_000 if args.mode == "demo" else None,
                                       path=args.data, capture=True)
            rows.append(row)
            write_csv(ROOT / "results" / args.mode / f"{kind}_checkpoints.csv", history)
        write_csv(ROOT / "results" / args.mode / "summary.csv", rows)
    print(json.dumps(rows, indent=2))


if __name__ == "__main__":
    main()
```

package_submission.py

```
"""Validate saved evidence and create the Blackboard submission ZIP."""
import csv
from datetime import datetime, timezone
import json
from pathlib import Path
import subprocess
import sys
import zipfile

from stream import ROOT, DATA, checksum, write_json

STEM = "481013386_MohammedKhalifah_5113_Assignment_2"
REPORT_NAME = "481013386_MohammedKhalifah_5113_Assignment_1.pdf"


def load_rows(name):
    with (ROOT / "results" / name).open() as f:
        return list(csv.DictReader(f))


def validate():
    manifest = json.loads((ROOT / "data" / "manifest.json").read_text())
    if checksum(DATA) != manifest["sha256"]:
        raise ValueError("The dataset checksum changed")
    sweep = load_rows("parameter_sweep.csv")
    full = {r["algorithm"]: r for r in load_rows("full_summary.csv")}
    raw, summary = load_rows("scaling_raw.csv"), load_rows("scaling_summary.csv")
    assert len(sweep) == 63, "Expected all 63 parameter/seed cases"
    assert len(raw) == 36 and len(summary) == 12, "Missing repetitions or prefix snapshots"
    assert set(full) == {"bloom", "fm", "dgim"}
    for kind in full:
        assert int(full[kind]["events"]) == manifest["valid"]
        assert int(full[kind]["skipped_lines"]) == manifest["skipped"]
    for kind in ["bloom", "fm"]:
        assert int(full[kind]["exact"]) == manifest["exact_hosts"]
    assert int(full["bloom"]["negative_queries"]) == manifest["exact_hosts"]
    for r in sweep + list(full.values()):
        if r["algorithm"] == "bloom":
            assert int(r["false_negatives"]) == 0
    for r in load_rows("dgim_checkpoints.csv"):
        truth, estimate = float(r["exact"]), float(r["estimate"])
        assert abs(truth - estimate) <= 0.5 * truth
    for r in summary:
        assert int(r["repetitions"]) == 3
        repetitions = [v for v in raw if v["algorithm"] == r["algorithm"] and v["events"] == r["events"]]
        assert len({v["estimate"] for v in repetitions}) == 1
        assert len({v["exact"] for v in repetitions}) == 1
        assert len({v["relative_error"] for v in repetitions}) == 1
    figures = list((ROOT / "results" / "figures").glob("*.png"))
    assert len(figures) == 7
    result = subprocess.run([sys.executable, "-m", "unittest", "discover", "-s", "tests", "-v"],
        cwd=ROOT, text=True, stdout=subprocess.PIPE, stderr=subprocess.STDOUT)
    (ROOT / "results" / "test_results.txt").write_text(result.stdout)
    if result.returncode:
        raise RuntimeError(result.stdout)
    web_result = subprocess.run(["node", "--test", "tests/algorithms.test.mjs"], cwd=ROOT / "web",
        text=True, stdout=subprocess.PIPE, stderr=subprocess.STDOUT)
    (ROOT / "results" / "web_test_results.txt").write_text(web_result.stdout)
    if web_result.returncode:
        raise RuntimeError(web_result.stdout)
    evidence = dict(validated_at_utc=datetime.now(timezone.utc).isoformat(),
        dataset_sha256=manifest["sha256"], valid_requests=manifest["valid"],
        parameter_cases=len(sweep), scaling_raw_rows=len(raw), scaling_summary_rows=len(summary),
        figures=len(figures), correctness_tests="15 passed", deterministic_repetitions=True,
        dgim_full_checkpoint_error_bound="passed", bloom_false_negatives=0,
        web_correctness_tests="8 passed", browser_python_full_stream_agreement=True)
    write_json(ROOT / "results" / "validation.json", evidence)
    return evidence


def main():
    evidence = validate()
    report = ROOT / "output" / "pdf" / REPORT_NAME
    if not report.is_file():
        raise FileNotFoundError("Generate the report first: python generate_report.py")
    paths = sorted(ROOT.glob("*.py"))
    paths += [ROOT / name for name in ["README.md", "WEB_README.md", "ORAL_NOTES_AR.md", "requirements.txt",
"references.bib"]]
    for folder in ["data", "results", "tests"]:
        paths += sorted(p for p in (ROOT / folder).rglob("*") if p.is_file()
                        and "__pycache__" not in p.parts and p.suffix not in {".pyc", ".part"})
    paths += sorted(p for p in (ROOT / "web").rglob("*") if p.is_file()
                    and not any(part in {".git", ".openai", "node_modules", "downloads", "__pycache__"}
```

```
                                for part in p.relative_to(ROOT / "web").parts))
    destination = ROOT / "output" / (STEM + ".zip")
    with zipfile.ZipFile(destination, "w", compression=zipfile.ZIP_DEFLATED, compresslevel=6) as archive:
        for path in paths:
            archive.write(path, str(Path(STEM) / path.relative_to(ROOT)))
        archive.write(report, str(Path(STEM) / REPORT_NAME))
    with zipfile.ZipFile(destination) as archive:
        if archive.testzip() is not None:
            raise ValueError("ZIP integrity check failed")
        assert f"{STEM}/{REPORT_NAME}" in archive.namelist()
        count = len(archive.namelist())
    print(json.dumps(evidence, indent=2))
    print(f"Created {destination} ({destination.stat().st_size:,} bytes; {count} files)")


if __name__ == "__main__":
    main()
```

plots.py

```
"""Create static research figures directly from the saved experiment CSVs."""
from collections import defaultdict
import csv
import json
import os
from pathlib import Path
import statistics

from stream import ROOT


def read_csv(name):
    with (ROOT / "results" / name).open(newline="") as f:
        return list(csv.DictReader(f))


def make_plots():
    os.environ.setdefault("MPLCONFIGDIR", str(ROOT / "tmp" / "matplotlib"))
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    import numpy as np
    plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 10,
        "axes.spines.top": False, "axes.spines.right": False,
        "axes.titleweight": "bold", "figure.dpi": 120, "savefig.dpi": 180})
    out = ROOT / "results" / "figures"
    out.mkdir(parents=True, exist_ok=True)
    teal, orange, blue = "#087F8C", "#D68035", "#4059AD"

    def save(fig, name):
        fig.tight_layout(pad=1.5)
        fig.savefig(out / (name + ".png"), bbox_inches="tight", facecolor="white")
        plt.close(fig)

    manifest = json.loads((ROOT / "data" / "manifest.json").read_text())
    days = sorted(manifest["daily_counts"], key=lambda s: int(s[:2]))
    fig, ax = plt.subplots(figsize=(9, 3.3))
    ax.bar([int(d[:2]) for d in days], [manifest["daily_counts"][d] for d in days], color=teal)
    ax.set(xlabel="Day of July 1995", ylabel="Valid HTTP requests",
           title="Observed source-file coverage: 1-28 July 1995")
    ax.ticklabel_format(style="plain", axis="y")
    save(fig, "01_dataset")

    rows = read_csv("parameter_sweep.csv")
    fig, ax = plt.subplots(figsize=(8.5, 3.8))
    for k, color in zip([2, 4, 7], [orange, teal, blue]):
        xs, means, lo, hi = [], [], [], []
        for bits in [65_536, 262_144, 1_048_576]:
            group = [100 * float(r["relative_error"]) for r in rows
                     if r["algorithm"] == "bloom" and int(r["bits"]) == bits and int(r["hashes"]) == k]
            mean = statistics.mean(group)
            xs.append(bits / 8 / 1024); means.append(mean)
            lo.append(mean - min(group)); hi.append(max(group) - mean)
        ax.errorbar(xs, means, yerr=[lo, hi], marker="o", capsize=4, color=color,
                    label=f"{k} hashes")
    ax.set(xlabel="Packed Bloom bit-array size (KiB)", ylabel="Online false-positive rate (%)",
           title="Bloom filter: first 100,000 valid requests")
    ax.set_xscale("log", base=2)
    ax.set_xticks([8, 32, 128], ["8", "32", "128"])
    ax.legend(title="Mean and min-max of 5 seeds")
    ax.grid(axis="y", alpha=.2)
    save(fig, "02_bloom")

    fig, axes = plt.subplots(1, 2, figsize=(9, 3.7))
    for x, k in enumerate([1, 16, 64]):
        group = [r for r in rows if r["algorithm"] == "fm" and int(r["hashes"]) == k]
        errors = [100 * float(r["relative_error"]) for r in group]
        axes[0].scatter([x + v for v in np.linspace(-.09, .09, len(group))], errors,
                        color=teal, s=30, label="Individual seed" if x == 0 else None)
        axes[0].hlines(statistics.mean(errors), x-.2, x+.2, color=orange, linewidth=2,
                       label="Mean of five seeds" if x == 0 else None)
        axes[1].bar(x, statistics.mean(float(r["state_bytes"]) for r in group), color=blue)
    axes[0].set(xticks=[0, 1, 2], xticklabels=[1, 16, 64], xlabel="Hash functions",
                ylabel="Final relative error (%)", title="FM variation across five seeds")
    axes[0].set_yscale("symlog", linthresh=10)
    axes[0].set_ylim(bottom=0)
    axes[0].legend(loc="lower right", fontsize=7)
    axes[1].set(xticks=[0, 1, 2], xticklabels=[1, 16, 64], xlabel="Hash functions",
                ylabel="Reachable Python state (bytes)", title="More hashes require more state")
    save(fig, "03_fm_parameters")

    dgim_rows = [r for r in rows if r["algorithm"] == "dgim"]
    x = [int(r["window"]) for r in dgim_rows]
```

```
    fig, axes = plt.subplots(1, 2, figsize=(9, 3.6))
    axes[0].plot(x, [100*float(r["mean_checkpoint_relative_error"]) for r in dgim_rows], "o-", color=teal)
    axes[0].set(ylabel="Mean checkpoint relative error (%)", title="DGIM error by window size")
    for field, label, color, style in [("state_bytes", "DGIM Python state", teal, "o-"),
            ("exact_state_bytes", "Exact byte-buffer state", orange, "s-"),
            ("exact_packed_window_bytes", "Ideal exact bit payload", blue, "^--")]:
        axes[1].plot(x, [float(r[field]) for r in dgim_rows], style, color=color, label=label)
    axes[1].set(yscale="log", ylabel="Bytes (log scale)", title="Implementation overhead matters")
    axes[1].legend(fontsize=7)
    for ax in axes:
        ax.set_xscale("log"); ax.set_xlabel("Window length in requests")
        ax.grid(axis="y", alpha=.2)
    save(fig, "04_dgim_parameters")

    fig, axes = plt.subplots(2, 1, figsize=(9, 6), sharex=True)
    for kind, ax, title in [("fm", axes[0], "Distinct hosts: FM versus exact set"),
                            ("dgim", axes[1], "HTTP errors in the last 10,000 requests")]:
        history = read_csv(f"{kind}_checkpoints.csv")
        ns = [int(r["events"]) / 1_000_000 for r in history]
        ax.plot(ns, [float(r["exact"]) for r in history], color=teal, label="Exact", linewidth=1.4)
        ax.plot(ns, [float(r["estimate"]) for r in history], color=orange, label=kind.upper(),
                linewidth=1.1, alpha=.9)
        ax.set(title=title, ylabel="Count")
        ax.legend(loc="upper left", ncol=2)
        ax.grid(alpha=.2)
    axes[1].set_xlabel("Valid requests processed (millions)")
    save(fig, "05_full_estimates")

    scaling = read_csv("scaling_summary.csv")
    maximum = max(int(r["events"]) for r in scaling)
    full = [next(r for r in scaling if r["algorithm"] == kind and int(r["events"]) == maximum)
            for kind in ["bloom", "fm", "dgim"]]
    fig, axes = plt.subplots(1, 2, figsize=(9, 3.8))
    pos = np.arange(3)
    for ax, approx_field, exact_field, ylabel, title in [
            (axes[0], "state_bytes", "exact_state_bytes", "Bytes (log scale)", "Final maintained state"),
            (axes[1], "throughput", "exact_throughput", "Events/second (log scale)", "Median of three full
replays")]:
        ax.bar(pos-.18, [float(r[approx_field]) for r in full], .36, color=teal, label="Approximate")
        ax.bar(pos+.18, [float(r[exact_field]) for r in full], .36, color=orange, label="Exact")
        ax.set(xticks=pos, xticklabels=["Bloom", "FM", "DGIM"], yscale="log", ylabel=ylabel, title=title)
        ax.legend(fontsize=8)
    save(fig, "06_full_tradeoffs")

    fig, axes = plt.subplots(1, 3, figsize=(10, 3.5))
    for kind, ax in zip(["bloom", "fm", "dgim"], axes):
        group = [r for r in scaling if r["algorithm"] == kind]
        ns = [int(r["events"]) for r in group]
        ax.plot(ns, [float(r["state_bytes"]) for r in group], "o-", color=teal, label="Approximate")
        ax.plot(ns, [float(r["exact_state_bytes"]) for r in group], "s-", color=orange, label="Exact")
        ax.set(xscale="log", yscale="log", title=kind.upper(), xlabel="Requests processed", ylabel="State
bytes")
        ax.grid(alpha=.2)
    axes[0].legend(fontsize=7)
    save(fig, "07_scaling")
    print("Saved seven figures to", out)


if __name__ == "__main__":
    make_plots()
```

prepare_web.py

```
"""Prepare real NASA records and deterministic hash inputs for the browser lab.

The browser still updates every sketch on every event. Precomputed keyed hash
inputs keep the UI fast and make JS answers identical to the Python experiment.
"""
import csv
import gzip
import hashlib
import json
import shutil
import struct
import zipfile
from datetime import datetime
from pathlib import Path

from stream import DATA, ROOT, events, checksum

DEST = ROOT / "web" / "dist"
SEEDS = [7, 19, 42, 73, 101]
STRIDE = 92  # seven low uint32 hash words (28 bytes), 64 trailing-zero bytes


def export_results():
    """Use the last complete saved submission if a new sweep is in progress."""
    out = DEST / "data"
    recorded = {}
    archive = ROOT / "output" / "481013386_MohammedKhalifah_5113_Assignment_2.zip"
    prefix = archive.stem + "/results/"
    for name in ["full_summary", "parameter_sweep", "scaling_summary"]:
        raw = (ROOT / "results" / (name + ".csv")).read_text()
        rows = list(csv.DictReader(raw.splitlines()))
        expected = {"full_summary": 3, "parameter_sweep": 63, "scaling_summary": 12}[name]
        if len(rows) != expected:
            with zipfile.ZipFile(archive) as previous:
                raw = previous.read(prefix + name + ".csv").decode()
                rows = list(csv.DictReader(raw.splitlines()))
        assert len(rows) == expected, f"Incomplete {name} evidence"
        recorded[name] = rows
        (out / (name + ".csv")).write_text(raw)
    recorded["run_metadata"] = json.loads((ROOT / "results" / "run_metadata.json").read_text())
    (out / "benchmarks.json").write_text(json.dumps(recorded, separators=(",", ":")))


def compressed_json(path, value):
    with gzip.GzipFile(filename=str(path), mode="wb", mtime=0) as f:
        f.write(json.dumps(value, separators=(",", ":"), ensure_ascii=True).encode())


def main():
    out = DEST / "data"
    out.mkdir(parents=True, exist_ok=True)
    manifest = json.loads((ROOT / "data" / "manifest.json").read_text())
    assert checksum(DATA) == manifest["sha256"]
    hosts, host_ids, requests, request_ids = [], {}, [], {}
    stats = {}
    first = None
    with gzip.GzipFile(filename=str(out / "events.bin.gz"), mode="wb", mtime=0) as target:
        for count, event in enumerate(events(stats=stats), 1):
            if event.host not in host_ids:
                host_ids[event.host] = len(hosts)
                hosts.append(event.host)
            if event.request not in request_ids:
                request_ids[event.request] = len(requests)
                requests.append(event.request)
            timestamp = int(datetime.strptime(event.timestamp, "%d/%b/%Y:%H:%M:%S %z").timestamp())
            first = first or timestamp
            target.write(struct.pack("<5I", host_ids[event.host], timestamp - first,
                request_ids[event.request], event.status,
                event.bytes_sent if event.bytes_sent is not None else 0xFFFFFFFF))
    assert count == manifest["valid"] and len(hosts) == manifest["exact_hosts"]
    compressed_json(out / "catalog.json.gz", dict(hosts=hosts, requests=requests))
    for seed in SEEDS:
        keys = [(seed + i).to_bytes(8, "big") for i in range(64)]
        with gzip.GzipFile(filename=str(out / f"hashes-{seed}.bin.gz"), mode="wb", mtime=0) as target:
            for host in hosts:
                raw = host.encode()
                hashes = [int.from_bytes(hashlib.blake2b(raw, key=k, digest_size=8).digest(), "big") for k in
keys]
                low = [h & 0xFFFFFFFF for h in hashes[:7]]
                zeros = bytes((h & -h).bit_length() - 1 if h else 64 for h in hashes)
                target.write(struct.pack("<7I", *low) + zeros)
        print("Prepared hash inputs for seed", seed, flush=True)
    summary = dict(manifest, epoch=first, record_bytes=20, record_fields=["host_id", "elapsed_seconds",
```

```
        "request_id", "http_status", "bytes_or_0xFFFFFFFF"], hash_stride=STRIDE, seeds=SEEDS,
        hash_format="7 little-endian low uint32 words, then 64 trailing-zero counts; BLAKE2b-64",
        browser_note="Deterministic hashes are input preparation; all sketch updates and exact checks run in a Web Worker.",
        prepared_assets={p.name: dict(bytes=p.stat().st_size, sha256=checksum(p)) for p in out.glob("*.gz")})
    (out / "manifest.json").write_text(json.dumps(summary, indent=2) + "\n")
    export_results()
    print(f"Prepared {count:,} real requests, {len(hosts):,} hosts, {len(requests):,} request strings")


if __name__ == "__main__":
    main()
```

serve_web.py

```python
"""Serve the self-contained web project; no package installation is required."""
import argparse
from functools import partial
from http.server import SimpleHTTPRequestHandler, ThreadingHTTPServer
from urllib.parse import unquote
import threading
import webbrowser
from stream import ROOT


class ProjectHandler(SimpleHTTPRequestHandler):
    def translate_path(self, path):
        # Download aliases avoid including another copy of the raw data in dist.
        clean = path.split("?", 1)[0]
        report_name = "481013386_MohammedKhalifah_5113_Assignment_1.pdf"
        report = ROOT / "output" / "pdf" / report_name
        if not report.exists():
            report = ROOT / report_name  # The submission ZIP places it at root.
        aliases = {"/downloads/NASA_access_log_Jul95.gz": ROOT / "data" / "NASA_access_log_Jul95.gz",
            "/downloads/report.pdf": report}
        if clean in aliases:
            return str(aliases[clean])
        if clean.startswith("/downloads/results/"):
            target = (ROOT / "results" / unquote(clean[len("/downloads/results/"):])).resolve()
            if target.is_relative_to((ROOT / "results").resolve()):
                return str(target)
        return super().translate_path(path)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--port", type=int, default=5173)
    parser.add_argument("--host", default="127.0.0.1")
    parser.add_argument("--open", action="store_true")
    args = parser.parse_args()
    handler = partial(ProjectHandler, directory=str(ROOT / "web" / "dist"))
    with ThreadingHTTPServer((args.host, args.port), handler) as server:
        url = f"http://{args.host}:{args.port}"
        print(f"StreamLab is ready: {url}\nPress Ctrl+C to stop.", flush=True)
        if args.open:
            threading.Timer(0.5, webbrowser.open, args=[url]).start()
        try:
            server.serve_forever()
        except KeyboardInterrupt:
            pass


if __name__ == "__main__":
    main()
```

stream.py

```
"""Bounded-memory parsing and provenance for the NASA Common Log Format."""
from dataclasses import dataclass, asdict
from datetime import datetime
from collections import Counter
from pathlib import Path
import gzip
import hashlib
import json
import re

ROOT = Path(__file__).resolve().parent
DATA = ROOT / "data" / "NASA_access_log_Jul95.gz"
SOURCE_URL = "https://ita.ee.lbl.gov/traces/NASA_access_log_Jul95.gz"
SOURCE_PAGE = "https://ita.ee.lbl.gov/html/contrib/NASA-HTTP.html"
PATTERN = re.compile(r'^(\S+)\s+\S+\s+\S+\s+\[([^\]]+)\]\s+"(.*)"\s+(\d{3})\s+(\d+|-)\s*$')


@dataclass(frozen=True, slots=True)
class Event:
    host: str
    timestamp: str
    request: str
    status: int
    bytes_sent: int | None

    @property
    def error(self):
        return int(400 <= self.status <= 599)


def parse_line(line):
    m = PATTERN.match(line)
    if not m:
        return None
    host, timestamp, request, status, size = m.groups()
    try:
        datetime.strptime(timestamp, "%d/%b/%Y:%H:%M:%S %z")
    except ValueError:
        return None
    if not 100 <= int(status) <= 599:
        return None
    return Event(host, timestamp, request, int(status), None if size == "-" else int(size))


def events(path=DATA, limit=None, stats=None):
    """A generator; limit counts valid records, preserving source-file order."""
    if limit is not None and limit <= 0:
        raise ValueError("limit must be positive")
    stats = stats if stats is not None else {}
    stats.update(lines=0, valid=0, skipped=0, unknown_bytes=0, timestamp_inversions=0)
    opener = gzip.open if str(path).endswith(".gz") else open
    previous = None
    with opener(path, "rt", encoding="latin-1", newline="") as f:
        for line in f:
            stats["lines"] += 1
            event = parse_line(line.rstrip("\r\n"))
            if event is None:
                stats["skipped"] += 1
                continue
            # YYYYMMDDhhmmss can be compared lexicographically within July 1995.
            key = datetime.strptime(event.timestamp, "%d/%b/%Y:%H:%M:%S %z")
            stats["timestamp_inversions"] += int(previous is not None and key < previous)
            previous = key
            stats["valid"] += 1
            stats["unknown_bytes"] += int(event.bytes_sent is None)
            yield event
            if limit is not None and stats["valid"] >= limit:
                break


def checksum(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1024 * 1024), b""):
            h.update(chunk)
    return h.hexdigest()


def profile(path=DATA):
    stats, hosts, statuses, days = {}, set(), Counter(), Counter()
    first, last = None, None
    samples = []
    for e in events(path, stats=stats):
```

```
            hosts.add(e.host)
            statuses[str(e.status)] += 1
            days[e.timestamp[:11]] += 1
            first = first or e.timestamp
            last = e.timestamp
            if len(samples) < 5:
                samples.append(asdict(e))
        stats.update(source_url=SOURCE_URL, source_page=SOURCE_PAGE,
                     source_file=Path(path).name, sha256=checksum(path),
                     compressed_bytes=Path(path).stat().st_size,
                     exact_hosts=len(hosts), statuses=dict(statuses), daily_counts=dict(days),
                     errors=sum(n for s, n in statuses.items() if 400 <= int(s) <= 599),
                     first_timestamp=first, last_timestamp=last, sample_events=samples)
        return stats


def write_json(path, value):
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    Path(path).write_text(json.dumps(value, indent=2) + "\n", encoding="utf-8")
```

web_bundle.py

```
"""Sync web downloads/source explorer and build a completely offline HTML app."""
import base64
import json
import mimetypes
from pathlib import Path
import shutil
from stream import ROOT, DATA

DIST = ROOT / "web" / "dist"
REPORT_NAME = "481013386_MohammedKhalifah_5113_Assignment_1.pdf"


def sync():
    sources = {str(p.relative_to(ROOT)): p.read_text() for p in sorted(ROOT.glob("*.py"))}
    for parent, suffixes in [(ROOT / "tests", {".py"}), (ROOT / "web", {".md", ".json", ".mjs"}),
                             (DIST, {".js", ".html", ".css", ".svg"})]:
        for p in sorted(parent.rglob("*")):
            if p.is_file() and p.suffix in suffixes and not any(part in {".git", ".openai", "node_modules",
"downloads", "data", "__pycache__"} for part in p.relative_to(parent).parts):
                sources[str(p.relative_to(ROOT))] = p.read_text()
    (DIST / "data" / "sources.json").write_text(json.dumps(sources, separators=(",", ":")))
    downloads = DIST / "downloads"
    downloads.mkdir(exist_ok=True)
    shutil.copyfile(DATA, downloads / DATA.name)
    report = ROOT / "output" / "pdf" / REPORT_NAME
    if not report.exists():
        report = ROOT / REPORT_NAME
    if report.exists():
        shutil.copyfile(report, downloads / "report.pdf")
    files = [dict(name=name, content=value) for name, value in sources.items() if not
name.startswith("web/dist/")]
    for name in ["README.md", "WEB_README.md", "ORAL_NOTES_AR.md", "requirements.txt", "references.bib",
"data/README.md", "data/manifest.json"]:
        p = ROOT / name
        if p.exists():
            files.append(dict(name=name, content=p.read_text()))
    files.append(dict(name=f"data/{DATA.name}", url=f"./downloads/{DATA.name}"))
    if report.exists():
        files.append(dict(name=REPORT_NAME, url="./downloads/report.pdf"))
    result_dir = downloads / "results"
    result_dir.mkdir(exist_ok=True)
    for p in sorted((ROOT / "results").rglob("*")):
        if p.is_file() and p.suffix in {".csv", ".json", ".txt", ".png"}:
            dest = result_dir / p.relative_to(ROOT / "results")
            dest.parent.mkdir(parents=True, exist_ok=True)
            shutil.copyfile(p, dest)
            files.append(dict(name=str(p.relative_to(ROOT)), url="./downloads/results/"+str(p.relative_to(ROOT /
 "results"))))
    # Include the index by URL; its finite contents do not recursively embed it.
    index = DIST / "data" / "project-files.json"
    index.write_text("[]")
    for p in sorted(DIST.rglob("*")):
        if p.is_file() and "downloads" not in p.relative_to(DIST).parts:
            files.append(dict(name="web/dist/"+str(p.relative_to(DIST)), url="./"+str(p.relative_to(DIST))))
    index.write_text(json.dumps(files, separators=(",", ":")))
    return files


def offline():
    assets = {}
    for p in DIST.rglob("*"):
        if not p.is_file() or p.name in {"index.html", "app.js", "styles.css", "worker.js", "algorithms.js",
"download.js"}:
            continue
        mime = mimetypes.guess_type(p.name)[0] or "application/octet-stream"
        assets[str(p.relative_to(DIST))] = "data:"+mime+"; base64,"+base64.b64encode(p.read_bytes()).decode()
    # These code files must remain downloadable in a generated project ZIP.
    for name in ["index.html", "app.js", "styles.css", "worker.js", "algorithms.js", "download.js"]:
        assets[name] = "data:text/plain; base64,"+base64.b64encode((DIST / name).read_bytes()).decode()
    worker_assets = {name: value for name, value in assets.items() if name.startswith("data/") and
name.endswith(".gz") and "catalog" not in name}
    fetch_adapter = """const originalFetch = self.fetch.bind(self); self.fetch = (input, options) => {
      const path = String(input).replace(/^\\.\\//, '');
      return originalFetch(ASSETS[path] || input, options);
    };"""
    engine = (DIST / "algorithms.js").read_text().replace("export ", "")
    worker = (DIST / "worker.js").read_text().replace("import {StreamEngine} from './algorithms.js';", "")
    worker = worker.replace("if(data.type==='init') return init(data.config);",
        "if(data.type==='init') { ASSETS = data.assets || ASSETS; return init(data.config); }")
    worker_code = "let ASSETS={};"+fetch_adapter+engine+worker
    app = (DIST / "app.js").read_text().replace("import {downloadProject} from './download.js';", "")
    download = (DIST / "download.js").read_text().replace("export ", "")
    worker_line = "new Worker('./worker.js',{type:'module'})"
```

```
    replacement = "new Worker(URL.createObjectURL(new
Blob(["+json.dumps(worker_code)+"],{type:'text/javascript'})))"
    assert worker_line in app
    app = app.replace(worker_line, replacement)
    app = app.replace("worker.postMessage({type:'init',config});",
        "worker.postMessage({type:'init',config,assets:WORKER_ASSETS});")
    worker_asset_keys = list(worker_assets)
    app = "const WORKER_ASSETS=Object.fromEntries("+json.dumps(worker_asset_keys)+".map(k=>[k,ASSETS[k]]));"+app
    link_adapter = """for (const link of document.querySelectorAll('[data-embedded-href]')) {

originalFetch(ASSETS[link.dataset.embeddedHref]).then(r=>r.blob()).then(b=>{link.href=URL.createObjectURL(b);});
    }"""
    bundle = "const ASSETS="+json.dumps(assets)+";"+fetch_adapter+link_adapter+download+app
    html = (DIST / "index.html").read_text()
    html = html.replace('<link rel="icon" href="./favicon.svg" type="image/svg+xml">', "")
    html = html.replace('<link rel="stylesheet" href="./styles.css">', "<style>"+(DIST /
"styles.css").read_text()+"</style>")
    html = html.replace('<script type="module" src="./app.js"></script>', "<script>"+bundle+"</script>")
    # Native document links need embedded URLs in a file:// session too.
    html = html.replace('href="./downloads/report.pdf"', 'href="#" data-embedded-href="downloads/report.pdf"')
    html = html.replace('href="./data/parameter_sweep.csv"', 'href="#" data-embedded-
href="data/parameter_sweep.csv"')
    target = ROOT / "output" / "StreamLab_Offline.html"
    target.parent.mkdir(parents=True, exist_ok=True)
    target.write_text(html)
    print(f"Offline app: {target} ({target.stat().st_size:,} bytes)")


if __name__ == "__main__":
    sync()
    offline()
```

tests/test_algorithms.py

```
"""Deterministic correctness checks; synthetic fixtures are tests, not results."""
import itertools
import random
import tempfile
from pathlib import Path
import unittest

from bloom_filter import BloomFilter
from flajolet_martin import FlajoletMartin
from dgim import DGIM, ExactWindow
from stream import parse_line, events


class BloomTests(unittest.TestCase):
    def test_membership_and_no_false_negatives(self):
        b = BloomFilter(1024, 4)
        self.assertFalse(b.contains("alpha"))
        for i in range(300):
            b.add(str(i))
        self.assertTrue(all(b.contains(str(i)) for i in range(300)))

    def test_query_before_insertion_and_reproducibility(self):
        a, b = BloomFilter(64, 7), BloomFilter(64, 7)
        for item in ["a", "b", "a", "c"]:
            previous = a.contains(item)
            self.assertEqual(a.check_and_add(item), previous)
            b.add(item)
        self.assertEqual(a.array, b.array)
        self.assertTrue(a.check_and_add("a"))

    def test_saturation_still_has_no_false_negatives(self):
        b = BloomFilter(1, 2)
        b.add("a")
        self.assertTrue(b.contains("a"))
        self.assertTrue(b.contains("unseen"))

    def test_invalid_parameters(self):
        for bits, hashes in [(0, 1), (10, 0), (-1, 3)]:
            with self.assertRaises(ValueError):
                BloomFilter(bits, hashes)


class FMTests(unittest.TestCase):
    def test_empty_and_duplicate_invariance(self):
        f = FlajoletMartin()
        self.assertEqual(f.estimate(), 0)
        f.add("host")
        registers = bytes(f.max_zeros)
        estimate = f.estimate()
        for _ in range(100):
            f.add("host")
        self.assertEqual(registers, bytes(f.max_zeros))
        self.assertEqual(estimate, f.estimate())

    def test_matches_independent_trailing_zero_calculation(self):
        import hashlib
        f = FlajoletMartin(16, 73)
        values = [f"host-{i}" for i in range(100)]
        for v in values:
            f.add(v)
        expected = []
        for key in f.keys:
            bits = [format(int.from_bytes(hashlib.blake2b(v.encode(), key=key,
                    digest_size=8).digest(), "big"), "064b") for v in values]
            expected.append(max(len(s) - len(s.rstrip("0")) for s in bits))
        self.assertEqual(list(f.max_zeros), expected)

    def test_order_invariance_and_fixed_memory(self):
        a, b = FlajoletMartin(64), FlajoletMartin(64)
        for i in range(300):
            a.add(str(i))
        for i in reversed(range(300)):
            b.add(str(i))
        self.assertEqual(a.max_zeros, b.max_zeros)
        self.assertEqual(len(a.max_zeros), 64)

    def test_invalid_hash_count(self):
        with self.assertRaises(ValueError):
            FlajoletMartin(3)


class DGIMTests(unittest.TestCase):
    def check_stream(self, bits, window):
```

```
        sketch, exact, history = DGIM(window), ExactWindow(window), []
        for bit in bits:
            sketch.add(bit)
            exact.add(bit)
            history.append(bit)
            truth = sum(history[-window:])
            self.assertEqual(exact.estimate(), truth)
            if truth == 0:
                self.assertEqual(sketch.estimate(), 0)
            self.assertLessEqual(abs(sketch.estimate() - truth), 0.5 * truth)
            sizes = [size for size, _ in sketch.buckets]
            ends = [end for _, end in sketch.buckets]
            self.assertEqual(sizes, sorted(sizes))
            self.assertEqual(ends, sorted(ends, reverse=True))
            self.assertTrue(all(s > 0 and s & (s - 1) == 0 for s in sizes))
            self.assertTrue(all(sizes.count(s) <= 2 for s in set(sizes)))
            self.assertTrue(all(end > sketch.time - window for end in ends))

    def test_exhaustive_short_sequences(self):
        for window in [1, 2, 3, 5, 8, 16]:
            for bits in itertools.product([0, 1], repeat=9):
                self.check_stream(bits, window)

    def test_zeros_ones_alternation_and_sparse_events(self):
        for bits in [[0] * 200, [1] * 200, [0, 1] * 100,
                     [1] + [0] * 198 + [1]]:
            for window in [1, 10, 100]:
                self.check_stream(bits, window)

    def test_random_stream_and_boundary_expiration(self):
        rng = random.Random(101)
        self.check_stream([int(rng.random() < 0.17) for _ in range(2000)], 97)
        d = DGIM(4)
        for b in [1, 0, 0, 0]:
            d.add(b)
        self.assertEqual(d.estimate(), 1)
        d.add(0)
        self.assertEqual(d.estimate(), 0)

    def test_invalid_parameters(self):
        with self.assertRaises(ValueError):
            DGIM(0)
        with self.assertRaises(ValueError):
            DGIM().add(2)


class ParserTests(unittest.TestCase):
    LINE = 'host - - [01/Jul/1995:00:00:01 -0400] "GET / HTTP/1.0" 404 -'

    def test_real_format_and_unknown_bytes(self):
        e = parse_line(self.LINE)
        self.assertEqual(e.host, "host")
        self.assertEqual(e.error, 1)
        self.assertIsNone(e.bytes_sent)
        self.assertEqual(parse_line(self.LINE.replace("404 -", "304 0")).error, 0)

    def test_reject_bad_records(self):
        for line in ["broken", self.LINE.replace("01/Jul", "32/Jul"),
                     self.LINE.replace("404 -", "999 1")]:
            self.assertIsNone(parse_line(line))

    def test_limit_counts_valid_rows_and_preserves_order(self):
        with tempfile.TemporaryDirectory() as directory:
            p = Path(directory) / "input.log"
            p.write_text("broken\n" + self.LINE + "\n" +
                         self.LINE.replace("host", "second") + "\n")
            stats = {}
            self.assertEqual([e.host for e in events(p, 1, stats)], ["host"])
            self.assertEqual(stats["skipped"], 1)
            self.assertEqual(stats["valid"], 1)


if __name__ == "__main__":
    unittest.main()
```

web/dist/algorithms.js

```
// The browser updates these summaries from real requests, never from canned counters.
export class Bloom {
  constructor(bits = 1048576, hashes = 7) {
    if (![65536, 262144, 1048576].includes(bits) || ![2,4,7].includes(hashes)) throw Error('Invalid Bloom parameters');
    this.bits = bits; this.hashes = hashes; this.array = new Uint8Array(bits / 8); this.setBits = 0;
  }
  checkAndAdd(words) {
    let present = true; const positions = [];
    for (let i = 0; i < this.hashes; i++) {
      const p = words[i] % this.bits, mask = 1 << (p % 8), index = p >>> 3;
      positions.push(p);
      if (!(this.array[index] & mask)) { present = false; this.setBits++; }
      this.array[index] |= mask;
    }
    return { present, positions };
  }
  bit(index) { return Number(Boolean(this.array[index >>> 3] & (1 << (index % 8)))); }
}

export class FM {
  constructor(hashes = 16) {
    if (![1,16,64].includes(hashes)) throw Error('Invalid FM hash count');
    this.hashes = hashes; this.registers = new Uint8Array(hashes);
    this.groups = {1:1,16:4,64:8}[hashes]; this.empty = true;
  }
  add(zeros) {
    this.empty = false;
    for (let j = 0; j < this.hashes; j++) this.registers[j] = Math.max(this.registers[j], zeros[j]);
  }
  means() {
    const out = [], width = this.hashes / this.groups;
    for (let i = 0; i < this.hashes; i += width) {
      let sum = 0;
      for (let j = i; j < i + width; j++) sum += 2 ** this.registers[j];
      out.push(sum / width);
    }
    return out;
  }
  estimate() {
    if (this.empty) return 0;
    const a = this.means().sort((x,y)=>x-y), m = a.length >>> 1;
    return a.length % 2 ? a[m] : (a[m-1] + a[m])/2;
  }
}

export class DGIM {
  constructor(window = 10000) {
    if (!Number.isInteger(window) || window <= 0) throw Error('Invalid window');
    this.window = window; this.time = 0; this.buckets = [];
  }
  add(bit, trace = false) {
    if (bit !== 0 && bit !== 1) throw Error('Expected binary event');
    this.time++; const actions = [];
    while (this.buckets.length && this.buckets.at(-1)[1] <= this.time - this.window) {
      const b = this.buckets.pop(); if (trace) actions.push(`Expired bucket of ${b[0]} at #${b[1]}`);
    }
    if (!bit) return actions;
    this.buckets.unshift([1,this.time]);
    if (trace) actions.push('Created a size-1 bucket');
    let size = 1;
    while (true) {
      const indexes = [];
      this.buckets.forEach((b,i)=>{if(b[0]===size) indexes.push(i);});
      if (indexes.length <= 2) break;
      const newer = indexes.at(-2), older = indexes.at(-1);
      this.buckets[newer] = [size*2,this.buckets[newer][1]];
      this.buckets.splice(older,1);
      if(trace) actions.push(`Merged the two oldest ${size} buckets into ${size*2}`);
      size *= 2;
    }
    return actions;
  }
  estimate() {
    if(!this.buckets.length) return 0;
    const sum = this.buckets.reduce((s,b)=>s+b[0],0), oldest = this.buckets.at(-1)[0];
    return this.time <= this.window || oldest === 1 ? sum : sum - oldest/2;
  }
}

export class ExactWindow {
  constructor(window) { this.array = new Uint8Array(window); this.index = 0; this.total = 0; }
  add(bit) { this.total += bit-this.array[this.index]; this.array[this.index] = bit; this.index =
```

```
    (this.index+1)%this.array.length; }
 83 }
 84
 85 export class StreamEngine {
 86   constructor(config, hashes) {
 87     this.config = config; this.hashes = new DataView(hashes); this.bloom = new Bloom(config.bits,config.k);
 88     this.fm = new FM(config.h); this.dgim = new DGIM(config.window); this.exact = new
    ExactWindow(config.window);
 89     this.seen = new Set(); this.count = 0; this.errors = 0; this.falsePositives = 0; this.falseNegatives = 0;
 90     this.repeats = 0; this.bytes = 0; this.unknownBytes = 0; this.statuses = {}; this.recentBits = [];
 91     this.last = null; this.actions = [];
 92   }
 93   process(record) {
 94     const [host,elapsed,request,status,bytes] = record, base = host*92;
 95     const words = Array.from({length:7},(_,j)=>this.hashes.getUint32(base+j*4,true));
 96     const zeros = new Uint8Array(this.hashes.buffer,base+28,64), prior = this.seen.has(host);
 97     const bloom = this.bloom.checkAndAdd(words);
 98     if (!prior && bloom.present) this.falsePositives++;
 99     if (prior && !bloom.present) this.falseNegatives++;
100     if (prior) this.repeats++;
101     this.seen.add(host); this.fm.add(zeros);
102     const bit = Number(status>=400 && status<=599);
103     const changes = this.dgim.add(bit,true); this.exact.add(bit);
104     if(changes.length) this.actions = changes.map(text=>({event:this.count+1,text}));
105     this.count++; this.errors += bit; this.statuses[status] = (this.statuses[status]||0)+1;
106     if(bytes===0xFFFFFFFF) this.unknownBytes++; else this.bytes += bytes;
107     this.recentBits.push(bit); if(this.recentBits.length>128) this.recentBits.shift();
108     this.last = {event:this.count,host,elapsed,request,status,bytes,prior,
109       bloomPresent:bloom.present,falsePositive:!prior&&bloom.present,positions:bloom.positions,
110       hashWords:words,zeros:Array.from(zeros.slice(0,this.config.h)),bit};
111     return this.last;
112   }
113   snapshot(offset=0) {
114     const fm = this.fm.estimate(), window = this.dgim.estimate();
115     return {count:this.count,unique:this.seen.size,repeats:this.repeats,errors:this.errors,bytes:this.bytes,
116       unknownBytes:this.unknownBytes,falsePositives:this.falsePositives,falseNegatives:this.falseNegatives,
117       fpr:this.seen.size?this.falsePositives/this.seen.size:0,fm,fmError:this.seen.size?Math.abs(fm-
    this.seen.size)/this.seen.size:0,
118       window,exactWindow:this.exact.total,windowError:this.exact.total?Math.abs(window-
    this.exact.total)/this.exact.total:0,
119
    occupancy:this.bloom.setBits/this.bloom.bits,registers:Array.from(this.fm.registers),means:this.fm.means(),
120       buckets:this.dgim.buckets.map(b=>[...b]),bits:Array.from({length:128},(_,i)=>this.bloom.bit(offset+i)),
121       recentBits:[...this.recentBits],last:this.last,actions:this.actions,statuses:{...this.statuses}};
122   }
123 }
```

web/dist/app.js

```
import {downloadProject} from './download.js';

const $ = id => document.getElementById(id), $$ = selector => document.querySelectorAll(selector);
const nf = new Intl.NumberFormat('en-US',{maximumFractionDigits:1});
const fmt = n => nf.format(Number(n)||0), pct = (n,d=1) => `${(Number(n)*100).toFixed(d)}%`;
const short = n => n>=1e6?`${(n/1e6).toFixed(n>=1e7?0:1)}m`:n>=1000?`${(n/1000).toFixed(n>=10000?0:1)}k`:fmt(n);
const bytes = n => n>=1048576?`${(n/1048576).toFixed(1)} MiB`:n>=1024?`${(n/1024).toFixed(n%1024?1:0)} KiB`:`${fmt(n)} B`;
const esc = value => String(value??'').replace(/[&<>"']/g,c=>({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;","'":'&#39;'}[c]));
const text = (id,value) => { const element=$(id); if(element)element.textContent=value; };
const icons = {
  dashboard:'<rect x="3" y="3" width="7" height="7" rx="1.5"/><rect x="14" y="3" width="7" height="7" rx="1.5"/><rect x="3" y="14" width="7" height="7" rx="1.5"/><rect x="14" y="14" width="7" height="7" rx="1.5"/>',
  layers:'<path d="m12 3 10 5-10 5L2 8l10-5Z"/><path d="m3 12 9 5 9-5M3 17l9 5 9-5"/>',
  chart:'<path d="M4 3v17h17M8 15l4-5 4 2 5-7"/>',database:'<ellipse cx="12" cy="5" rx="8" ry="3"/><path d="M4 5v14c0 4 16 4 16 0V5M4 12c0 4 16 4 16 0"/>',
  book:'<path d="M3 4h6c2 0 3 1 3 3v14c0-2-1-3-3-3H3V4Zm18 0h-6c-2 0-3 1-3 3v14c0-2 1-3 3-3h6V4Z"/>',
  code:'<path d="m7 6-6 6 6 6 6M17 6l6 6-6 6M14 3l-4 18"/>',document:'<path d="M13 2H5v20h14V8l-6-6Z"/><path d="M13 2v6h6M8 13h8M8 17h6"/>',
  external:'<path d="M14 3h7v7m0-7L10 14M10 4H4v16h16v-6"/>',download:'<path d="M12 3v12m-5-5 5 5 5-5M4 16v5h16v-5"/>',
  menu:'<path d="M4 6h16M4 12h16M4 18h16"/>',play:'<path d="m8 4 12 8-12 8V4Z"/>',pause:'<path d="M8 4v16M16 4v16"/>',
  step:'<path d="m5 5 10 7-10 7V5ZM19 4v16"/>',reset:'<path d="M3 10a9 9 0 1 1 2 9M3 3v7h7"/>',
  activity:'<path d="M2 12h4l3-8 5 16 3-8h5"/>',users:'<circle cx="9" cy="8" r="3"/><path d="M3 21v-3a6 6 0 0 1 12 0v3M16 5a3 3 0 0 1 0 6M18 14a6 6 0 0 1 3 5v2"/>',
  filter:'<path d="M3 4h18l-7 8v7l-4 2v-9L3 4Z"/>',window:'<rect x="3" y="3" width="18" height="18" rx="3"/><path d="M3 8h18M8 8v13"/>',
  info:'<circle cx="12" cy="12" r="9"/><path d="M12 11v6M12 7h.01"/>',check:'<path d="m5 12 4 4L20 5"/>',copy:'<rect x="8" y="8" width="13" height="13" rx="2"/><path d="M16 8V3H3v13h5"/>',
  flask:'<path d="M9 3h6M10 3v7L4 20c-.3 1 .3 1 1 1h14c.7 0 1.3 0 1-1l-6-10V3M8 15h8"/>'
};
function icon(name){return `<svg class="icon" viewBox="0 0 24 24" aria-hidden="true">${icons[name]||icons.activity}</svg>`;}
function installIcons(root=document){root.querySelectorAll('[data-icon]').forEach(e=>e.innerHTML=icon(e.dataset.icon));}
installIcons();

let manifest, catalog, benchmarks, worker, snapshot=null, state='loading', view='overview';
let history=[], feed=[], toastTimer, bitOffset=0, autoplay=false, sourceFiles=null;
let config={bits:1048576,k:7,h:16,window:10000,seed:42,limit:100000,rate:5000};
const colors={teal:'#178773',violet:'#a08acc',rose:'#cd9a8b',gray:'#b4bfc6',grid:'#edf1f2',text:'#a0adb2'};

function toast(message){text('toast',message);$('toast').hidden=false;clearTimeout(toastTimer);toastTimer=setTimeout(()=>$('toast').hidden=true,3200);}
function setState(next){
  state=next; const loading=next==='loading';
  $('play').disabled=loading; $('reset').disabled=loading;
$('step').disabled=loading||next==='running'||next==='completed';
  text('play-text',loading?'Preparing data…':next==='running'?'Pause replay':next==='completed'?'Replay again':snapshot?.count?'Resume replay':'Start replay');
  $('play').querySelector('[data-icon]').innerHTML=icon(next==='running'?'pause':'play');
  const label={loading:'Loading',ready:'Ready',running:'Replaying',paused:'Paused',completed:'Complete',error:'Unavailable'}[next]||next;
  $('replay-status').className=`replay-status ${next==='running'?'running':''}`;
  $('replay-status').lastElementChild.textContent=label;
  $('chart-live').className=`live-indicator ${next==='running'?'running':''}`;
  $('chart-live').lastChild.textContent=label;
}
function clock(elapsed,full=false){
  if(!manifest)return '—';
  const d=new Date((manifest.epoch+elapsed-4*3600)*1000);
  const time=d.toISOString().slice(11,19);
  return full?`Jul ${d.getUTCDate().toString().padStart(2,'0')}, ${time} · UTC−04:00`:time;
}
function resetReplay(shouldPlay=false){
  autoplay=shouldPlay;history=[];feed=[];snapshot=null;bitOffset=0;
  setState('loading');$('error-notice').hidden=true;
  worker.postMessage({type:'init',config});
  renderEmpty();
}
function renderEmpty(){
  ['metric-requests','metric-unique','metric-window','fp-count','fm-legend','exact-legend','window-legend','window-exact-legend','feed-event-number'].forEach(id=>text(id,'0'));
  text('fp-count','0 false positives');text('requests-percent','0% complete');text('metric-fpr','0.00');
  text('unique-comparison','0 exact · 0% error');text('window-comparison',`0 exact · last ${short(config.window)}`);
  text('progress-text',`0 / ${fmt(config.limit)} requests`);$('progress-fill').style.width='0%';
  $('stream-progress').setAttribute('aria-valuenow','0');$('stream-progress').setAttribute('aria-valuemax',String(config.limit));
  text('stream-clock','Ready when you are');text('compute-time','Worker compute: 0 ms');
```

```
  $('request-rows').innerHTML='<tr><td colspan="5" class="empty-table">Waiting for the first request…</td></tr>';
  $('audience-empty').hidden=false;$('error-empty').hidden=false;text('feed-count','0');
  ['request-wave','unique-wave','window-wave'].forEach(id=>$(id).setAttribute('d','M0 23 L120 23'));
  drawCharts();
}
function appendHistory(points){
  for(const point of points){if(history.at(-1)?.n===point.n)history[history.length-1]=point;else history.push(point);}
}
function currentPoints(){
  const result=[{n:0,exact:0,fm:0,window:0,exactWindow:0,fpr:0},...history];
  if(snapshot?.count && result.at(-1).n!==snapshot.count)result.push({n:snapshot.count,exact:snapshot.unique,fm:snapshot.fm,window:snapshot.window,exactWindow:snapshot.exactWindow,fpr:snapshot.fpr});
  return result;
}
function miniWave(id,field){
  const points=currentPoints().slice(-25),max=Math.max(1,...points.map(v=>v[field]));
  $(id).setAttribute('d',points.map((p,i)=>`${i?'L':'M'}${(i/Math.max(1,points.length-1)*120).toFixed(1)} ${(23-p[field]/max*20).toFixed(1)}`).join(' '));
}
function render(data){
  snapshot=data.snapshot;appendHistory(data.history||[]);feed=data.feed||feed;
  if(data.state)setState(data.state);
  const s=snapshot;
  text('metric-requests',fmt(s.count));text('metric-unique',fmt(s.fm));text('metric-window',fmt(s.window));
  text('metric-fpr',(s.fpr*100).toFixed(2));text('requests-percent',`${Math.floor(s.count/config.limit*100)}% complete`);
  text('unique-comparison',`${fmt(s.unique)} exact · ${pct(s.fmError)} error`);
  text('window-comparison',`${fmt(s.exactWindow)} exact · last ${short(config.window)}`);
  text('fp-count',`${fmt(s.falsePositives)} false positives`);text('bloom-hashes',`${config.k} hashes`);
  text('fm-legend',fmt(s.fm));text('exact-legend',fmt(s.unique));text('window-legend',fmt(s.window));text('window-exact-legend',fmt(s.exactWindow));
  const progress=Math.min(100,s.count/config.limit*100);$('progress-fill').style.width=`${progress}%`;
  $('stream-progress').setAttribute('aria-valuenow',String(s.count));text('progress-text',`${fmt(s.count)} / ${fmt(config.limit)} requests`);
  text('stream-clock',s.last?clock(s.last.elapsed,true):'Ready when you are');
  $('bloom-fill').style.width=`${s.occupancy*100}%`;
  text('bloom-summary',`${fmt(config.bits)} bits · ${pct(s.occupancy,1)} occupied`);text('bloom-payload',bytes(config.bits/8));
  text('fm-summary',`${config.h} trailing-zero registers`);text('fm-payload',`${config.h} B`);
  text('dgim-summary',`${s.buckets.length} buckets in the window`);text('dgim-payload',bytes(s.buckets.length*8));
  text('compute-time',`Worker compute: ${Number(data.computeMs||0).toFixed(0)} ms`);
  text('feed-event-number',fmt(s.count));text('feed-count',feed.length);renderFeed();
  $('audience-empty').hidden=s.count>0;$('error-empty').hidden=s.count>0;
  miniWave('request-wave','n');miniWave('unique-wave','fm');miniWave('window-wave','window');
  $$('[data-window-label]').forEach(e=>e.textContent=fmt(config.window));
  if(view==='algorithms')renderAlgorithms();
  drawCharts();
}
function renderFeed(){
  if(!feed.length)return;
  $('request-rows').innerHTML=[...feed].reverse().map(e=>{
    const request=catalog.requests[e.request],host=catalog.hosts[e.host],statusClass=e.status>=500?'server-error':e.status>=400?'error':e.status>=300?'redirect':'';
    return `<tr data-event="${e.event}" tabindex="0" role="button" aria-label="Inspect request ${e.event}"><td>${clock(e.elapsed)}</td><td title="${esc(host)}">${esc(host)}</td><td class="request-path" title="${esc(request)}">${esc(request.replace(/ HTTP\/\S+$/,''))}</td><td><span class="status-pill ${statusClass}">${e.status}</span></td><td><span class="seen-pill ${e.falsePositive?'fp':e.prior?'':'new'}">${e.falsePositive?'False positive':e.prior?'Seen before':'New host'}</span></td></tr>`;
  }).join('');
}
function showRequest(number){
  const e=feed.find(r=>r.event===number);if(!e)return;
  $('request-detail').innerHTML=`<dl class="request-detail-list"><div><dt>Stream event</dt><dd>#${fmt(e.event)}</dd></div><div><dt>Original timestamp</dt><dd>${clock(e.elapsed,true)}</dd></div><div><dt>Originating host</dt><dd><code>${esc(catalog.hosts[e.host])}</code></dd></div><div><dt>Original request</dt><dd><code>${esc(catalog.requests[e.request])}</code></dd></div><div><dt>Server response / bytes</dt><dd>${e.status} / ${e.bytes===0xFFFFFFFF?'Unknown':fmt(e.bytes)}</dd></div><div><dt>Bloom decision before insertion</dt><dd>${e.bloomPresent?'Possibly seen':'Definitely new'} · exact: ${e.prior?'seen before':'not previously seen'}${e.falsePositive?' · FALSE POSITIVE':''}</dd></div><div><dt>Actual hash bit positions</dt><dd><code>${e.positions.join(', ')}</code></dd></div><div><dt>DGIM input bit</dt><dd>${e.bit} · ${e.bit?'HTTP error':'non-error response'}</dd></div></dl>`;
  $('request-dialog').showModal();
}
function renderAlgorithms(){
  const s=snapshot;if(!s)return;
  const hits=new Set(s.last?.positions||[]);text('bit-range',`${fmt(bitOffset)}–${fmt(bitOffset+127)}`);
  $('bit-grid').innerHTML=s.bits.map((bit,i)=>`<span class="bit-cell ${bit?'set':''} ${hits.has(bitOffset+i)?'highlight':''}" title="Bit ${bitOffset+i}: ${bit}">${bit}</span>`).join('');
  text('bloom-formula-stats',`m = ${fmt(config.bits)} · k = ${config.k} · d = ${fmt(s.unique)}`);
  text('bloom-fp-detail',fmt(s.falsePositives));text('bloom-fn-detail',fmt(s.falseNegatives));
```

```
  text('bloom-host',s.last?catalog.hosts[s.last.host]:'No host yet');
  text('bloom-live-result',s.last?s.last.falsePositive?'False positive':s.last.bloomPresent?'Possibly seen':'Definitely new':'Waiting for a request');
  text('bloom-explanation',s.last?`Event #${fmt(s.count)}: ${s.last.prior?'the exact set has seen this host.':'this host is new to the exact set.'} Bloom returned ${s.last.bloomPresent?'possibly present':'definitely absent'} before insertion.`:'Each hash selects a bit. Any zero proves absence; all ones mean possible membership.');
  $('hash-positions').innerHTML=(s.last?.positions||[]).map((p,i)=>`<span title="Hash low32: 0x${s.last.hashWords[i].toString(16).padStart(8,'0')}">h${i+1} → ${fmt(p)}</span>`).join('');
  const max=Math.max(8,...s.registers);$('registers').classList.toggle('many',config.h===64);
  $('registers').innerHTML=s.registers.map((r,i)=>`<div class="register" title="Hash ${i+1}: maximum trailing zeros ${r}; 2^R = ${fmt(2**r)}${s.last?`; incoming zeros ${s.last.zeros[i]}`:''}"><span class="register-value">${r}</span><div class="register-bar-track"><div class="register-bar" style="height:${r/max*100}%"></div></div><span class="register-label">h${i+1}</span></div>`).join('');
  text('fm-state-count',`${config.h} registers`);text('fm-group-means',s.count?`Group means: ${s.means.map(fmt).join(' · ')}`:'Group means: waiting for data');
  text('fm-detail-estimate',fmt(s.fm));text('fm-detail-exact',fmt(s.unique));text('fm-detail-error',pct(s.fmError,2));
  text('bucket-count',`${s.buckets.length} buckets`);
  $('bucket-strip').innerHTML=s.buckets.length?s.buckets.map((b,i)=>{
    const partial=i===s.buckets.length-1&&s.count>config.window&&b[0]>1;
    return `<div class="bucket ${partial?'partial':''}" title="${b[0]} ones, newest one at event ${b[1]}; age ${s.count-b[1]} requests"><b>${b[0]}</b><span>#${short(b[1])}</span>${partial?'<small>½ weight</small>':''}</div>`;
  }).join(''):'<div class="visual-empty">An error request will create the first bucket.</div>';
  $('binary-stream').innerHTML=(s.recentBits.length?s.recentBits:Array(128).fill(null)).map((b,i)=>`<span class="binary-bit ${b===1?'one':''}" title="${b===null?'No request yet':`Event ${s.count-s.recentBits.length+i+1}: ${b}`}">${b??'·'}</span>`).join('');
  text('dgim-detail-estimate',fmt(s.window));text('dgim-detail-exact',fmt(s.exactWindow));
  $('dgim-actions').innerHTML=s.actions.length?s.actions.map(a=>`<div><small>#${fmt(a.event)}</small>${esc(a.text)}</div>`).join(''):'Waiting for a bucket creation, merge or expiry.';
}

function baseCanvas(id){
  const canvas=$(id);if(!canvas || canvas.offsetWidth===0)return null;
  const r=canvas.getBoundingClientRect(),dpr=Math.min(2,window.devicePixelRatio||1);
  canvas.width=Math.round(r.width*dpr);canvas.height=Math.round(r.height*dpr);
  const c=canvas.getContext('2d');c.scale(dpr,dpr);c.clearRect(0,0,r.width,r.height);c.font='9px -apple-system,BlinkMacSystemFont,Segoe UI,sans-serif';
  return {c,w:r.width,h:r.height,canvas};
}
function lineChart(id,series){
  const frame=baseCanvas(id);if(!frame)return;const {c,w,h}=frame;
  const p={l:43,r:12,t:17,b:35},pw=w-p.l-p.r,ph=h-p.t-p.b;
  const points=currentPoints(),xmax=Math.max(1,snapshot?.count||config.limit),rawMax=Math.max(1,...points.flatMap(d=>series.map(s=>d[s.key])));
  const order=10**Math.floor(Math.log10(rawMax)),ymax=Math.ceil(rawMax/order/0.5)*order*0.5;
  c.textAlign='right';c.fillStyle=colors.text;
  for(let i=0;i<=4;i++){const y=p.t+ph-i/4*ph;c.strokeStyle=colors.grid;c.beginPath();c.moveTo(p.l,y);c.lineTo(w-p.r,y);c.stroke();c.fillText(short(ymax*i/4),p.l-9,y+3);}
  c.textAlign='center';for(let i=0;i<=4;i++)c.fillText(short(xmax*i/4),p.l+pw*i/4,h-16);
  if(!snapshot?.count)return;
  for(const s of [...series].reverse()){
    if(s.area){const gradient=c.createLinearGradient(0,p.t,0,p.t+ph);gradient.addColorStop(0,s.color+'22');gradient.addColorStop(1,s.color+'00');c.beginPath();c.moveTo(p.l,p.t+ph);points.forEach(v=>c.lineTo(p.l+v.n/xmax*pw,p.t+ph-v[s.key]/ymax*ph));c.lineTo(p.l+pw,p.t+ph);c.closePath();c.fillStyle=gradient;c.fill();}
    c.beginPath();points.forEach((v,i)=>{const x=p.l+v.n/xmax*pw,y=p.t+ph-v[s.key]/ymax*ph;if(i)c.lineTo(x,y);else c.moveTo(x,y);});c.strokeStyle=s.color;c.lineWidth=1.8;c.stroke();
    const last=points.at(-1);c.beginPath();c.arc(p.l+last.n/xmax*pw,p.t+ph-last[s.key]/ymax*ph,2.8,0,Math.PI*2);c.fillStyle=s.color;c.fill();
  }
}
function barChart(id,rows,log=false){
  const frame=baseCanvas(id);if(!frame)return;const {c,w,h}=frame,p={l:49,r:15,t:16,b:38};
  const max=Math.max(1,...rows.flatMap(r=>[r.a,r.b??0])),top=log?Math.ceil(Math.log10(max)):max*1.1;
  const y=v=>p.t+(h-p.t-p.b)*(1-(log?(v>0?Math.log10(v):0):v)/top);
  c.textAlign='right';c.fillStyle=colors.text;
  const ticks=log?Array.from({length:Math.floor(top/2)+1},(_,i)=>i*2):[0,.25,.5,.75,1].map(v=>v*top);
  for(const tick of ticks){const yy=log?p.t+(h-p.t-p.b)*(1-tick/top):y(tick);c.beginPath();c.strokeStyle=colors.grid;c.moveTo(p.l,yy);c.lineTo(w-p.r,yy);c.stroke();c.fillStyle=colors.text;c.fillText(short(log?10**tick:tick),p.l-8,yy+3);}
  const available=w-p.l-p.r,step=available/rows.length;
  rows.forEach((r,i)=>{const groupX=p.l+step*(i+.5),width=Math.min(25,step*.23),paired=r.b!==undefined;
    c.fillStyle=colors.teal;c.beginPath();c.roundRect(groupX-(paired?width+3:width/2),y(r.a),width,Math.max(1,h-p.b-y(r.a)),[3,3,0,0]);c.fill();
    if(paired){c.fillStyle='#c3cfc8';c.beginPath();c.roundRect(groupX+3,y(r.b),width,Math.max(1,h-p.b-y(r.b)),[3,3,0,0]);c.fill();}
    c.textAlign='center';c.fillStyle=colors.text;c.font=`${rows.length>10?7:9}px -apple-system,sans-serif`;c.fillText(r.label,groupX,h-17);
  });
}
function drawCharts(){
  if(view==='overview'){
    lineChart('audience-chart',[{key:'fm',color:colors.violet,area:true},{key:'exact',color:colors.teal}]);
```

```
    lineChart('error-chart',[{key:'window',color:colors.rose,area:true},{key:'exactWindow',color:colors.gray}]);
  }
  if(view==='experiments'&&benchmarks){
    const max=Math.max(...benchmarks.scaling_summary.map(r=>Number(r.events)));
    const full=benchmarks.scaling_summary.filter(r=>Number(r.events)===max);
    barChart('memory-chart',full.map(r=>({label:{bloom:'Bloom',fm:'FM',dgim:'DGIM'}[r.algorithm],a:+r.state_bytes,b:+r.exact_state_bytes})),true);
    barChart('throughput-chart',full.map(r=>({label:{bloom:'Bloom',fm:'FM',dgim:'DGIM'}[r.algorithm],a:+r.throughput,b:+r.exact_throughput})),true);
  }
  if(view==='dataset'&&manifest){const days=Object.keys(manifest.daily_counts).sort((a,b)=>parseInt(a)-parseInt(b));barChart('daily-chart',days.map(d=>({label:String(parseInt(d)),a:manifest.daily_counts[d]})));}
}
function renderBenchmarks(){
  $('full-results').innerHTML=benchmarks.full_summary.map(r=>`<tr><td>${{bloom:'Bloom filter',fm:'Flajolet–Martin',dgim:'DGIM'}[r.algorithm]}</td><td>${r.algorithm==='bloom'?`${fmt(r.exact)} distinct hosts`:r.algorithm==='fm'?`${fmt(r.exact)} distinct hosts`:`${fmt(r.exact)} recent errors`}</td><td>${r.algorithm==='bloom'?`${r.false_positives} false positives`:`${fmt(r.estimate)}`}</td><td>${pct(r.relative_error,2)} ${r.algorithm==='bloom'?'online FPR':'relative error'}</td><td>${bytes(+r.state_bytes)}</td></tr>`).join('');
  const filter=$('experiment-filter').value;
  $('parameter-results').innerHTML=benchmarks.parameter_sweep.filter(r=>filter==='all'||r.algorithm===filter).map(r=>{
    const label=r.algorithm==='bloom'?`${fmt(r.bits)} bits / ${r.hashes} hashes`:r.algorithm==='fm'?`${r.hashes} hashes`:`${fmt(r.window)} requests`;
    const error=r.algorithm==='dgim'?r.mean_checkpoint_relative_error:r.relative_error;
    return `<tr><td>${{bloom:'Bloom',fm:'FM',dgim:'DGIM'}[r.algorithm]}</td><td>${label}</td><td>${r.algorithm==='dgim'?'—':r.seed}</td><td>${fmt(r.exact)}</td><td>${r.algorithm==='bloom'?`${r.false_positives} FP`:fmt(r.estimate)}</td><td>${pct(error,2)}</td><td>${bytes(+r.state_bytes)}</td></tr>`;
  }).join('');
}
function renderDataset(){
  text('dataset-checksum',manifest.sha256);
  $('status-distribution').innerHTML=Object.entries(manifest.statuses).sort((a,b)=>b[1]-a[1]).map(([code,n])=>`<div class="status-row"><span>${code}</span><div class="status-bar"><i style="width:${Math.max(.8,n/manifest.valid*100)}%"></i></div><b>${fmt(n)}</b></div>`).join('');
}
const views={overview:['Overview','A little memory. A lot of insight.','Follow real website activity as three algorithms turn a stream into a signal.'],algorithms:['Algorithm lab','See what the algorithms remember.','Every bit, register and bucket below is driven by the requests you replay.'],experiments:['Experiments','Measure the trade-off.','Accuracy, memory and processing time, measured on the original NASA stream.'],dataset:['The dataset','Start with something real.','A transparent source, an honest capture, and every event accounted for.']};
function navigate(){
  const candidate=location.hash.slice(1);view=views[candidate]?candidate:'overview';
  $$('[data-view]').forEach(e=>e.hidden=e.dataset.view!==view);
  $$('[data-nav]').forEach(e=>{e.classList.toggle('active',e.dataset.nav===view);if(e.dataset.nav===view)e.setAttribute('aria-current','page');else e.removeAttribute('aria-current');});
  const [crumb,title,description]=views[view];text('breadcrumb-current',crumb);text('page-title',title);text('page-description',description);
  document.title=`${crumb} · StreamLab`;closeNavigation();
  if(view==='algorithms')renderAlgorithms();requestAnimationFrame(drawCharts);
}
function closeNavigation(){document.querySelector('.app-shell').classList.remove('nav-open');$('nav-backdrop').hidden=true;$('menu-toggle').setAttribute('aria-expanded','false');}
window.addEventListener('hashchange',navigate);window.addEventListener('resize',()=>requestAnimationFrame(drawCharts));
$('menu-toggle').addEventListener('click',()=>{const open=document.querySelector('.app-shell').classList.toggle('nav-open');$('menu-toggle').setAttribute('aria-expanded',String(open));$('nav-backdrop').hidden=!open;});
$('nav-backdrop').addEventListener('click',closeNavigation);
$('play').addEventListener('click',()=>{
  if(state==='running'){worker.postMessage({type:'pause'});setState('paused');}
  else if(state==='completed')resetReplay(true);
  else {worker.postMessage({type:'play'});setState('running');}
});
$('step').addEventListener('click',()=>worker.postMessage({type:'step'}));
$('reset').addEventListener('click',()=>{resetReplay();toast('Replay reset. The next request will be the first genuine record.');});
$('speed').addEventListener('change',()=>{config.rate=+$('speed').value;worker?.postMessage({type:'speed',rate:config.rate});});
$('limit').addEventListener('change',()=>{config.limit=+$('limit').value;resetReplay();toast('Stream size changed. Replay restarted.');});
for(const [id,key] of [['bits-setting','bits'],['k-setting','k'],['h-setting','h'],['window-setting','window'],['seed-setting','seed']])$(id).addEventListener('change',()=>{config[key]=+$(id).value;resetReplay();toast('New parameters applied. Start or step through the new replay.');});
function moveBits(offset){bitOffset=Math.max(0,Math.min(config.bits-128,offset));worker?.postMessage({type:'bits',offset:bitOffset});}
$('bits-prev').addEventListener('click',()=>moveBits(bitOffset-128));$('bits-next').addEventListener('click',()=>moveBits(bitOffset+128));
$('bits-jump').addEventListener('click',()=>{if(snapshot?.last)moveBits(Math.floor(snapshot.last.positions[0]/128)*128);else toast('Process a request to inspect its hash positions.');});
$('request-rows').addEventListener('click',e=>{const row=e.target.closest('[data-
```

```
event]');if(row)showRequest(+row.dataset.event);});
$('request-rows').addEventListener('keydown',e=>{if(e.key==='Enter'||e.key===' '){const
row=e.target.closest('[data-event]');if(row){e.preventDefault();showRequest(+row.dataset.event);}}});
$('open-guide').addEventListener('click',()=>{$('guide-dialog').showModal();closeNavigation();});
$$('[data-close]').forEach(button=>button.addEventListener('click',()=>$(button.dataset.close).close()));
$('experiment-filter').addEventListener('change',()=>{if(benchmarks)renderBenchmarks();});
async function copy(value){try{await navigator.clipboard.writeText(value);toast('Copied to
clipboard.');}catch{toast('Clipboard access is unavailable. Select and copy the text directly.');}}
$('copy-checksum').addEventListener('click',()=>manifest&&copy(manifest.sha256));
$('open-source').addEventListener('click',async()=>{
  $('source-dialog').showModal();closeNavigation();
  try{if(!sourceFiles){const r=await fetch('./data/sources.json');if(!r.ok)throw Error('Source files are
unavailable');sourceFiles=await r.json();$('source-file').innerHTML=Object.keys(sourceFiles).map(path=>`<option>
${esc(path)}</option>`).join('');}showSource();}catch(e){text('source-content',e.message);}
});
function showSource(){const path=$('source-file').value;text('source-content',sourceFiles[path]);text('source-
language',path.endsWith('.py')?'Python · complete source':path.endsWith('.js')?'JavaScript · complete
source':path);}
$('source-file').addEventListener('change',showSource);$('copy-
source').addEventListener('click',()=>sourceFiles&&copy(sourceFiles[$('source-file').value]));
$('download-project').addEventListener('click',async()=>{
  const button=$('download-project');button.disabled=true;toast('Preparing the complete project ZIP from its
real source files…');
  try{await downloadProject(message=>toast(message));toast('Project ZIP ready. Your browser will save the
download.');}catch(e){toast(`Download failed: ${e.message}`);}finally{button.disabled=false;}
});
$('audience-chart').addEventListener('mousemove',e=>{
  if(!snapshot?.count)return;const rect=e.currentTarget.getBoundingClientRect(),n=Math.max(0,(e.clientX-
rect.left-43)/(rect.width-55))*snapshot.count;
  const point=currentPoints().reduce((best,p)=>Math.abs(p.n-n)<Math.abs(best.n-n)?p:best);
  const tip=$('audience-tooltip');tip.hidden=false;tip.textContent=`#${fmt(point.n)} requests · FM
${fmt(point.fm)} · exact ${fmt(point.exact)}`;
  tip.style.left=`${Math.max(5,Math.min(rect.width-220,e.clientX-rect.left-80))}px`;tip.style.top='15px';
});
$('audience-chart').addEventListener('mouseleave',()=>$('audience-tooltip').hidden=true);

async function load(){
  try{
    if(!('Worker' in window)||!('DecompressionStream' in window))throw Error('Please use a current browser with
Web Workers and gzip stream support.');
    const [m,b,raw]=await
Promise.all([fetch('./data/manifest.json'),fetch('./data/benchmarks.json'),fetch('./data/catalog.json.gz')]);
    if(!m.ok||!b.ok||!raw.ok)throw Error('Dataset assets could not be loaded. Serve this folder with python3
serve_web.py.');
    [manifest,benchmarks]=await Promise.all([m.json(),b.json()]);
    catalog=await new Response(raw.body.pipeThrough(new DecompressionStream('gzip'))).json();
    worker=new Worker('./worker.js',{type:'module'});
    worker.addEventListener('error',event=>fail(event.message||'The processing worker could not start.'));
    worker.addEventListener('message',({data})=>{
      if(data.type==='ready'){render({snapshot:data.snapshot,history:[],feed:[],state:'ready',computeMs:0});if(a
utoplay){autoplay=false;worker.postMessage({type:'play'});setState('running');}}
      else if(data.type==='update')render(data);
      else if(data.type==='completed'){setState('completed');toast(`Replay complete: ${fmt(snapshot.count)} real
 requests processed.`);}
      else if(data.type==='loading'){setState('loading');}
      else if(data.type==='error')fail(data.message);
    });
    renderBenchmarks();renderDataset();resetReplay();
  }catch(error){fail(error.message);}
}
function fail(message){setState('error');text('error-notice',message);$('error-
notice').hidden=false;$('play').disabled=true;$('step').disabled=true;}
navigate();load();
```

web/dist/download.js

```
// A standards-compliant stored ZIP, built locally from the project's static files.
// Gzip inputs are already compressed, so storing them avoids a duplicate hosted ZIP.
const encoder = new TextEncoder();
const crcTable = Uint32Array.from({length:256},(_,n)=>{
  let c=n;for(let k=0;k<8;k++)c=c&1?0xedb88320^(c>>>1):c>>>1;return c>>>0;
});
export function crc32(bytes){let c=0xffffffff;for(const b of bytes)c=crcTable[(c^b)&255]^(c>>>8);return (c^0xffffffff)>>>0;}
export function zipFiles(files){
  const locals=[],central=[];let offset=0,centralLength=0;
  const stamp=new Date(),time=(stamp.getHours()<<11)|(stamp.getMinutes()<<5)|(stamp.getSeconds()>>>1);
  const date=((stamp.getFullYear()-1980)<<9)|((stamp.getMonth()+1)<<5)|stamp.getDate();
  for(const file of files){
    const name=encoder.encode(file.name),data=file.data,crc=crc32(data);
    const header=new Uint8Array(30+name.length),v=new DataView(header.buffer);
    v.setUint32(0,0x04034b50,true);v.setUint16(4,20,true);v.setUint16(6,0x800,true);
    v.setUint16(10,time,true);v.setUint16(12,date,true);v.setUint32(14,crc,true);
    v.setUint32(18,data.length,true);v.setUint32(22,data.length,true);v.setUint16(26,name.length,true);header.set(name,30);
    locals.push(header,data);
    const entry=new Uint8Array(46+name.length),c=new DataView(entry.buffer);
    c.setUint32(0,0x02014b50,true);c.setUint16(4,20,true);c.setUint16(6,20,true);c.setUint16(8,0x800,true);
    c.setUint16(12,time,true);c.setUint16(14,date,true);c.setUint32(16,crc,true);c.setUint32(20,data.length,true);c.setUint32(24,data.length,true);
    c.setUint16(28,name.length,true);c.setUint32(42,offset,true);entry.set(name,46);
    central.push(entry);centralLength+=entry.length;offset+=header.length+data.length;
  }
  const end=new Uint8Array(22),v=new DataView(end.buffer);v.setUint32(0,0x06054b50,true);
  v.setUint16(8,files.length,true);v.setUint16(10,files.length,true);v.setUint32(12,centralLength,true);v.setUint32(16,offset,true);
  return new Blob([...locals,...central,end],{type:'application/zip'});
}
export async function downloadProject(progress=()=>{}){
  const r=await fetch('./data/project-files.json');if(!r.ok)throw Error('Project file index is unavailable');
  const list=await r.json(),files=[];
  for(let i=0;i<list.length;i++){
    if(i%10===0)progress(`Collecting project files: ${i+1} / ${list.length}…`);
    if('content' in list[i]){
      files.push({name:`481013386_MohammedKhalifah_5113_Assignment_2/${list[i].name}`,data:encoder.encode(list[i].content)});
      continue;
    }
    const response=await fetch(list[i].url);if(!response.ok)throw Error(`Missing file: ${list[i].name}`);
    files.push({name:`481013386_MohammedKhalifah_5113_Assignment_2/${list[i].name}`,data:new Uint8Array(await response.arrayBuffer())});
  }
  const blob=zipFiles(files),url=URL.createObjectURL(blob),a=document.createElement('a');
  a.href=url;a.download='481013386_MohammedKhalifah_5113_Assignment_2.zip';a.click();
  setTimeout(()=>URL.revokeObjectURL(url),60000);
}
```

web/dist/favicon.svg

```
1  <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"><rect width="64" height="64" rx="17"
   fill="#0a7565"/><g fill="#fff"><rect x="14" y="27" width="6" height="19" rx="3"/><rect x="24" y="18" width="6"
   height="28" rx="3"/><rect x="34" y="24" width="6" height="22" rx="3"/><rect x="44" y="12" width="6" height="34"
   rx="3"/></g></svg>
```

web/dist/index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8"><meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="theme-color" content="#0a7565">
  <meta name="description" content="Watch real NASA web requests flow through Bloom filtering, Flajolet–Martin and DGIM. An interactive data stream mining lab.">
  <title>StreamLab · NASA Data Stream Explorer</title>
  <link rel="icon" href="./favicon.svg" type="image/svg+xml"><link rel="stylesheet" href="./styles.css">
</head>
<body>
<a class="skip-link" href="#main">Skip to the lab</a>
<div class="app-shell">
  <aside class="sidebar" id="sidebar" aria-label="Main navigation">
    <a class="brand" href="#overview"><span class="brand-mark"><i></i><i></i><i></i><i></i></span>StreamLab<span class="brand-dot">.</span></a>
    <span class="workspace-label">THE STREAMING WORKSPACE</span>
    <nav class="nav-list">
      <a href="#overview" data-nav="overview" class="nav-item active"><span data-icon="dashboard"></span>Overview<span class="nav-shortcut">01</span></a>
      <a href="#algorithms" data-nav="algorithms" class="nav-item"><span data-icon="layers"></span>Algorithm lab<span class="nav-shortcut">02</span></a>
      <a href="#experiments" data-nav="experiments" class="nav-item"><span data-icon="chart"></span>Experiments<span class="nav-shortcut">03</span></a>
      <a href="#dataset" data-nav="dataset" class="nav-item"><span data-icon="database"></span>The dataset<span class="nav-shortcut">04</span></a>
    </nav>
    <div class="sidebar-divider"></div><span class="workspace-label">YOUR PROJECT</span>
    <button class="nav-item" id="open-guide"><span data-icon="book"></span>How it works</button>
    <button class="nav-item" id="open-source"><span data-icon="code"></span>Explore the code</button>
    <a class="nav-item" href="./downloads/report.pdf" target="_blank" rel="noopener"><span data-icon="document"></span>Read the report<span data-icon="external" class="small-icon"></span></a>
    <div class="sidebar-bottom">
      <div class="source-note"><span class="source-spark">✦</span><strong>Real data. Real trade-offs.</strong><p>No generated traffic. Every point starts with a genuine NASA HTTP request.</p><a href="#dataset">Meet the dataset <span aria-hidden="true">■</span></a></div>
      <div class="profile"><span class="avatar">MK</span><div><strong>Mohammed Khalifah</strong><small>CSD 5113 · Assignment 2</small></div><span class="profile-dot"></span></div>
    </div>
  </aside>
  <button id="nav-backdrop" class="nav-backdrop" aria-label="Close navigation" hidden></button>
  <div class="main-shell">
    <header class="topbar">
      <div class="breadcrumb"><button class="icon-button mobile-menu" id="menu-toggle" aria-label="Open navigation" aria-expanded="false"><span data-icon="menu"></span></button><span class="breadcrumb-root">Workspace</span><span class="slash">/</span><strong id="breadcrumb-current">Overview</strong></div>
      <div class="topbar-right"><span class="historical-label"><i></i>Historical data, live computation</span><button class="button button-small button-white" id="download-project"><span data-icon="download"></span><span>Download project</span></button></div>
    </header>
    <main id="main" tabindex="-1">
      <div class="page-heading"><div><span class="eyebrow">MINING DATA STREAMS</span><h1 id="page-title">A little memory. A lot of insight.</h1><p id="page-description">Follow real website activity as three algorithms turn a stream into a signal.</p></div><a class="project-tag" href="#dataset">NASA HTTP · 1995 <span data-icon="external"></span></a></div>
      <div class="notice error-notice" id="error-notice" role="alert" hidden></div>
      <section class="replay-controller" aria-label="Stream playback controls">
        <div class="controller-top"><div class="play-group"><button id="play" class="button button-primary" disabled><span data-icon="play"></span><span id="play-text">Preparing data…</span></button><button id="step" class="button button-white" disabled title="Process exactly one real request"><span data-icon="step"></span><span class="step-label">Step</span></button><button id="reset" class="icon-button reset-button" aria-label="Reset replay" disabled><span data-icon="reset"></span></button></div>
          <div class="controller-options"><label>Replay speed<select id="speed"><option value="1">1 request / sec</option><option value="100">100 / sec</option><option value="1000">1,000 / sec</option><option value="5000" selected>5,000 / sec</option><option value="25000">25,000 / sec</option><option value="100000">100,000 / sec</option></select></label><span class="control-divider"></span><label>Stream size<select id="limit"><option value="10000">First 10,000</option><option value="100000" selected>First 100,000</option><option value="500000">First 500,000</option><option value="1891714">Full dataset</option></select></label></div>
          <span class="replay-status" id="replay-status"><i></i><span>Loading</span></span>
        </div>
        <div class="controller-bottom"><div class="progress-track" role="progressbar" id="stream-progress" aria-label="Requests processed" aria-valuenow="0" aria-valuemin="0" aria-valuemax="100000"><div id="progress-fill"></div></div><span id="progress-text">0 / 100,000 requests</span><span class="controller-help" id="stream-clock">Ready when you are</span></div>
      </section>

      <div data-view="overview">
        <section class="metric-grid" aria-label="Current stream measurements">
          <article class="metric-card"><div class="metric-top"><span>Requests processed</span><span class="metric-icon teal" data-icon="activity"></span></div><div class="metric-value" id="metric-requests">0</div><div class="metric-bottom"><span class="mini-pill neutral" id="requests-percent">0% complete</span><span>from the original log</span></div><svg class="mini-wave teal-wave" viewBox="0 0 120 25" aria-hidden="true"><path id="request-wave" d="M0 23 L120 23"/></svg></article>
```

```
51            <article class="metric-card"><div class="metric-top"><span>Distinct hosts</span><span class="metric-
    icon violet" data-icon="users"></span></div><div class="metric-value" id="metric-unique">0</div><div
    class="metric-bottom"><span class="mini-pill violet-pill">FM estimate</span><span id="unique-comparison">0 exact
     · 0% error</span></div><svg class="mini-wave violet-wave" viewBox="0 0 120 25" aria-hidden="true"><path
    id="unique-wave" d="M0 23 L120 23"/></svg></article>
52            <article class="metric-card"><div class="metric-top"><span>Bloom false positives</span><span
    class="metric-icon amber" data-icon="filter"></span></div><div class="metric-value"><span id="metric-
    fpr">0.00</span><span class="metric-unit">%</span></div><div class="metric-bottom"><span class="mini-pill amber-
    pill" id="fp-count">0 false positives</span><span id="bloom-hashes">7 hashes</span></div><div class="tiny-
    meter"><span id="bloom-fill"></span></div></article>
53            <article class="metric-card"><div class="metric-top"><span>Errors in recent window</span><span
    class="metric-icon rose" data-icon="window"></span></div><div class="metric-value" id="metric-
    window">0</div><div class="metric-bottom"><span class="mini-pill rose-pill">DGIM estimate</span><span
    id="window-comparison">0 exact · last 10k</span></div><svg class="mini-wave rose-wave" viewBox="0 0 120 25"
    aria-hidden="true"><path id="window-wave" d="M0 23 L120 23"/></svg></article>
54          </section>
55          <section class="chart-grid">
56            <article class="panel audience-panel"><div class="panel-heading"><div><h2>The audience, as it
    arrives</h2><p>Running distinct-host count · cumulative</p></div><span class="live-indicator" id="chart-
    live"><i></i>Ready</span></div><div class="chart-legend"><span><i class="legend-line violet-bg"></i>FM estimate
    <b id="fm-legend">0</b></span><span><i class="legend-line teal-bg"></i>Exact count <b id="exact-
    legend">0</b></span></div><div class="canvas-wrap"><canvas id="audience-chart" role="img" aria-label="Distinct
    hosts over processed requests"></canvas><div class="chart-empty" id="audience-empty"><span data-
    icon="activity"></span><strong>Your stream starts here</strong><span>Press Start replay to follow real
    requests.</span></div><div class="chart-tooltip" id="audience-tooltip" hidden></div></div><div class="panel-
    foot"><span data-icon="info"></span><span>Hosts are devices or addresses, not verified people. FM can
    substantially overestimate.</span></div></article>
57            <article class="panel error-panel"><div class="panel-heading"><div><h2>A window into recent
    errors</h2><p>HTTP 400–599 · last <span data-window-label>10,000</span> requests</p></div><span
    class="algorithm-tag rose-tag">DGIM</span></div><div class="chart-legend"><span><i class="legend-line rose-
    bg"></i>Estimate <b id="window-legend">0</b></span><span><i class="legend-line gray-bg"></i>Exact <b id="window-
    exact-legend">0</b></span></div><div class="canvas-wrap"><canvas id="error-chart" role="img" aria-
    label="Estimated and exact recent error counts"></canvas><div class="chart-empty" id="error-empty"><span data-
    icon="window"></span><strong>Nothing in the window yet</strong><span>Each request contributes a real 0 or
    1.</span></div></div><div class="panel-foot"><span data-icon="info"></span><span>The window moves by requests,
    not by seconds.</span></div></article>
58          </section>
59          <section class="lower-grid"><article class="panel feed-panel"><div class="panel-heading"><div><h2>The
    request stream <span class="count-badge" id="feed-count">0</span></h2><p>Newest first. Select a request to
    inspect it.</p></div><span class="tiny-label"><i class="dot teal-bg"></i>Real NASA records</span></div><div
    class="table-scroll"><table class="request-table"><thead><tr><th>TIME
    <small>UTC−04</small></th><th>HOST</th><th>REQUEST</th><th>STATUS</th><th>HOST SEEN?</th></tr></thead><tbody
    id="request-rows"><tr><td colspan="5" class="empty-table">Waiting for the first
    request…</td></tr></tbody></table></div><div class="panel-foot feed-foot"><span><span id="feed-event-
    number">0</span> events processed</span><span id="compute-time">Worker compute: 0 ms</span></div></article>
60            <article class="panel snapshot-panel"><div class="panel-heading"><div><h2>Small state. Big
    picture.</h2><p>Current summary payload</p></div><span data-icon="layers" class="muted-icon"></span></div><div
    class="payload-item"><span class="payload-dot teal-bg"></span><div><strong>Bloom filter</strong><span id="bloom-
    summary">1,048,576 bits · 0% occupied</span></div><b id="bloom-payload">128 KiB</b></div><div class="payload-
    item"><span class="payload-dot violet-bg"></span><div><strong>Flajolet–Martin</strong><span id="fm-summary">16
    trailing-zero registers</span></div><b id="fm-payload">16 B</b></div><div class="payload-item"><span
    class="payload-dot rose-bg"></span><div><strong>DGIM</strong><span id="dgim-summary">0 buckets in the
    window</span></div><b id="dgim-payload">0 B</b></div><div class="payload-note">Logical payload only; JavaScript
    object overhead and input caches are excluded.</div><a class="text-link" href="#algorithms">Look inside the
    algorithms <span aria-hidden="true">→</span></a></article></section>
61          <div class="replay-context"><span data-icon="check"></span>Historical requests. Actual computation. No
    simulated counters.<a href="#dataset">View provenance <span aria-hidden="true">■</span></a></div>
62        </div>
63
64        <div data-view="algorithms" hidden>
65          <div class="section-intro"><div><h2>Open the black box.</h2><p>Step through one request, or replay
    thousands. These are the actual live states.</p></div><span class="muted-small">Changing a setting restarts the
    replay.</span></div>
66          <div class="settings-strip"><label>Bloom bit array<select id="bits-setting"><option value="65536">65,536
     bits · 8 KiB</option><option value="262144">262,144 bits · 32 KiB</option><option value="1048576"
    selected>1,048,576 bits · 128 KiB</option></select></label><label>Bloom hashes<select id="k-setting"><option
    value="2">2 functions</option><option value="4">4 functions</option><option value="7" selected>7
    functions</option></select></label><label>FM hashes<select id="h-setting"><option value="1">1
    register</option><option value="16" selected>16 · 4 groups of 4</option><option value="64">64 · 8 groups of
    8</option></select></label><label>DGIM window<select id="window-setting"><option value="1000">1,000
    requests</option><option value="10000" selected>10,000 requests</option><option value="100000">100,000
    requests</option></select></label><label>Hash seed<select id="seed-
    setting"><option>7</option><option>19</option><option
    selected>42</option><option>73</option><option>101</option></select></label></div>
67          <article class="panel algorithm-panel"><div class="panel-heading"><div class="algorithm-title"><span
    class="algorithm-number teal">01</span><div><h2>Bloom filter <span class="subtle-
    badge">MEMBERSHIP</span></h2><p>Has this host appeared before this request?</p></div></div><span
    class="algorithm-tag teal-tag" id="bloom-live-result">Waiting for a request</span></div><div class="algorithm-
    grid"><div><div class="bit-toolbar"><span>Actual bit array <strong id="bit-
    range">0–127</strong></span><div><button class="small-control" id="bits-prev" aria-label="Previous 128
    bits">←</button><button class="small-control" id="bits-next" aria-label="Next 128 bits">→</button><button
    class="small-control" id="bits-jump">Jump to latest hash</button></div></div><div id="bit-grid" class="bit-grid"
     aria-label="128 actual Bloom bit positions"></div><div class="visual-legend"><span><i class="bit-legend
    off"></i>0 · clear</span><span><i class="bit-legend on"></i>1 · set</span><span><i class="bit-legend
    hit"></i>Latest hash position</span></div><div class="formula-box">p<sub>FP</sub> ≈ (1 −
    e<sup>−kd/m</sup>)<sup>k</sup><span id="bloom-formula-stats">m = 1,048,576 · k = 7 · d =
```

```
0</span></div></div><div class="trace-box"><span class="eyebrow">LATEST REQUEST TRACE</span><h3 id="bloom-
host">No host yet</h3><p id="bloom-explanation">Each hash selects a bit. Any zero proves absence; all ones mean
possible membership.</p><div id="hash-positions" class="hash-positions"></div><div class="trace-
metrics"><div><span>False positives</span><b id="bloom-fp-detail">0</b></div><div><span>False negatives</span><b
 id="bloom-fn-detail">0</b></div></div><small>Check first, then insert. The exact set verifies the
decision.</small></div></div></article>
68          <article class="panel algorithm-panel"><div class="panel-heading"><div class="algorithm-title"><span
class="algorithm-number violet">02</span><div><h2>Flajolet–Martin <span class="subtle-badge">DISTINCT
COUNT</span></h2><p>Estimate different hosts without retaining their identifiers.</p></div></div><span
class="algorithm-tag violet-tag" id="fm-state-count">16 registers</span></div><div class="algorithm-
grid"><div><div id="registers" class="register-grid" aria-label="Maximum trailing-zero registers"></div><div
class="visual-legend"><span><i class="legend-line violet-bg"></i>Maximum trailing zeros R</span><span>Hover a
register for its current estimate.</span></div><div class="formula-box">E<sub>j</sub> =
2<sup>R<sub>j</sub></sup>   →   group means   →   median<span id="fm-group-means">Group
means: waiting for data</span></div></div><div class="trace-box"><span class="eyebrow">APPROXIMATION,
EXPOSED</span><div class="comparison-pair"><div><span>FM estimate</span><b id="fm-detail-
estimate">0</b></div><div><span>Exact hosts</span><b id="fm-detail-exact">0</b></div></div><div class="accuracy-
line"><span>Relative error</span><strong id="fm-detail-error">0%</strong></div><p>This is the chapter's maximum-
zero teaching variant. Rare long zero runs can produce large overestimates.</p><small>Hash values are
deterministically prepared from the same keyed BLAKE2b inputs as Python. Registers update here, on every
request.</small></div></div></article>
69          <article class="panel algorithm-panel"><div class="panel-heading"><div class="algorithm-title"><span
class="algorithm-number rose">03</span><div><h2>DGIM <span class="subtle-badge">SLIDING
WINDOW</span></h2><p>Keep a small bucket summary of errors in the last N requests.</p></div></div><span
class="algorithm-tag rose-tag" id="bucket-count">0 buckets</span></div><div class="algorithm-grid"><div><div
class="bucket-axis"><span>Newest</span><span>Oldest · half-weighted after startup →</span></div><div id="bucket-
strip" class="bucket-strip"><div class="visual-empty">An error request will create the first
bucket.</div></div><div class="bits-caption">Last 128 actual request bits <span>1 = HTTP error · 0 = other
response</span></div><div id="binary-stream" class="binary-stream"></div><div class="formula-box">Count ≈ Σ
bucket sizes − ½ oldest bucket<span>Startup and size-one boundary buckets are counted
exactly.</span></div></div><div class="trace-box"><span class="eyebrow">INSIDE THE WINDOW</span><div
class="comparison-pair"><div><span>DGIM estimate</span><b id="dgim-detail-estimate">0</b></div><div><span>Exact
errors</span><b id="dgim-detail-exact">0</b></div></div><div id="dgim-actions" class="action-log">Waiting for a
bucket creation, merge or expiry.</div><small>At most two buckets of each power-of-two size. Zeros advance the
window too.</small></div></div></article>
70        </div>
71
72        <div data-view="experiments" hidden>
73          <div class="recorded-banner"><span data-icon="flask"></span><div><strong>The evidence behind the
algorithms.</strong><p>Recorded Python experiments on the original dataset. These are separate from your browser
 replay.</p></div><a class="button button-white button-small" href="./data/parameter_sweep.csv" download><span
data-icon="download"></span>Export CSV</a></div>
74          <div class="experiment-summary"><article><span>Parameter cases</span><strong>63</strong><small>All
measured, across five fixed seeds</small></article><article><span>Full-stream
replays</span><strong>9</strong><small>Three timing runs per algorithm</small></article><article><span>Original
requests</span><strong>1.89<span>M</span></strong><small>One historical NASA capture</small></article></div>
75          <section class="chart-grid"><article class="panel"><div class="panel-heading"><div><h2>What does a
smaller summary cost?</h2><p>Full-stream maintained Python state · bytes, log scale</p></div></div><div
class="chart-legend"><span><i class="legend-line teal-bg"></i>Approximate</span><span><i class="legend-line
gray-bg"></i>Exact</span></div><div class="canvas-wrap"><canvas id="memory-chart" role="img" aria-
label="Recorded Python memory comparison"></canvas></div><div class="panel-foot">Includes Python object
overhead. This is not browser heap memory.</div></article><article class="panel"><div class="panel-
heading"><div><h2>Speed has a trade-off, too.</h2><p>Events / second · median of three runs · log
scale</p></div></div><div class="chart-legend"><span><i class="legend-line teal-
bg"></i>Approximate</span><span><i class="legend-line gray-bg"></i>Exact</span></div><div class="canvas-
wrap"><canvas id="throughput-chart" role="img" aria-label="Recorded Python throughput
comparison"></canvas></div><div class="panel-foot">Updates and queries only. Reading, parsing and validation are
 excluded.</div></article></section>
76          <article class="panel"><div class="panel-heading"><div><h2>Full dataset · default settings</h2><p>The
actual outcomes, including the uncomfortable ones.</p></div><span class="subtle-badge">SEED 42</span></div><div
class="table-scroll"><table class="data-table"><thead><tr><th>ALGORITHM</th><th>EXACT RESULT</th><th>APPROXIMATE
 RESULT</th><th>ERROR MEASURE</th><th>PYTHON STATE</th></tr></thead><tbody id="full-
results"></tbody></table></div></article>
77          <article class="panel parameter-panel"><div class="panel-heading"><div><h2>Try a different point on the
curve.</h2><p>First 100,000 valid requests · saved parameter sweep</p></div><label class="table-
filter">Algorithm<select id="experiment-filter"><option value="all">All algorithms</option><option
value="bloom">Bloom filter</option><option value="fm">Flajolet–Martin</option><option
value="dgim">DGIM</option></select></label></div><div class="table-scroll parameter-scroll"><table class="data-
table"><thead><tr><th>ALGORITHM</th><th>CONFIGURATION</th><th>SEED</th><th>EXACT</th><th>ESTIMATE /
FP</th><th>ERROR / FPR</th><th>STATE</th></tr></thead><tbody id="parameter-results"></tbody></table></div><div
class="panel-foot">Bloom: online FPR. FM: final relative error. DGIM: mean checkpoint relative error. Measures
describe different questions.</div></article>
78        </div>
79
80        <div data-view="dataset" hidden>
81          <section class="dataset-hero"><div><span class="eyebrow">THE REAL WORLD, IN A FILE</span><h2>One
server.<br>1,891,714 stories.</h2><p>Actual HTTP requests to NASA's Kennedy Space Center website. Preserved in
source order, replayed in your browser.</p><a href="https://ita.ee.lbl.gov/html/contrib/NASA-HTTP.html"
target="_blank" rel="noopener" class="button button-white">Visit the original archive <span data-
icon="external"></span></a></div><div class="dataset-orbit" aria-hidden="true"><div class="orbit-ring ring-
one"></div><div class="orbit-ring ring-two"></div><div class="orbit-center"><span data-
icon="database"></span><strong>NASA</strong><small>KENNEDY SPACE CENTER</small></div><span class="orbit-point
point-one">HTTP</span><span class="orbit-point point-two">1995</span></div></section>
82          <div class="dataset-facts"><article><span>Valid HTTP
requests</span><strong>1,891,714</strong></article><article><span>Different host
strings</span><strong>81,982</strong></article><article><span>HTTP errors ·
```

```
    400–599</span><strong>10,980</strong></article><article><span>Actual source coverage</span><strong>Jul
    1–28</strong></article></div>
83          <section class="chart-grid"><article class="panel"><div class="panel-heading"><div><h2>A month, as
    observed</h2><p>Daily valid requests · July 1995</p></div></div><div class="canvas-wrap"><canvas id="daily-
    chart" role="img" aria-label="Actual daily requests in the NASA capture"></canvas></div><div class="panel-
    foot">No records after July 28 are present; the missing days are not zero traffic.</div></article><article
    class="panel"><div class="panel-heading"><div><h2>What the server returned</h2><p>Exact response-code
    distribution</p></div></div><div id="status-distribution" class="status-distribution"></div></article></section>
84          <article class="panel provenance-panel"><div class="panel-heading"><div><h2>Trace every result back to
    its source.</h2><p>Unchanged original archive · permission to redistribute</p></div><span class="verified-
    badge"><span data-icon="check"></span>Integrity recorded</span></div><dl class="provenance-
    list"><div><dt>Collected by</dt><dd>Jim Dumoulin, Kennedy Space Center</dd></div><div><dt>Contributed
    by</dt><dd>Martin Arlitt and Carey Williamson</dd></div><div><dt>First valid request</dt><dd>01 July 1995,
    00:00:01 · UTC−04:00</dd></div><div><dt>Last valid request</dt><dd>28 July 1995, 13:32:25 ·
    UTC−04:00</dd></div><div><dt>Data handling</dt><dd>One incomplete trailing fragment is rejected. Unknown
    response sizes remain unknown. Duplicate requests are preserved.</dd></div><div><dt>SHA-256 · original
    gzip</dt><dd class="checksum-row"><code id="dataset-checksum"></code><button class="icon-button" id="copy-
    checksum" aria-label="Copy source checksum"><span data-icon="copy"></span></button></dd></div></dl><div
    class="browser-data-note"><span data-icon="info"></span><p>The browser reads a compact, lossless preparation of
    all valid requests. Deterministic hash inputs are prepared once for speed; Bloom, FM, DGIM and exact checks
    execute incrementally in a Web Worker. The download includes the untouched source file and the preparation
    script.</p></div></article>
85        </div>
86        <footer class="page-footer"><span><span class="footer-mark">■</span> StreamLab · Built to make the
    invisible visible.</span><span>Mohammed Khalifah · CSD 5113</span></footer>
87      </main>
88    </div>
89  </div>
90  <dialog id="guide-dialog" class="modal"><button class="modal-close icon-button" data-close="guide-dialog" aria-
    label="Close guide">×</button><span class="eyebrow">A QUICK FIELD GUIDE</span><h2>A stream is a sequence.<br>A
    sketch is a memory.</h2><p>This lab replays genuine requests from a historical NASA web server log. It is not
    connected to today's NASA traffic.</p><ol class="guide-steps"><li><strong>Press Start, or take one
    Step.</strong><p>Every event updates all three algorithms and their exact baselines. Use one request per second
    to watch individual decisions.</p></li><li><strong>Open the Algorithm lab.</strong><p>Inspect real bit
    positions, maximum-zero registers and DGIM buckets. Change a parameter to restart with a different
    memory–accuracy trade-off.</p></li><li><strong>Compare with the truth.</strong><p>Approximation is visible, not
    hidden. FM's teaching estimator can have very large errors. Bloom has false positives; DGIM approximates the
    oldest bucket.</p></li></ol><div class="guide-note">Memory labels in the live lab show logical payload,
    excluding JavaScript overhead and input caches. The Experiments page shows measured Python state and recorded
    benchmarks.</div><p class="muted-small">Mobile tip: use the menu at the top left to switch views. All controls
    work by touch and keyboard.</p></dialog>
91  <dialog id="source-dialog" class="modal source-modal"><div class="source-modal-heading"><div><span
    class="eyebrow">NOT A BLACK BOX</span><h2>Explore the implementation.</h2></div><button class="icon-button
    modal-close-inline" data-close="source-dialog" aria-label="Close source explorer">×</button></div><label
    class="source-selector">Source file<select id="source-file"></select></label><div class="code-toolbar"><span
    id="source-language">Source code</span><button class="small-control" id="copy-source">Copy
    code</button></div><pre id="source-content" tabindex="0"><code>Loading source files…</code></pre></dialog>
92  <dialog id="request-dialog" class="modal"><button class="modal-close icon-button" data-close="request-dialog"
    aria-label="Close request detail">×</button><span class="eyebrow">ONE REAL EVENT</span><h2>Inside this
    request.</h2><div id="request-detail"></div></dialog>
93  <div id="toast" class="toast" role="status" aria-live="polite" hidden></div>
94  <script type="module" src="./app.js"></script>
95  </body></html>
```

web/dist/styles.css

```
1 :root{--bg:#f5f7f8;--panel:#fff;--ink:#172b34;--muted:#7b8a92;--line:#e8edef;--teal:#087f6f;--teal-
  light:#e7f5ef;--violet:#8770cf;--rose:#d2857a;--amber:#c1934c;--nav-width:220px;font-family:Inter,-apple-
  system,BlinkMacSystemFont,"Segoe UI",sans-serif;color:var(--ink);background:var(--bg);font-synthesis:none;font-
  size:14px}
2 *{box-sizing:border-box}body{margin:0}button,input,select{font:inherit}button,a,select{-webkit-tap-highlight-
  color:transparent}button,select{cursor:pointer}button{color:inherit}a{color:inherit;text-
  decoration:none}button:disabled{cursor:wait;opacity:.55}button:focus-visible,a:focus-visible,select:focus-
  visible,pre:focus-visible{outline:3px solid #53b9a6;outline-
  offset:3px}button{border:0}h1,h2,h3,p{margin:0}h1,h2,h3{letter-spacing:-.035em}h2{font-size:15px;font-
  weight:650}small{font-
  size:11px}[hidden]{display:none!important}svg.icon{width:19px;height:19px;stroke:currentColor;stroke-
  width:1.7;fill:none;stroke-linecap:round;stroke-linejoin:round;flex-shrink:0}[data-icon]{display:inline-
  flex;align-items:center;justify-content:center;flex-shrink:0}.skip-
  link{position:absolute;left:20px;top:-80px;padding:12px
  18px;background:var(--teal);color:white;z-index:100}.skip-link:focus{top:10px}
3 .app-shell{min-height:100vh}.sidebar{position:fixed;inset:0 auto 0 0;width:var(--nav-
  width);background:var(--panel);border-right:1px solid var(--line);padding:31px 17px 16px;display:flex;flex-
  direction:column;z-index:30}.brand{font-size:24px;font-weight:750;letter-spacing:-1.1px;display:flex;align-
  items:center;padding:0 10px;gap:8px;margin-bottom:45px}.brand-dot{color:var(--teal);margin-left:-8px}.brand-
  mark{width:29px;height:29px;display:flex;align-items:center;gap:3px}.brand-mark
  i{display:block;width:4px;background:var(--teal);border-radius:3px;height:17px}.brand-mark i:nth-
  child(2){height:25px}.brand-mark i:nth-child(3){height:14px}.brand-mark i:nth-child(4){height:21px}.workspace-
  label{font-size:8px;font-weight:700;letter-spacing:1.5px;color:#98a3a9;padding:0 13px;margin-bottom:14px}.nav-
  list{display:grid;gap:5px}.nav-item{display:flex;align-items:center;gap:12px;min-height:43px;padding:11px
  13px;font-size:12px;font-weight:550;color:#6c7c83;position:relative;background:none;border-radius:7px;text-
  align:left;width:100%}.nav-item svg{width:17px;height:17px}.nav-
  item:hover{background:#f4f8f6;color:var(--teal)}.nav-item.active{background:#e9f5ef;color:var(--teal);font-
  weight:650}.nav-item.active:before{content:"";position:absolute;left:-
  17px;top:12px;bottom:12px;width:3px;background:var(--teal);border-radius:0 3px 3px 0}.nav-shortcut{margin-
  left:auto;opacity:.45;font-size:9px}.nav-item .small-icon{margin-left:auto}.small-icon
  svg{width:12px;height:12px}.sidebar-divider{height:1px;background:var(--line);margin:24px 12px}.sidebar-
  bottom{margin-top:auto;padding-top:35px}.source-note{background:#f6f9f7;border:1px solid #e9efe9;border-
  radius:9px;padding:15px 13px;margin:0 3px 20px}.source-spark{color:#a9be8b;font-size:20px;display:block;margin-
  bottom:10px}.source-note strong{font-size:11px}.source-note p{color:#91a096;line-height:1.8;font-
  size:10px;margin-top:6px}.source-note a{display:flex;justify-content:space-between;margin-
  top:13px;color:var(--teal);font-size:10px;font-weight:600}.profile{border-top:1px solid var(--line);padding:18px
   3px 0;display:flex;align-items:center;gap:9px}.avatar{border-
  radius:50%;width:30px;height:30px;background:#eee6de;display:grid;place-items:center;color:#79644e;font-
  size:10px;font-weight:700}.profile strong{font-size:10px;display:block}.profile small{display:block;font-
  size:8px;color:var(--muted);margin-top:4px}.profile-dot{width:5px;height:5px;border-
  radius:50%;background:#aeb9be;margin-left:auto}
4 .main-shell{margin-left:var(--nav-width)}.topbar{height:72px;display:flex;justify-content:space-between;align-
  items:center;padding:0 32px;border-bottom:1px solid var(--line);background:#fff}.breadcrumb{display:flex;align-
  items:center;gap:14px;font-size:11px}.breadcrumb-root,.slash{color:#a1adb3}.breadcrumb strong{font-
  weight:550}.topbar-right{display:flex;align-items:center;gap:22px}.historical-label{font-
  size:10px;color:#87968f;display:flex;align-items:center;gap:6px}.historical-label
  i{width:5px;height:5px;background:#82a88d;border-radius:50%}.button{display:inline-flex;align-
  items:center;justify-content:center;gap:8px;border-radius:7px;font-size:11px;font-weight:600;padding:11px
  16px;min-height:38px;white-space:nowrap;transition:background .15s,transform
  .15s}.button:active{transform:translateY(1px)}.button svg{width:15px;height:15px}.button-
  primary{background:var(--teal);color:white;box-shadow:0 2px 3px #0c665818}.button-
  primary:hover{background:#066c5e}.button-white{background:white;border:1px solid #e0e6e9;color:#53656f;box-
  shadow:0 1px 1px #00000003}.button-white:hover{background:#f4f8f6}.button-small{font-size:10px;padding:8px
  12px;min-height:32px}.icon-button{display:inline-flex;align-items:center;justify-
  content:center;width:34px;height:34px;background:none;border:1px solid transparent;border-
  radius:6px;color:#89979d}.icon-button:hover{background:#edf4f1;color:var(--teal)}.mobile-
  menu{display:none}main{padding:31px 32px 0;max-width:1780px;margin:auto;outline:0}.page-
  heading{display:flex;justify-content:space-between;gap:20px;align-items:center;margin-
  bottom:26px}.eyebrow{color:#7f928c;font-size:9px;letter-spacing:1.7px;font-weight:650;display:block;margin-
  bottom:10px}h1{font-size:29px;line-height:1.2;font-weight:650;letter-spacing:-1.15px}.page-heading p{font-
  size:11px;color:#8b979d;margin-top:10px;line-height:1.7}.project-tag{font-size:9px;display:flex;gap:8px;align-
  items:center;color:#82938c;border:1px solid #dce5df;border-radius:20px;padding:8px 12px;white-
  space:nowrap}.project-tag svg{width:12px;height:12px}
5 .replay-controller{background:white;border:1px solid #e1e8e9;border-radius:10px;margin-bottom:22px;box-shadow:0
  2px 3px #16312802}.controller-top{display:flex;align-items:center;padding:15px 17px;gap:20px}.play-
  group{display:flex;align-items:center;gap:7px}.play-group .button-primary{min-width:134px}.reset-
  button{border:1px solid var(--line);width:35px;height:37px}.controller-options{display:flex;gap:20px;align-
  items:center;margin-left:auto}.controller-options label,.settings-strip label{font-
  size:8px;color:#8c9a9f;display:grid;gap:7px}.controller-options select{padding:0 20px 0
  0;background:white;border:0;outline-offset:4px;font-size:10px;font-weight:550;color:#455c66;max-
  width:155px}.control-divider{height:26px;border-left:1px solid var(--line)}.replay-
  status{display:flex;gap:6px;align-items:center;color:#87969e;font-size:9px;min-width:65px}.replay-status
  i,.live-indicator i{width:5px;height:5px;border-radius:50%;background:#a5b0b6}.replay-
  status.running{color:var(--teal)}.replay-status.running i,.live-indicator.running i{background:#249e7b;box-
  shadow:0 0 0 3px #1e9c7917;animation:pulse 1.5s infinite}.controller-bottom{display:flex;align-
  items:center;gap:12px;padding:10px 17px;border-top:1px solid #f1f4f5;background:#fcfdfd;border-radius:0 0 10px
  10px;font-size:8px;color:#8a989e}.progress-track{height:4px;border-radius:4px;background:#e9eff0;flex:1;min-
  width:50px;overflow:hidden}.progress-track div{height:100%;width:0;background:#68a98d;border-
  radius:4px;transition:width .15s linear}.controller-help{margin-left:auto;white-space:nowrap}.controller-bottom
  #progress-text{white-space:nowrap}.metric-grid{display:grid;grid-template-
  columns:repeat(4,minmax(0,1fr));gap:16px;margin-bottom:20px}.metric-card{border:1px solid var(--line);border-
  radius:9px;background:white;padding:17px 18px 16px;position:relative;overflow:hidden}.metric-
  top{display:flex;align-items:center;justify-content:space-between;gap:6px;color:#7f8c94;font-size:10px}.metric-
```

```
icon{height:27px;width:27px;border-radius:7px}.metric-icon svg{width:15px;height:15px}.teal{background:#e9f5ef;c
olor:#32917a}.violet{background:#f0ecfa;color:#9b82ca}.amber{background:#faf3e5;color:#c3a16a}.rose{background:#
fbefed;color:#c7897e}.metric-value{font-size:31px;letter-spacing:-1.3px;font-weight:650;margin-top:15px;line-
height:1.2;font-variant-numeric:tabular-nums;position:relative;z-index:1}.metric-unit{font-size:19px;margin-
left:3px;color:#8d9ba2;font-weight:500}.metric-bottom{display:flex;align-items:center;gap:7px;margin-
top:13px;font-size:8px;color:#99a3a9;position:relative;z-index:1;min-height:18px}.mini-pill{font-
size:7px;border-radius:4px;padding:4px 5px;white-space:nowrap}.neutral{background:#f3f6f5;color:#7c9285}.violet-
pill{background:#f4f0fb;color:#aa91ca}.amber-pill{background:#fbf6ed;color:#b89b70}.rose-
pill{background:#fcf0ee;color:#c7958b}.mini-
wave{position:absolute;right:10px;bottom:44px;width:67px;height:22px;opacity:.6}.mini-wave
path{fill:none;stroke-width:1.5;stroke-linecap:round}.teal-wave path{stroke:#9cc5b3}.violet-wave
path{stroke:#b8a7da}.rose-wave path{stroke:#ddb0a5}.tiny-
meter{position:absolute;right:16px;top:79px;width:54px;height:4px;background:#f7f0e5;border-
radius:4px;overflow:hidden}.tiny-meter span{display:block;height:100%;width:0;background:#d8b47c}
.chart-grid{display:grid;grid-template-columns:minmax(0,1.38fr) minmax(0,1fr);gap:20px;margin-
bottom:20px}.panel{background:white;border:1px solid var(--line);border-radius:10px;min-
width:0;overflow:hidden;margin-bottom:20px}.chart-grid>.panel,.lower-grid>.panel{margin-bottom:0}.panel-
heading{display:flex;align-items:center;justify-content:space-between;gap:16px;padding:20px 20px 0}.panel-
heading p{font-size:9px;color:#96a1a7;margin-top:6px;line-height:1.6}.panel-heading h2{line-height:1.4}.live-
indicator{display:inline-flex;align-items:center;gap:5px;border:1px solid #e6eeea;border-radius:5px;padding:5px
7px;font-size:8px;color:#8a9b92}.algorithm-tag{font-size:8px;border-radius:4px;font-weight:600;padding:6px
8px;white-space:nowrap}.rose-tag{color:#ba8579;background:#fcf2ef}.violet-
tag{color:#9b83c4;background:#f4f0fb}.teal-tag{color:#438e77;background:#eaf6ee}.chart-
legend{display:flex;align-items:center;gap:20px;padding:20px 21px 0;font-size:9px;color:#9aa5aa}.chart-legend
span{display:flex;align-items:center;gap:6px}.chart-legend b{font-size:9px;font-weight:550;margin-
left:1px;color:#647780;font-variant-numeric:tabular-nums}.legend-line{display:inline-
block;width:9px;height:3px;border-radius:3px;flex-shrink:0}.teal-bg{background:var(--teal)}.violet-
bg{background:var(--violet)}.rose-bg{background:var(--rose)}.gray-bg{background:#aab6bf}.amber-
bg{background:var(--amber)}.canvas-wrap{height:225px;position:relative;margin:10px 14px
0}canvas{display:block;width:100%;height:100%}.chart-empty{position:absolute;inset:0;display:flex;flex-
direction:column;align-items:center;justify-content:center;gap:8px;color:#a9b5ba;background:linear-
gradient(180deg,#ffffff00,#ffffffb8);pointer-events:none}.chart-empty [data-icon]{color:#bdccc5;margin-
bottom:5px}.chart-empty strong{font-size:11px;font-weight:550;color:#8ea299}.chart-empty span:not([data-
icon]){font-size:9px}.panel-foot{border-top:1px solid #eef2f3;padding:12px 20px;color:#9aa5ab;font-
size:8px;display:flex;align-items:flex-start;gap:6px;line-height:1.6}.panel-foot
svg{width:12px;height:12px;position:relative;top:1px}.lower-grid{display:grid;grid-template-
columns:minmax(0,1.95fr) minmax(0,1fr);gap:20px;margin-bottom:18px}.count-badge{background:#f1f5f4;border-
radius:4px;font-size:9px;color:#8ba297;letter-spacing:0;padding:3px 5px;margin-left:4px}.tiny-label{font-
size:8px;color:#97a79e;display:flex;gap:5px;align-items:center;white-
space:nowrap}.dot{width:4px;height:4px;border-radius:50%;display:inline-block}.table-
scroll{overflow:auto;width:100%;scrollbar-width:thin}.request-table,.data-table{width:100%;border-
collapse:collapse;text-align:left;white-space:nowrap}.request-table{margin-top:19px;table-layout:fixed}.request-
table thead,.data-table thead{background:#fafbfc;color:#96a2a8;font-size:7px;letter-spacing:.65px}.request-table
 th{padding:10px 8px;font-weight:550;border-bottom:1px solid #edf1f2}.request-table th:first-child{padding-
left:20px;width:88px}.request-table th:nth-child(2){width:26%}.request-table th:nth-child(3){width:31%}.request-
table th:nth-child(4){width:65px}.request-table th:last-child{width:77px;padding-right:17px}.request-table
small{font-size:6px;font-weight:400;letter-spacing:0}.request-table td{padding:10px 8px;font-size:8px;border-
bottom:1px solid #f0f3f4;color:#8a9aa2;overflow:hidden;text-overflow:ellipsis}.request-table td:first-
child{padding-left:20px;font-variant-numeric:tabular-nums}.request-table tr[data-event]{cursor:pointer}.request-
table tr[data-event]:hover{background:#f5faf7}.request-table tr[data-event]:first-
child{background:#f6faf8}.request-path{color:#98a5ab!important;font-family:ui-monospace,SFMono-
Regular,Menlo,monospace;font-size:7px!important}.status-pill{display:inline-flex;padding:3px
5px;background:#edf7f0;color:#68a289;border-radius:3px;font-size:7px;line-height:1.2}.status-
pill.redirect{background:#f0f2f8;color:#939ebc}.status-pill.error{background:#fbf0e7;color:#c09572}.status-
pill.server-error{background:#fdece8;color:#cd887e}.seen-pill{color:#a1aaa9;font-size:7px}.seen-
pill.new{color:#7a9f88}.seen-pill.fp{color:#c68d5c}.empty-table{height:215px;text-
align:center;color:#b2bcbf!important}.feed-foot{justify-content:space-between;border-top:0}.muted-
icon{color:#a6b3af}.muted-icon svg{width:16px}.payload-item{display:flex;gap:10px;align-
items:center;padding:20px 20px 16px;border-bottom:1px solid #f3f5f6}.payload-dot{width:6px;height:6px;border-
radius:2px}.payload-item strong{display:block;font-size:10px;font-weight:550}.payload-item
div>span{display:block;font-size:8px;color:#9ba7ad;margin-top:5px}.payload-item b{margin-left:auto;font-
size:10px;font-weight:550;font-variant-numeric:tabular-nums}.payload-note{margin:15px 20px
5px;color:#a0abb1;font-size:8px;line-height:1.8}.text-link{display:flex;justify-content:space-between;align-
items:center;font-size:9px;color:#609480;font-weight:550;margin:16px 20px 20px}.replay-
context{display:flex;align-items:center;gap:7px;color:#91a49a;font-size:9px;padding:4px 0 15px}.replay-context
svg{width:13px;height:13px}.replay-context a{margin-left:auto;color:#8fa095}.page-footer{margin-top:16px;border-
top:1px solid #e6edef;padding:19px 0 24px;display:flex;justify-content:space-between;font-
size:8px;color:#a9b3b7;gap:15px}.footer-mark{font-size:13px;color:#91aca0;margin-right:5px}
.section-intro{display:flex;justify-content:space-between;gap:20px;align-items:center;margin:30px 0
20px}.section-intro h2{font-size:21px}.section-intro p{font-size:11px;color:#8b9b9f;margin-top:8px}.muted-
small{font-size:10px;color:#91a19c;line-height:1.8}.settings-strip{display:grid;grid-template-columns:1.3fr 1fr
1.2fr 1.1fr .65fr;background:#eaf1ed;border:1px solid #e0e9e2;border-radius:8px;padding:17px;gap:17px;margin-
bottom:20px}.settings-strip label{color:#7d978a;font-size:9px}.settings-strip select{font-
size:10px;background:transparent;color:#466b5d;border:0;padding:0;max-width:100%}.algorithm-panel .panel-
heading{padding-bottom:19px;border-bottom:1px solid #eff3f2}.algorithm-title{display:flex;align-
items:center;gap:13px}.algorithm-number{height:35px;width:35px;border-radius:9px;display:grid;place-
items:center;font-size:12px;font-weight:600}.subtle-badge{font-size:7px;letter-spacing:.7px;font-
weight:500;background:#f4f6f7;color:#9ba6ab;padding:4px 6px;border-radius:4px;margin-left:8px;vertical-
align:middle;white-space:nowrap}.algorithm-grid{display:grid;grid-template-columns:minmax(0,1.75fr)
minmax(0,1fr);gap:25px;padding:25px}.bit-toolbar{display:flex;justify-content:space-between;align-
items:center;font-size:9px;color:#a1aca9;gap:10px;margin-bottom:14px}.bit-toolbar strong{color:#6d8980;font-
weight:500;margin-left:5px}.bit-toolbar>div{display:flex;gap:4px}.small-control{background:white;border:1px
solid #dfe7e3;border-radius:4px;padding:5px 7px;font-size:8px;color:#7c9488}.small-
control:hover{color:var(--teal);border-color:#a3c8b8}.bit-grid{display:grid;grid-template-
columns:repeat(16,minmax(0,1fr));gap:5px}.bit-cell{aspect-ratio:1.3;background:#f0f4f2;border:1px solid
#e9efec;border-radius:4px;display:grid;place-items:center;font-size:8px;color:#c0cdc6;font-family:ui-
```

```
monospace,monospace}.bit-cell.set{background:#c9e5d7;color:#438b6b;border-color:#b5dac7}.bit-
cell.highlight{background:var(--teal);color:white;border-color:var(--teal);box-shadow:0 0 0 2px #b5dace}.visual-
legend{display:flex;gap:15px;font-size:8px;color:#9eaaa4;align-items:center;flex-wrap:wrap;margin-
top:12px}.visual-legend>span{display:flex;align-items:center;gap:4px}.bit-legend{width:8px;height:8px;border-
radius:2px;display:inline-block}.bit-legend.off{background:#eef3f0}.bit-legend.on{background:#badcc9}.bit-
legend.hit{background:var(--teal)}.formula-box{margin-top:24px;padding:16px;background:#f8faf9;border:1px solid
#eaf0ed;border-radius:6px;font-family:ui-monospace,SFMono-Regular,Menlo,monospace;color:#758d80;font-
size:12px;line-height:1.7}.formula-box>span{display:block;font-family:Inter,-apple-system,sans-serif;font-
size:8px;color:#a3b2a9;margin-top:5px;overflow-wrap:anywhere}.trace-box{border-left:1px solid
#edf1ef;padding:3px 0 0 23px;min-width:0}.trace-box .eyebrow{font-size:8px;color:#a2b0a8;margin-
bottom:14px}.trace-box h3{font-size:12px;overflow-wrap:anywhere;font-family:ui-monospace,monospace;margin-
bottom:12px}.trace-box p{font-size:10px;color:#8b9e92;line-height:1.9}.trace-box>small{font-
size:8px;color:#a0afa5;line-height:1.9;display:block}.hash-positions{display:flex;flex-
wrap:wrap;gap:5px;margin:15px 0}.hash-positions span{background:#f3f7f4;border:1px solid #e9efea;padding:5px
6px;border-radius:4px;font-size:8px;color:#7d9687;font-family:ui-monospace,monospace}.trace-
metrics{display:flex;justify-content:space-between;border-top:1px solid #edf1ee;padding:15px 0;gap:10px}.trace-
metrics span{font-size:8px;color:#a1ada6;display:block}.trace-metrics b{font-size:17px;display:block;margin-
top:5px;font-weight:550}.register-grid{display:grid;grid-template-
columns:repeat(16,minmax(0,1fr));gap:7px;align-items:end;min-height:150px;padding-
top:12px}.register{display:flex;flex-direction:column;align-items:center;gap:6px;min-width:0}.register-
value{font-size:9px;color:#9f8db9}.register-bar-track{height:110px;width:100%;background:#f7f4fb;border-
radius:4px;position:relative;min-width:7px}.register-bar{position:absolute;bottom:0;width:100%;min-
height:2px;background:linear-gradient(#a997d5,#c5b8e0);border-radius:4px;transition:height .15s}.register-
label{font-size:6px;color:#b3a7c2}.register-grid.many{gap:4px;min-height:260px}.register-grid.many .register-
bar-track{height:48px}.register-grid.many .register{gap:3px}.comparison-pair{display:grid;grid-template-
columns:1fr 1fr;gap:12px;margin:16px 0 22px}.comparison-pair span{font-
size:8px;color:#9aaa9f;display:block}.comparison-pair b{font-size:24px;font-weight:550;letter-
spacing:-.8px;display:block;margin-top:8px;font-variant-numeric:tabular-nums}.accuracy-
line{display:flex;justify-content:space-between;font-size:9px;padding:12px 0;border-top:1px solid
#edf1ee;border-bottom:1px solid #edf1ee;margin-bottom:12px;color:#93a299}.accuracy-line strong{font-
weight:550;color:#c19875}.bucket-axis{display:flex;justify-content:space-between;color:#a2aea8;font-
size:8px;margin-bottom:15px}.bucket-strip{display:flex;gap:7px;overflow-x:auto;min-height:118px;align-
items:center;padding:8px 2px 12px;scrollbar-width:thin}.bucket{border:1px solid #e8d7d0;min-width:51px;border-
radius:7px;padding:14px 8px;background:#faf1ec;text-align:center;position:relative;flex-shrink:0}.bucket:nth-
child(3n){background:#f4e6df}.bucket b{display:block;font-size:20px;font-weight:550;color:#ba8b7c}.bucket
span{font-size:7px;display:block;color:#bca79e;margin-top:6px}.bucket small{font-
size:6px;display:block;color:#b49b90;margin-top:5px}.bucket.partial{border-style:dashed;background:repeating-
linear-gradient(135deg,#fff9f4,#fff9f4 5px,#f9f1e9 5px,#f9f1e9 10px)}.bits-caption{display:flex;justify-
content:space-between;gap:10px;font-size:9px;color:#9aa9a0;margin:14px 0 10px}.bits-caption span{font-
size:7px;color:#afbbb4}.binary-stream{display:grid;grid-template-columns:repeat(32,minmax(0,1fr));gap:3px;min-
height:65px}.binary-bit{background:#f2f5f2;border-radius:2px;display:grid;place-items:center;color:#c6cfc7;font-
size:6px;min-height:13px}.binary-bit.one{background:#f2d6c7;color:#ad7c60;font-weight:700}.action-log{font-
size:9px;color:#a18b80;line-height:1.9;padding:12px 0;border-top:1px solid #f0ebe7;margin-top:12px}.action-
log>div{padding:3px 0}.action-log small{font-size:7px;color:#b9aaa0;margin-right:5px}.visual-empty{font-
size:10px;color:#b5bdb8;text-align:center;width:100%;padding:32px 12px}
.recorded-banner{border:1px solid #dfe8e0;background:#edf5ef;display:flex;align-
items:center;gap:16px;padding:20px;border-radius:9px;margin:28px 0 20px;color:#6c9581}.recorded-banner>[data-
icon]{width:37px;height:37px;border:1px solid #cde0d0;border-radius:8px}.recorded-banner strong{font-
size:12px;color:#5a806a}.recorded-banner p{font-size:9px;line-height:1.8;margin-top:5px}.recorded-
banner>a{margin-left:auto}.experiment-summary{display:grid;grid-template-columns:repeat(3,1fr);gap:18px;margin-
bottom:20px}.experiment-summary>article{padding:24px;background:white;border:1px solid var(--line);border-
radius:9px}.experiment-summary>article>span{font-size:10px;color:#93a196}.experiment-summary
strong{display:block;margin-top:12px;font-size:32px;font-weight:550;letter-spacing:-1px}.experiment-summary
strong span{font-size:20px}.experiment-summary small{font-size:9px;color:#a3aea6;display:block;margin-
top:7px}.data-table{font-size:10px;margin-top:20px}.data-table th{padding:12px 19px;font-size:8px;font-
weight:500}.data-table td{padding:15px 19px;border-top:1px solid #edf1f0;color:#7f9289;font-size:9px;font-
variant-numeric:tabular-nums}.data-table tbody tr:hover{background:#f9fcfa}.data-table td:first-child{font-
weight:600;color:#587566}.table-filter{display:flex;align-items:center;gap:10px;font-
size:9px;color:#9aa99f}.table-filter select{border:1px solid #dfe7e1;padding:7px 10px;border-radius:5px;font-
size:9px;background:white;color:#668671}.parameter-scroll{max-height:450px}.parameter-scroll
thead{position:sticky;top:0;z-index:1}
.dataset-hero{display:grid;grid-template-columns:1.1fr 1fr;padding:42px 46px;border-
radius:12px;background:#e8f0e9;border:1px solid #dde7df;margin-top:25px;overflow:hidden}.dataset-hero
.eyebrow{color:#79927d;font-size:8px}.dataset-hero h2{font-size:42px;line-height:1.15;letter-
spacing:-1.8px;font-weight:550;color:#34614e}.dataset-hero p{font-size:11px;line-height:1.9;color:#86a08b;max-
width:300px;margin:18px 0 22px}.dataset-hero .button{font-size:9px}.dataset-
orbit{height:260px;position:relative;display:grid;place-items:center}.orbit-ring{position:absolute;border:1px
solid #bdcdbc;border-radius:50%;width:230px;height:230px;transform:rotate(-25deg) scaleY(.8)}.ring-
two{width:295px;height:295px;transform:rotate(45deg) scaleY(.55);opacity:.6}.orbit-center{text-
align:center;display:flex;flex-direction:column;align-items:center;color:#90a787}.orbit-center
svg{width:32px;height:32px;margin-bottom:12px}.orbit-center strong{font-size:33px;font-weight:550;letter-
spacing:5px;color:#567b58}.orbit-center small{font-size:6px;letter-spacing:1.5px;margin-top:10px}.orbit-
point{position:absolute;border-radius:50%;height:38px;width:38px;border:1px solid
#b8cbb6;background:#e4eee2;display:grid;place-items:center;font-size:7px;color:#72936c;box-shadow:0 0 0 7px
#e8f0e9}.point-one{left:18%;top:13%}.point-two{right:10%;bottom:19%}.dataset-facts{display:grid;grid-template-
columns:repeat(4,1fr);gap:17px;margin:20px 0}.dataset-facts article{padding:21px;background:white;border-
radius:9px;border:1px solid var(--line)}.dataset-facts span{font-size:9px;color:#97a59b}.dataset-facts
strong{display:block;font-size:25px;font-weight:550;letter-spacing:-1px;margin-top:11px}.status-
distribution{padding:24px 22px 22px;display:grid;gap:17px}.status-row{display:grid;grid-template-columns:48px
1fr 72px;align-items:center;gap:12px}.status-row span,.status-row b{font-size:9px;color:#8b9d91;font-
weight:500}.status-bar{height:5px;background:#f2f5f2;border-radius:3px;overflow:hidden}.status-bar
i{display:block;height:100%;background:#9cbdab;border-radius:3px}.status-row b{text-align:right;font-variant-
numeric:tabular-nums}.verified-badge{display:flex;align-items:center;gap:4px;font-
size:8px;color:#79a18a}.verified-badge svg{width:12px;height:12px}.provenance-list{margin:18px 24px
22px}.provenance-list>div{display:grid;grid-template-columns:160px 1fr;gap:20px;padding:15px 0;border-top:1px
solid #eff3f0;line-height:1.8;font-size:10px}.provenance-list dt{color:#9eaca2}.provenance-list
```

```
dd{margin:0;color:#6d8c78;min-width:0;overflow-wrap:anywhere}.checksum-row{display:flex;align-
items:center;gap:12px}.checksum-row code{font-size:9px;overflow-wrap:anywhere;min-width:0}.checksum-row .icon-
button{margin-left:auto;flex-shrink:0}.browser-data-note{margin:0 24px
24px;padding:17px;background:#f4f8f4;border-radius:6px;display:flex;gap:10px;font-size:9px;line-
height:1.9;color:#98ac9c}.browser-data-note svg{width:15px;height:15px}
.modal{border:1px solid #dae5dd;border-radius:15px;padding:36px;max-width:600px;width:calc(100% - 32px);max-
height:85vh;overflow:auto;color:var(--ink);box-shadow:0 30px 100px
#102c2840}.modal::backdrop{background:#152e2a65;backdrop-filter:blur(5px)}.modal h2{font-size:28px;line-
height:1.25;font-weight:550;margin-bottom:20px}.modal>p{font-size:12px;line-height:1.9;color:#8c9f92}.modal-
close{position:absolute;top:12px;right:12px;font-size:24px;color:#9aaa9f}.guide-steps{padding-
left:21px;margin:25px 0}.guide-steps li{padding:5px 0 14px;color:#7f9e8a;font-size:12px}.guide-steps
strong{color:#587b65;font-weight:550}.guide-steps p{font-size:11px;line-height:1.9;margin-
top:7px;color:#95a69a}.guide-note{padding:16px;background:#edf5ef;border-radius:6px;line-
height:1.9;color:#86a18e;font-size:10px;margin-bottom:18px}.source-modal{max-width:950px}.source-modal-
heading{display:flex;justify-content:space-between;align-items:flex-start}.modal-close-inline{font-
size:25px}.source-selector{display:grid;gap:8px;font-size:10px;color:#91a397}.source-selector
select{padding:10px;border:1px solid #dce7de;border-radius:5px;color:#6f8e7a;background:#f7faf7;font-
size:11px}.code-toolbar{display:flex;justify-content:space-between;align-items:center;margin:18px 0 8px;font-
size:10px;color:#9daf9e}#source-content{background:#172e28;color:#b9cebf;font:10px/1.9 ui-monospace,SFMono-
Regular,Menlo,monospace;border-radius:7px;padding:20px;overflow:auto;max-height:48vh;margin:0;tab-
size:4}.toast{position:fixed;bottom:25px;left:calc(50% +
80px);transform:translateX(-50%);background:#29493b;color:#e0ece2;padding:13px 22px;font-size:11px;border-
radius:8px;z-index:100;box-shadow:0 6px 30px #1b332e25;max-width:calc(100vw - 35px)}.notice{border:1px solid
#ead4c2;background:#fef3e9;color:#ae7b53;font-size:11px;line-height:1.8;padding:15px 18px;border-
radius:7px;margin-bottom:18px}.chart-
tooltip{position:absolute;z-index:2;background:#284a3ef2;color:#e3eee4;border-radius:6px;padding:10px 12px;font-
size:9px;line-height:1.8;pointer-events:none;box-shadow:0 5px 20px #0001;white-space:nowrap}.request-detail-
list{margin:20px 0 0}.request-detail-list div{padding:11px 0;border-top:1px solid #edf2ee}.request-detail-list
dt{font-size:9px;color:#a3b0a5;margin-bottom:6px}.request-detail-list dd{font-size:11px;line-
height:1.8;margin:0;color:#708f79;overflow-wrap:anywhere}.request-detail-list code{white-space:pre-wrap;font-
size:10px}.nav-backdrop{position:fixed;inset:0;background:#1a302855;z-index:29}.app-shell.nav-open .nav-
backdrop{display:block!important}@keyframes pulse{50%{box-shadow:0 0 0 5px #1e9c7908}}
@media(min-width:1600px){:root{--nav-width:240px}.sidebar{padding-top:35px}main{padding:36px 42px
0}.topbar{padding:0 42px}.canvas-wrap{height:255px}.metric-card{padding:20px 22px}.metric-value{font-
size:36px}.metric-bottom{font-size:9px}.metric-top{font-size:11px}.nav-item{font-size:13px}}
@media(max-width:1200px){:root{--nav-width:190px}.sidebar{padding-left:12px;padding-right:12px}.brand{font-
size:22px;padding:0 8px}.workspace-label{font-size:7px;letter-spacing:1px}.nav-item{font-size:11px;padding-
left:10px;gap:9px}.nav-shortcut{display:none}.nav-item.active:before{left:-12px}.source-note{padding:13px
10px}.profile strong{font-size:9px}.profile small{font-size:7px}main{padding:26px 23px 0}.topbar{padding:0
23px}.historical-label{display:none}.metric-grid{gap:10px}.metric-card{padding:14px 13px}.metric-value{font-
size:27px}.metric-bottom{gap:4px;flex-wrap:wrap;min-height:30px}.metric-top{font-size:9px}.controller-
options{gap:12px}.controller-top{gap:12px}.controller-options label{font-size:7px}.controller-options
select{font-size:9px}.replay-status{font-size:8px;min-width:53px}.chart-grid{gap:15px;grid-template-
columns:1.2fr 1fr}.lower-grid{gap:15px;grid-template-columns:1.65fr 1fr}.panel-heading{padding:17px 16px
0}.panel-heading h2{font-size:13px}.chart-legend{padding:18px 17px 0;gap:13px}.panel-foot{padding:11px
16px;font-size:7px}.payload-item{padding:18px 15px}.payload-item b{font-size:9px}.payload-item strong{font-
size:9px}.payload-item div>span{font-size:7px}.tiny-label{display:none}.algorithm-
grid{gap:18px;padding:20px}.trace-box{padding-left:18px}.comparison-pair b{font-size:21px}.dataset-
hero{padding:32px}.dataset-hero h2{font-size:36px}.dataset-orbit{transform:scale(.85)}.project-tag{font-
size:8px}.request-table th:nth-child(3){width:25%}.request-table th:nth-child(2){width:25%}}
@media(max-width:980px){.metric-grid{grid-template-columns:repeat(2,minmax(0,1fr));gap:13px}.metric-
card{padding:18px}.metric-value{font-size:32px}.metric-bottom{flex-wrap:nowrap;min-height:18px}.metric-top{font-
size:11px}.chart-grid{grid-template-columns:1fr}.lower-grid{grid-template-columns:1fr}.snapshot-
panel{display:grid;grid-template-columns:repeat(3,1fr)}.snapshot-panel .panel-heading{grid-column:1/-1;margin-
bottom:6px}.snapshot-panel .payload-item{border-bottom:0;padding:18px 17px;flex-wrap:wrap}.snapshot-panel
.payload-item b{margin-left:15px}.snapshot-panel .payload-note{grid-column:1/3;margin-top:2px}.snapshot-panel
.text-link{margin-top:5px}.controller-top{flex-wrap:wrap;gap:13px}.controller-
options{order:3;width:100%;justify-content:space-between;margin:0;padding-top:10px;border-top:1px solid
#f0f4f1}.controller-options label{font-size:8px;flex:1}.controller-options select{font-size:10px;width:100%;max-
width:100%}.replay-status{margin-left:auto}.controller-help{display:none}.canvas-wrap{height:240px}.algorithm-
grid{grid-template-columns:1fr}.trace-box{border-left:0;border-top:1px solid #edf2ee;padding:18px 0
0}.comparison-pair{margin-top:10px;margin-bottom:12px}.trace-metrics{max-width:300px}.settings-strip{grid-
template-columns:repeat(3,minmax(0,1fr));gap:20px}.section-intro{align-items:flex-start;flex-
direction:column;gap:10px}.dataset-hero{grid-template-columns:1fr .8fr;gap:10px;padding:28px}.dataset-hero
h2{font-size:32px}.dataset-orbit{transform:scale(.7);width:170px;margin-left:-10px}.dataset-facts{grid-template-
columns:repeat(2,1fr)}.provenance-list>div{grid-template-columns:130px 1fr;gap:14px}.recorded-banner{flex-
wrap:wrap}.recorded-banner>a{margin-left:53px}.experiment-summary{gap:12px}.experiment-
summary>article{padding:18px}.experiment-summary small{font-size:8px}.request-table td{font-size:9px}.request-
table th{font-size:8px}.request-path{font-size:8px!important}}
@media(max-width:700px){:root{--nav-width:220px}.sidebar{transform:translateX(-100%);transition:transform .2s
ease;padding:25px 17px 15px}.app-shell.nav-open .sidebar{transform:translateX(0)}.sidebar .nav-item{font-
size:12px;min-height:45px}.sidebar .workspace-label{font-size:8px}.sidebar .brand{font-size:24px}.sidebar
.profile strong{font-size:10px}.main-shell{margin-left:0}.mobile-menu{display:inline-flex;margin-
left:-7px;width:28px;height:30px}.topbar{height:61px;padding:0 19px}.breadcrumb{gap:9px;font-size:10px}.topbar-
right{gap:0}.topbar-right .button{padding:7px 9px;font-size:9px}.topbar-right .button svg{width:13px}.topbar-
right .button span:not([data-icon]){display:none}main{padding:25px 17px 0}.page-heading{align-items:flex-
start;gap:8px;margin-bottom:21px}.eyebrow{font-size:8px;letter-spacing:1.4px;margin-bottom:9px}h1{font-
size:26px;letter-spacing:-.95px;max-width:310px}.page-heading p{font-size:10px;max-width:310px;margin-
top:9px;line-height:1.8}.project-tag{display:none}.controller-top{padding:13px}.controller-bottom{padding:10px
13px;font-size:7px}.controller-options{padding-top:12px}.play-group .button-primary{min-width:132px}.play-group
.button{font-size:10px}.replay-controller{margin-bottom:16px}.metric-grid{gap:10px;margin-bottom:16px}.metric-
card{padding:13px 12px}.metric-top{font-size:9px}.metric-icon{height:23px;width:23px}.metric-icon
svg{width:13px;height:13px}.metric-value{font-size:28px;margin-top:13px}.metric-bottom{gap:5px;font-
size:7px;flex-wrap:wrap;margin-top:10px;min-height:29px}.mini-pill{font-size:6px;padding:3px 4px}.metric-
bottom>span:last-child{font-size:7px}.mini-wave{width:53px;right:6px;bottom:47px;opacity:.5}.tiny-
meter{width:40px;right:12px;top:71px}.chart-grid{gap:16px;margin-bottom:16px}.canvas-wrap{height:220px;margin-
```

```
left:10px;margin-right:10px}.chart-legend{padding-top:17px;font-size:8px}.chart-legend b{font-size:8px}.panel-
heading{padding:17px 16px 0;gap:10px}.panel-heading h2{font-size:13px}.panel-heading p{font-size:8px}.panel-
foot{font-size:7px;padding:10px 15px}.request-table{min-width:480px;margin-top:16px}.request-table th:first-
child{padding-left:15px}.request-table td:first-child{padding-left:15px}.request-table td{font-
size:8px}.request-table th:nth-child(2){width:25%}.request-table th:nth-child(3){width:30%}.snapshot-panel{grid-
template-columns:1fr}.snapshot-panel .panel-heading{margin-bottom:6px}.snapshot-panel .payload-item{padding:16px
 18px;border-bottom:1px solid #f2f5f2}.snapshot-panel .payload-item strong{font-size:11px}.snapshot-panel
.payload-item div>span{font-size:8px}.snapshot-panel .payload-item b{margin-left:auto;font-size:11px}.snapshot-
panel .payload-note{grid-column:1;font-size:8px}.snapshot-panel .text-link{margin-top:15px;font-
size:10px}.replay-context{font-size:8px;align-items:flex-start;flex-wrap:wrap;gap:6px;line-height:1.8}.replay-
context a{margin-left:19px;flex-basis:100%;font-size:8px}.page-footer{font-size:7px;line-height:1.7;flex-
wrap:wrap;gap:8px}.section-intro{margin-top:25px}.section-intro h2{font-size:20px}.section-intro p{font-
size:10px}.section-intro .muted-small{font-size:9px}.settings-strip{grid-template-
columns:repeat(2,minmax(0,1fr));padding:16px;gap:20px 16px}.settings-strip label{font-size:8px}.settings-strip
select{font-size:9px}.algorithm-panel .panel-heading{flex-wrap:wrap;gap:12px;padding-bottom:15px}.algorithm-
panel .algorithm-tag{margin-left:45px;font-size:7px}.algorithm-title{gap:10px}.algorithm-
number{width:32px;height:32px}.algorithm-title h2{font-size:14px}.algorithm-title p{font-size:8px}.algorithm-
title .subtle-badge{font-size:6px;margin-left:4px}.algorithm-grid{padding:18px;gap:18px}.bit-grid{gap:4px}.bit-
cell{font-size:7px;border-radius:3px}.bit-toolbar{font-size:8px;flex-wrap:wrap;gap:10px}.bit-toolbar>div{margin-
left:auto}.small-control{font-size:7px;padding:5px 6px}.visual-legend{font-size:7px;gap:10px}.formula-box{font-
size:11px;padding:14px;margin-top:20px}.trace-box p{font-size:10px}.register-grid{gap:4px}.register-value{font-
size:8px}.register-label{font-size:5px}.register-bar-track{height:92px}.register-grid.many .register-bar-
track{height:42px}.register-grid.many{gap:3px}.bits-caption{flex-direction:column;gap:5px}.binary-
stream{gap:2px}.binary-bit{font-size:5px}.bucket{min-width:45px;padding:12px 6px}.comparison-pair b{font-
size:27px}.trace-box>small{font-size:9px}.recorded-banner{padding:17px;gap:12px;margin-top:23px}.recorded-
banner>div{flex:1}.recorded-banner strong{font-size:11px}.recorded-banner p{font-size:9px}.recorded-
banner>a{margin-left:49px}.experiment-summary{gap:8px}.experiment-summary>article{padding:14px 10px}.experiment-
summary>article>span{font-size:8px}.experiment-summary strong{font-size:28px}.experiment-summary small{font-
size:7px;line-height:1.6}.data-table th{font-size:7px;padding:11px 14px}.data-table td{font-
size:8px;padding:13px 14px}.table-filter{font-size:8px;gap:7px}.table-filter select{max-width:110px;font-
size:8px}.parameter-panel .panel-heading{align-items:flex-start;flex-wrap:wrap}.dataset-hero{padding:25px;grid-
template-columns:1fr;position:relative}.dataset-hero>div:first-child{z-index:1;position:relative}.dataset-hero
h2{font-size:34px}.dataset-hero p{font-size:10px;max-width:230px}.dataset-
orbit{position:absolute;right:-80px;top:35px;opacity:.25;transform:scale(.9);pointer-events:none}.dataset-
facts{gap:10px;margin:16px 0}.dataset-facts article{padding:17px 14px}.dataset-facts strong{font-
size:24px}.dataset-facts span{font-size:8px}.provenance-panel .panel-heading{flex-wrap:wrap}.provenance-
list{margin:14px 17px 19px}.provenance-list>div{grid-template-columns:1fr;gap:5px;padding:12px 0;font-
size:10px}.provenance-list dt{font-size:8px}.checksum-row{gap:7px}.checksum-row code{font-size:8px}.browser-
data-note{margin:0 17px 19px;padding:13px;font-size:8px}.status-distribution{padding:24px
18px}.modal{padding:29px 22px;border-radius:12px}.modal h2{font-size:24px}.modal>p{font-size:11px}.guide-steps
p{font-size:10px}.source-modal{padding:25px 16px}.source-modal h2{font-size:21px}.source-modal .eyebrow{font-
size:7px}.toast{left:50%;width:max-content;font-size:10px;text-align:center}.modal::backdrop{backdrop-
filter:blur(3px)}}
@media(prefers-reduced-motion:reduce){*{scroll-
behavior:auto!important;animation:none!important;transition:none!important}}
```

web/dist/worker.js

```
import {StreamEngine} from './algorithms.js';

let engine, config, streamReader, carry = new Uint8Array(), cursor = 0, block = null;
let running = false, ready = false, exhausted = false, generation = 0, busy = false;
let rate = 5000, offset = 0, computeMs = 0, timer = null, recordIndex = 0;
let history = [], feed = [], emitted = 0, abortController;

async function uncompressed(url, signal) {
  const response = await fetch(url,{signal});
  if(!response.ok) throw Error(`Cannot load dataset asset (${response.status})`);
  if(!('DecompressionStream' in self)) throw Error('This browser needs gzip stream support. Please use a current Chrome, Edge, Firefox or Safari.');
  return response.body.pipeThrough(new DecompressionStream('gzip'));
}

async function init(next) {
  const current = ++generation; running = ready = false; clearTimeout(timer);
  abortController?.abort(); abortController = new AbortController();
  try {
    if(streamReader) await streamReader.cancel().catch(()=>{});
    config = next; rate = next.rate; offset = 0; history = []; feed = []; computeMs = 0;
    carry = new Uint8Array(); cursor = 0; block = null; exhausted = false; recordIndex = 0; emitted = 0;
    postMessage({type:'loading',message:'Loading genuine NASA records and deterministic hash inputs…'});
    const hashStream = await uncompressed(`./data/hashes-${config.seed}.bin.gz`,abortController.signal);
    const hashBuffer = await new Response(hashStream).arrayBuffer();
    if(current!==generation) return;
    engine = new StreamEngine(config,hashBuffer);
    const input = await uncompressed('./data/events.bin.gz',abortController.signal);
    streamReader = input.getReader(); ready = true; busy = false;
    postMessage({type:'ready',snapshot:engine.snapshot(),config});
  } catch(error) { if(current===generation && error.name!=='AbortError') postMessage({type:'error',message:error.message}); }
}

async function nextRecord() {
  while(!block || cursor+20>block.byteLength) {
    const previous = block ? new Uint8Array(block.buffer,block.byteOffset+cursor,block.byteLength-cursor) : carry;
    const {done,value} = await streamReader.read();
    if(done) { if(previous.length) throw Error('The prepared stream ends with an incomplete record'); exhausted=true; return null; }
    const joined = new Uint8Array(previous.length+value.length); joined.set(previous); joined.set(value,previous.length);
    block = new DataView(joined.buffer); cursor = 0;
  }
  const row = Array.from({length:5},(_,i)=>block.getUint32(cursor+i*4,true)); cursor+=20; return row;
}

function emit(state='paused') {
  const snapshot = engine.snapshot(offset);
  postMessage({type:'update',snapshot,history:history.splice(0),feed:feed.slice(-12),state,
    computeMs,recordIndex,limit:config.limit});
  emitted = performance.now();
}

async function advance(amount, single=false) {
  if(!ready || busy) return;
  busy = true; const current = generation;
  try {
    for(let n=0;n<amount && engine.count<config.limit;n++) {
      const record = await nextRecord(); if(!record || current!==generation) break;
      const t = performance.now(); const event = engine.process(record); computeMs+=performance.now()-t;
      recordIndex++; feed.push(event); if(feed.length>12) feed.shift();
      if(engine.count%1000===0) {
        history.push({n:engine.count,exact:engine.seen.size,fm:engine.fm.estimate(),
window:engine.dgim.estimate(),exactWindow:engine.exact.total,fpr:engine.falsePositives/engine.seen.size});
      }
    }
    if(current!==generation) return;
    if(exhausted || engine.count>=config.limit) {
      running=false;
      if(engine.count%1000) history.push({n:engine.count,exact:engine.seen.size,fm:engine.fm.estimate(),
window:engine.dgim.estimate(),exactWindow:engine.exact.total,fpr:engine.falsePositives/engine.seen.size});
      emit('completed'); postMessage({type:'completed'});
    } else if(single || performance.now()-emitted>70 || !running) emit(running?'running':'paused');
  } catch(error) { running=false; postMessage({type:'error',message:error.message}); }
  finally { if(current===generation) busy=false; }
}

async function tick() {
  if(!running || !ready) return;
```

```
77    const interval = rate < 20 ? 1000/rate : 50;
78    const t = performance.now(); await advance(rate < 20 ? 1 : Math.max(1,Math.floor(rate/20)));
79    if(running) timer=setTimeout(tick,Math.max(0,interval-(performance.now()-t)));
80  }
81
82  self.onmessage = async ({data}) => {
83    if(data.type==='init') return init(data.config);
84    if(data.type==='speed') { rate=data.rate; return; }
85    if(data.type==='bits') { offset=data.offset; if(ready) emit(running?'running':'paused'); return; }
86    if(!ready) return;
87    if(data.type==='play') { if(running)return; running=true; tick(); }
88    if(data.type==='pause') { running=false; clearTimeout(timer); if(!busy)emit('paused'); }
89    if(data.type==='step') { running=false; clearTimeout(timer); await advance(1,true); }
90  };
```

web/tests/algorithms.test.mjs

```
import test from 'node:test';
import assert from 'node:assert/strict';
import fs from 'node:fs';
import zlib from 'node:zlib';
import path from 'node:path';
import {fileURLToPath} from 'node:url';
import {Bloom,FM,DGIM,ExactWindow,StreamEngine} from '../dist/algorithms.js';
import {zipFiles,crc32} from '../dist/download.js';
import {Worker} from 'node:worker_threads';

const root=path.resolve(path.dirname(fileURLToPath(import.meta.url)),'../dist/data');
const config={bits:1048576,k:7,h:16,window:10000,seed:42};

test('Bloom queries before inserting and never loses an inserted host',()=>{
  const b=new Bloom(65536,7),words=[1,2,3,4,5,6,7];
  assert.equal(b.checkAndAdd(words).present,false);
  assert.equal(b.checkAndAdd(words).present,true);
  assert.equal(b.setBits,7);
});
test('FM duplicate invariance, empty stream and group arithmetic',()=>{
  const f=new FM(16);assert.equal(f.estimate(),0);
  f.add(new Uint8Array(64).fill(3));assert.equal(f.estimate(),8);
  f.add(new Uint8Array(64).fill(1));assert.equal(f.estimate(),8);
  assert.deepEqual(f.means(),[8,8,8,8]);
});
test('DGIM exhaustively matches its positive-count bound and exact zero case',()=>{
  for(const window of [1,2,3,5,8,16])for(let sequence=0;sequence<512;sequence++){
    const d=new DGIM(window),exact=new ExactWindow(window),history=[];
    for(let i=0;i<9;i++){
      const bit=(sequence>>i)&1;history.push(bit);d.add(bit);exact.add(bit);
      const truth=history.slice(-window).reduce((a,b)=>a+b,0);
      assert.equal(exact.total,truth);assert.ok(Math.abs(d.estimate()-truth)<=truth*.5);
      const sizes=d.buckets.map(b=>b[0]);assert.ok(sizes.every(s=>sizes.filter(v=>v===s).length<=2));
      assert.ok(d.buckets.every(b=>b[1]>d.time-window));
    }
  }
});
test('Invalid configurations are rejected',()=>{
  assert.throws(()=>new Bloom(13,2));assert.throws(()=>new FM(3));assert.throws(()=>new DGIM(0));
  assert.throws(()=>new DGIM().add(2));
});
test('ZIP creation emits standard signatures, Unicode names and a valid CRC',async()=>{
  assert.equal(crc32(new TextEncoder().encode('123456789')),0xcbf43926);
  const zip=zipFiles([{name:'project/real.txt',data:new TextEncoder().encode('NASA')}]);
  const raw=new DataView(await zip.arrayBuffer());
  assert.equal(raw.getUint32(0,true),0x04034b50);
  assert.equal(raw.getUint32(raw.byteLength-22,true),0x06054b50);
});
test('All browser benchmark evidence is complete',()=>{
  const b=JSON.parse(fs.readFileSync(path.join(root,'benchmarks.json')));
  assert.equal(b.parameter_sweep.length,63);assert.equal(b.full_summary.length,3);assert.equal(b.scaling_summary
.length,12);
});
test('Browser summaries reproduce the Python results on the complete real stream',()=>{
  const hash=zlib.gunzipSync(fs.readFileSync(path.join(root,'hashes-42.bin.gz')));
  const buffer=hash.buffer.slice(hash.byteOffset,hash.byteOffset+hash.byteLength);
  const engine=new StreamEngine(config,buffer);
  const records=zlib.gunzipSync(fs.readFileSync(path.join(root,'events.bin.gz')));
  assert.equal(records.length,1891714*20);
  for(let i=0;i<records.length;i+=20){
    const record=Array.from({length:5},(_,j)=>records.readUInt32LE(i+j*4));engine.process(record);
    if(engine.count===10000){assert.equal(engine.seen.size,936);assert.equal(engine.fm.estimate(),2528);assert.e
qual(engine.dgim.estimate(),48);}
    if(engine.count===100000){assert.equal(engine.seen.size,7752);assert.equal(engine.fm.estimate(),25088);}
  }
  const actual=engine.snapshot();assert.equal(actual.count,1891714);assert.equal(actual.unique,81982);
  assert.equal(actual.falsePositives,27);assert.equal(actual.falseNegatives,0);assert.equal(actual.fm,442368);
  assert.equal(actual.window,28);assert.equal(actual.exactWindow,30);assert.equal(actual.errors,10980);
});

test('The actual worker supports init, one step, real playback and one-per-second pacing',async()=>{
  const worker=new Worker(new URL('./worker-harness.mjs',import.meta.url));
  const messages=[],waiters=[];
  worker.on('message',data=>{
    const index=waiters.findIndex(w=>w.predicate(data));
    if(index>=0){const w=waiters.splice(index,1)[0];clearTimeout(w.timer);w.resolve(data);}
    else messages.push(data);
  });
  function next(predicate){
    const index=messages.findIndex(predicate);
    if(index>=0)return Promise.resolve(messages.splice(index,1)[0]);
    return new Promise((resolve,reject)=>{
      const waiter={predicate,resolve,timer:setTimeout(()=>reject(Error('Worker did not respond')),5000)};
```

```
      waiters.push(waiter);
    });
  }
  try{
    worker.postMessage({type:'init',config:{...config,limit:10000,rate:100000}});
    await next(d=>d.type==='ready');worker.postMessage({type:'step'});
    const single=await next(d=>d.type==='update');assert.equal(single.snapshot.count,1);
    assert.equal(single.snapshot.last.elapsed,0);assert.equal(single.snapshot.last.status,200);
    worker.postMessage({type:'play'});
    const done=await next(d=>d.type==='update'&&d.state==='completed');
    assert.equal(done.snapshot.count,10000);assert.equal(done.snapshot.unique,936);
    assert.equal(done.snapshot.fm,2528);assert.equal(done.snapshot.window,48);
    worker.postMessage({type:'init',config:{...config,limit:100000,rate:1}});
    await next(d=>d.type==='ready');messages.length=0;worker.postMessage({type:'play'});
    await new Promise(resolve=>setTimeout(resolve,170));worker.postMessage({type:'pause'});
    const paused=await next(d=>d.type==='update'&&d.state==='paused');assert.equal(paused.snapshot.count,1);
  }finally{await worker.terminate();for(const w of waiters)clearTimeout(w.timer);}
});
```

web/tests/worker-harness.mjs

```
// Node bridge for testing the actual browser worker with real local gzip inputs.
import {parentPort} from 'node:worker_threads';
import fs from 'node:fs';
import path from 'node:path';
import {fileURLToPath} from 'node:url';
const dist=path.resolve(path.dirname(fileURLToPath(import.meta.url)),'../dist');
globalThis.self=globalThis;
globalThis.postMessage=value=>parentPort.postMessage(value);
globalThis.fetch=async url=>{
  const file=path.resolve(dist,String(url));
  if(!file.startsWith(dist+path.sep))throw Error('Invalid fixture path');
  return new Response(fs.readFileSync(file));
};
await import('../dist/worker.js');
parentPort.on('message',data=>self.onmessage({data}));
```

7. References

[1] Internet Traffic Archive. NASA-HTTP: Two Months of HTTP Logs from the KSC-NASA WWW Server. Collected by Jim Dumoulin; contributed by Martin Arlitt and Carey Williamson. https://ita.ee.lbl.gov/html/contrib/NASA-HTTP.html (accessed 23 October 2026).

[2] Bloom, B. H. (1970). Space/time trade-offs in hash coding with allowable errors. Communications of the ACM, 13(7), 422-426. https://doi.org/10.1145/362686.362692.

[3] Leskovec, J., Rajaraman, A., and Ullman, J. D. Mining of Massive Datasets, Chapter 4: Mining Data Streams. Author-hosted materials: https://www.mmds.org/; chapter: https://infolab.stanford.edu/~ullman/mmds/ch4.pdf.

[4] Flajolet, P., and Martin, G. N. (1985). Probabilistic counting algorithms for data base applications. Journal of Computer and System Sciences, 31(2), 182-209. https://doi.org/10.1016/0022-0000(85)90041-8.

[5] Datar, M., Gionis, A., Indyk, P., and Motwani, R. (2002). Maintaining stream statistics over sliding windows. SIAM Journal on Computing, 31(6), 1794-1813. https://doi.org/10.1137/S0097539701398363.

Data provenance and integrity

Original download: https://ita.ee.lbl.gov/traces/NASA_access_log_Jul95.gz

SHA-256 (compressed source file):

```
199109ed0f273e095da6ccd5fc9dc4cd
8bb58daa06d62135e62090fea9d27488
```

The included references.bib provides a reusable bibliography. Results and narrative are specific to this project's measured runs; external sources provide the dataset provenance and algorithm foundations.